Lab 01 - CPU Monitoring (Linux)

Objectives

- Offer an introduction to Performance Monitoring
- Present the main CPU metrics and how to interpret them
- Get you to use various tools for monitoring the performance of the CPU
- Familiarize you with the x86 Hardware Performance Counters

Contents

Tasks

- 01. [20p] Vmstat
- 02. [20p] Mpstat
- 03. [20p] Zip with compression levels
- 04. [40p] Hardware counters
- 05. [10p] Feedback

Introduction

01. Performance Monitoring

Performance monitoring is the process of regularly checking a set of metrics and tracking the overall health of a specific system. Monitoring is tightly coupled with performance tuning, and a Linux system administrator should be proficient in these two subjects, as one of their main responsibilities is to identify bottlenecks and find solutions to help the operating system surpass them. Pinpointing a Linux system bottleneck requires a deep understanding of how various components of this operating system work (e.g. how processes are scheduled on the CPU, how memory is managed , the way that I/O interrupts are handled, the details of network layer implementation, etc). From a high level, the main subsystems that you should think of when tuning are CPU, Memory, I/O and Network.

These four subsystems are vastly depending on each other and tuning the whole system implies keeping them in harmony. To quote a famous idiom, “a chain is no stronger than its weakest link”. Thus, when investigating a system performance issue, all the subsystems must be checked and analysed.

Being able to discover the bottleneck in a system requires also understanding of what types of processes are running on it. The application stack of a system can be broken down in two categories:

- CPU Bound - performance is limited by the CPU
  - Requires heavy use of the CPU (e.g. for batch processing, mathematical operations, etc)
  - e.g. High volume web servers
- I/O Bound - performance is limited by the I/O subsystem
  - Requires heavy use of memory and storage system
  - An I/O Bound application is usually processing large amounts of data
  - An often behaviour is to use CPU resources for making I/O requests and to enter a sleeping state
  - e.g. Database applications

Before going further with the CPU specific metrics and tools, here is a methodical approach which can guide you when tuning the performance of a system:

- Understand the factors which affect the performance
- Create a baseline measurement with the normal performance of the system
- Reproduce the issue and compare the measurements with the baseline to narrow down the bottleneck to a specific subsystem
- Try a single change at a time and test the results

02. Introducing the CPU and CPU Metrics

Before looking at the numerous performance measurement tools present in the Linux operating system, it is important to understand some key concepts and metrics, along with their interpretation regarding the performance of the system.

The kernel contains a scheduler which is in charge of scheduling two types of resources: interrupts and threads. The resources are assigned by the scheduler with different priorities. The following list presents the priorities:

- User Processes - All the processes running in the user space - having the lowest priority in the scheduling mechanism
- System Processes - All kernel processing
- Interrupts - Devices announcing the kernel that they are done processing

Context Switches

While executing a process, the necessary set of data is stored in registers on the processor and cache. This group of information is called a context. Each thread owns an allotted time quantum to spend on the CPU, and when the time finishes or it is preempted by a higher priority task, a new ready to run process will be scheduled. When the next process is scheduled to run, the context of the current will be stored and the context of the new one is restored to the registers, this process being named context switch. Having a great volume of context switching is not desired because the CPU has to flush its register and cache each time, to gain room for the new process, which leads to performance issues.

The Run Queue

Each CPU preserves its own run queue of threads. In an ideal scenario, the scheduler would be constantly executing threads. Threads can be in different states: runnable - processes which are ready to be executed or in a sleep state - being blocked while waiting for I/O. If the system has performance issues or it’s overloaded, then the queue starts to fill up and a process thread will take longer to execute.

The same concept is known also as “load”. This term is measured by load average, which is a rolling average of the sum of the processes waiting to be processed and the processes waiting for uninterruptible task to be completed. Unix systems traditionally present the CPU load as 1-minute, 5-minute and 15-minute averages.

CPU Utilisation

The CPU Utilisation is a meaningful metric to observe how the running processes make use of the given processing resources. You can find the following categories the vast majority of performance monitoring tools:

- User time - the time percentage a CPU spends on user processes
  - High user time values are recommended because this usually means that the system carries out actual work
- System time - the time percentage a CPU spends on kernel threads and interrupts
  - High system time values could mean bottlenecks in the network and driver stack
- Waiting I/O - the time percentage a CPU waits for a I/O event to occur
  - A system should not spend too much time waiting for I/O operations
- Idle time - the time percentage a CPU spends waiting for tasks
- Nice time - the time percentage spends on changing the priority and execution order of processes. It is often included in the user time

03. CPU Performance Monitoring

The Linux distributions have various monitoring tools available. Some of the utilities deal with metrics in a single tool, providing well formatted output which eases the understanding of the system performance. Other tools are specialized on more specific metrics and give us detailed information.

Some of the most important Linux CPU performance monitoring tools:

Tool	Most useful function
vmstat	System activity
top	Process activity
uptime, w	Average system load
ps, pstree	Displays the processes
iostat	Average CPU load
sar	Collect and report system activity
mpstat	Multiprocessor usage

04. Examples

Understanding how well a CPU is performing is a matter of interpreting the run queue, its utilisation, and the amount of context switching performed. Although performance is relative to baseline statistics, in the absence of these statistics, the following general performance expectations of a system can be used as a guideline:

- Run Queues – A run queue should not have more than 3 threads queued per processor. For example, a dual processor system should not have more than 6 threads in the run queue.
- CPU Utilisation – A fully utilised CPU should have the following utilisation distribution:
  - 65% – 70% User Time
  - 30% – 35% System Time
  - 0% – 5% Idle Time
- Context Switches – The amount of context switches is directly relevant to CPU utilisation. As long as the CPU sustains the previously presented utilisation distribution, it is acceptable to have a high amount of context switches.

The following two examples give interpretations of the outputs generated by **vmstat**.

Example A - Sustained CPU Utilisation

```
# vmstat 1
procs                  memory      swap          io     system         cpu
 r  b   swpd   free   buff  cache   si   so    bi    bo   in    cs us sy wa id
 3  0 206564  15092  80336 176080    0    0     0     0  718    26 81 19  0  0
 2  0 206564  14772  80336 176120    0    0     0     0  758    23 96  4  0  0
 1  0 206564  14208  80336 176136    0    0     0     0  820    20 96  4  0  0
 1  0 206956  13884  79180 175964    0  412     0  2680 1008    80 93  7  0  0
 2  0 207348  14448  78800 175576    0  412     0   412  763    70 84 16  0  0
 2  0 207348  15756  78800 175424    0    0     0     0  874    25 89 11  0  0
 1  0 207348  16368  78800 175596    0    0     0     0  940    24 86 14  0  0
 1  0 207348  16600  78800 175604    0    0     0     0  929    27 95  3  0  2
 3  0 207348  16976  78548 175876    0    0     0  2508  969    35 93  7  0  0
 4  0 207348  16216  78548 175704    0    0     0     0  874    36 93  6  0  1
 4  0 207348  16424  78548 175776    0    0     0     0  850    26 77 23  0  0
 2  0 207348  17496  78556 175840    0    0     0     0  736    23 83 17  0  0
 0  0 207348  17680  78556 175868    0    0     0     0  861    21 91  8  0  1
```

The following observations can be made based on this output:

- There are a high amount of interrupts (**in**) and a low amount of context switches (**cs**). It appears that a single process is making requests to hardware devices.
- To further prove the presence of a single application, the user (**us**) time is constantly at 85% and above. Along with the low amount of context switches, we deduce that the process comes on the processor and stays on the processor.
- The run queue is just about at the limits of acceptable performance. On a couple occasions, it goes beyond acceptable limits.

Example B - Overloaded Scheduler

```
# vmstat 1
procs                  memory      swap          io     system         cpu
 r  b   swpd   free   buff  cache   si   so    bi    bo   in    cs us sy wa id
 2  1 207740  98476  81344 180972    0    0  2496     0  900  2883  4 12 57 27
 0  1 207740  96448  83304 180984    0    0  1968   328  810  2559  8  9 83  0
 0  1 207740  94404  85348 180984    0    0  2044     0  829  2879  9  6 78  7
 0  1 207740  92576  87176 180984    0    0  1828     0  689  2088  3  9 78 10
 2  0 207740  91300  88452 180984    0    0  1276     0  565  2182  7  6 83  4
 3  1 207740  90124  89628 180984    0    0  1176     0  551  2219  2  7 91  0
 4  2 207740  89240  90512 180984    0    0   880   520  443   907 22 10 67  0
 5  3 207740  88056  91680 180984    0    0  1168     0  628  1248 12 11 77  0
 4  2 207740  86852  92880 180984    0    0  1200     0  654  1505  6  7 87  0
 6  1 207740  85736  93996 180984    0    0  1116     0  526  1512  5 10 85  0
 0  1 207740  84844  94888 180984    0    0   892     0  438  1556  6  4 90  0
```

The following observations can be made based on this output:

- The amount of context switches is higher than interrupts, suggesting that the kernel has to spend a considerable amount of time context switching threads.
- The high volume of context switches is causing an unhealthy balance of CPU utilisation. This is evident by the fact that the wait on IO percentage is extremely high and the user percentage is extremely low.
- Because the CPU is blocked waiting for I/O, the run queue starts to fill and the amount of threads blocked waiting on I/O also fills.

These examples are from Darren Hoch's Linux System and Performance Monitoring [http://ufsdump.org/papers/oscon2009-linux-monitoring.pdf].

Tasks

01. [20p] Vmstat

The **vmstat** utility provides a good low-overhead view of system performance. Since **vmstat** is such a low-overhead tool, it is practical to have it running even on heavily loaded servers when it is needed to monitor the system's health.

[10p] Task A - Monitoring stress

Run **vmstat** on your machine with a 1 second delay between updates. Notice the CPU utilisation (info about the output columns here [https://medium.com/@damianmyerscough/vmstat-explained-83b3e87493b3]).

In another terminal, use the **stress** command to start N CPU workers, where N is the number of cores on your system. Do not pass the number directly. In stead, use command substitution.

[10p] Task B - How does it work?

Let us look at how **vmstat** works under the hood. We can assume that all these statistics (memory, swap, etc.) can not be normally gathered in userspace. So how does **vmstat** get these values from the kernel? Or rather, how does any process interact with the kernel? Most obvious answer: ***system calls***.

```
$ strace vmstat
```

"All well and good. But what am I looking at?"

What you *should* be looking at are the system calls after the two **write**s that display the output header (hint: it has to do with **/proc/** file system). So, what are these files that **vmstat** opens?

```
$ file /proc/meminfo
$ cat /proc/meminfo

$ man 5 proc
```

The manual should contain enough information about what these kernel interfaces can provide. However, if you are interested in *how* the kernel generates the statistics in **/proc/meminfo** (for example), a good place to start would be meminfo.c [https://elixir.bootlin.com/linux/v4.15/source/fs/proc/meminfo.c] (but first, SO2 wiki [https://ocw.cs.pub.ro/courses/so2]).

NON-DEMO TASK

Write a one-liner that uses **vmstat** to report complete **disk statistics** and sort the output in **descending** order based on **total reads** column. Include both the bash command and the output in your submission.

02. [20p] Mpstat

Open fact_rcrs.zip and look at the code.

[10p] Task A - Python recursion depth

Try to run the script while passing 1000 as a command line argument. Why does it crash?

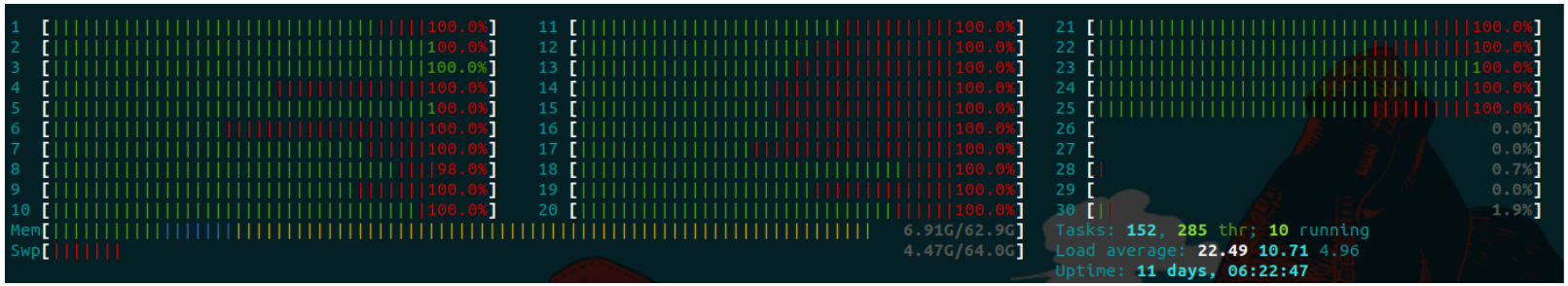
Luckily, python allows you to both retrieve the current recursion limit *and* set a new value for it. Increase the recursion limit so that the process will never crash, regardless of input (assume that it still has a reasonable upper bound).

[10p] Task B - CPU affinity

Run the script again, this time passing 10000. Use **mpstat** to monitor the load on each *individual* CPU at 1s intervals. The one with close to 100% load will be the one running our script. Note that the process might be passed around from one core to another.

Stop the process. Use **stress** to create N-1 CPU workers, where N is the number of cores on your system. Use **taskset** to set the CPU affinity of the N-1 workers and then run the script again. You should notice that the process is scheduled on cpu0.

Note: to get the best performance when running a process, make sure that it stays on the same core for as long as possible. Don't let the scheduler decide this for you, if you can help it. Allowing it to bounce your process between cores can drastically impact the efficient use of the cache and the TLB. This holds especially true when you are working with servers rather than your personal PCs. While the problem may not manifest on a system with only 4 cores, you can't guarantee that it also won't manifest on one with 40 cores. When running several experiments in parallel, aim for something like this:



NON-DEMO TASK

Write a bash command that binds CPU **stress** workers on your odd-numbered cores (i.e.: 1,3,5,…). The list of cores and the number of stress workers must NOT be hardcoded, but constructed based on **nproc** (or whatever else you fancy).
In your submission, include both the bash command and a **mpstat** capture to prove that the command is working.

03. [20p] Zip with compression levels

The **zip** command is used for compression and file packaging under Linux/Unix operating system. It provides 10 levels of compression, where:

- **level 0** : provides no compression, only packaging
- **level 6** : used as default compression level
- **level 9** : provides maximum compression

```
$ zip -5 file.zip file.txt
```

[10p] Task A - Measurements

Write a script to measure the compression rate and the time required for each level. Use the following files:

- two largest bitmaps from here [https://www.fileformat.info/format/bmp/sample/index.htm]
- this large text file here [https://norvig.com/big.txt]

[10p] Task B - Plot

Fill the data you obtained into the **python3** script in plot.zip.
Make sure you have **python3** and **python3-matplotlib** installed.

NON-DEMO TASK

Include the generated plot in your submission.

04. [40p] Hardware counters

A significant portion of the system statistics that can be generated involve hardware counters. As the name implies, these are special registers that count the number of occurrences of specific events in the CPU. These counters are implemented through **Model Specific Registers** (MSR), control registers used by developers for debugging, tracing, monitoring, etc. Since these registers may be subject to changes from one iteration of a microarchitecture to the next, we will need to consult chapters 18 and 19 from Intel 64 and IA-32 Architectures Developer's Manual: Vol. 3B [https://www.intel.com/content/www/us/en/architecture-and-technology/64-ia-32-architectures-software-developer-vol-3b-part-2-manual.html].

The instructions that are used to interact with these counters are RDMSR [https://www.felixcloutier.com/x86/rdmsr], WRMSR [https://www.felixcloutier.com/x86/wrmsr] and RDPMC [https://www.felixcloutier.com/x86/rdpmc]. Normally, these are considered privileged instruction (that can be executed only in ring0, aka. kernel space). As a result, acquiring these information from ring3 (user space) requires a context switch into ring0, which we all know to be a costly operation. The objective of this exercise is to prove that this is not necessarily the case and that it is possible to configure and examine these counters from ring3 in as few as a couple of clock cycles.

Before getting started, one thing to note is that there are two types of performance counters:

1. Fixed Function Counters
    - each can monitor a single, distinct and predetermined event (burned in hardware)
    - are configured a bit differently than the other type
    - are not of interest to us in this laboratory
2. General Purpose Counters
    - can be configured to monitor a specific event from a list of over 200 (see chapters 19.1 and 19.2)

Download hw_counter.zip.

Here is an overview of the following five tasks:

- **Task A**: check the version ID of your CPU to determine what it's capable of monitoring.
- **Task B**: set a certain bit in CR4 to enable ring3 usage of the RDPMC instruction.
- **Task C**: use some ring3 tools to enable the hardware counters.
- **Task D**: start counting L2 cache misses.

- **Task E**: use RDPMC to measure the cache misses for a familiar program.

[5p] Task A - Hardware Info

First of all, we need to know what we are working with. Namely, the microarchitecture *version ID* and the *number of counters* per core. To this end, we will use cpuid [https://linux.die.net/man/1/cpuid] (basically a wrapper over the CPUID [https://www.felixcloutier.com/x86/cpuid] instruction.) All the information that we need will be contained in the 0AH leaf (might want to get the raw output of **cpuid**):

- **CPUID.0AH:EAX[15:8]** : number of general purpose counters
- **CPUID.0AH:EAX[7:0]** : version ID
- **CPUID.0AH:EDX[7:0]** : number of fixed function counters

Note: the first two columns of the output represent the EAX and ECX registers used when calling CPUID. If the most significant bit in EAX is 1 (i.e.: starts with 0x8) the output is for extended options. ECX is a relatively new addition. So when looking for the 0AH leaf, search for a line starting with **0x0000000a**. The register contents following ':' represent the output of the instruction.

Point out to your assistant which is which in the **cpuid** output.

[5p] Task B - Unlock RDPMC in ring3

This is pretty straightforward. All you need to do is set the **Performance-Monitor Counter Enable** bit in CR4 [https://en.wikipedia.org/wiki/Control_register#CR4]. Naturally, this can't be done from ring3. As such, we provide a kernel module that does it for you (see *hack_cr4.c*.) When the module is loaded, it will set the aforementioned bit. Similarly, when the module is unloaded, it will revert the change. Try compiling the module, loading and unloading it and finally, check the kernel message log to verify that it works.

```
$ make
$ sudo insmod hack_cr4.ko
$ sudo rmmod hack_cr4
$ dmesg
```

Note: the module must remain loaded in the kernel in order to keep the bit set. If during Task E you get a segfault, the reason is that you (probably) unloaded the module and you no longer have permission to run the instruction in ring3. This does NOT invalidate your work in Tasks C and D; simply load the module once more.

[10p] Task C - Configure IA32_PERF_GLOBAL_CTRL

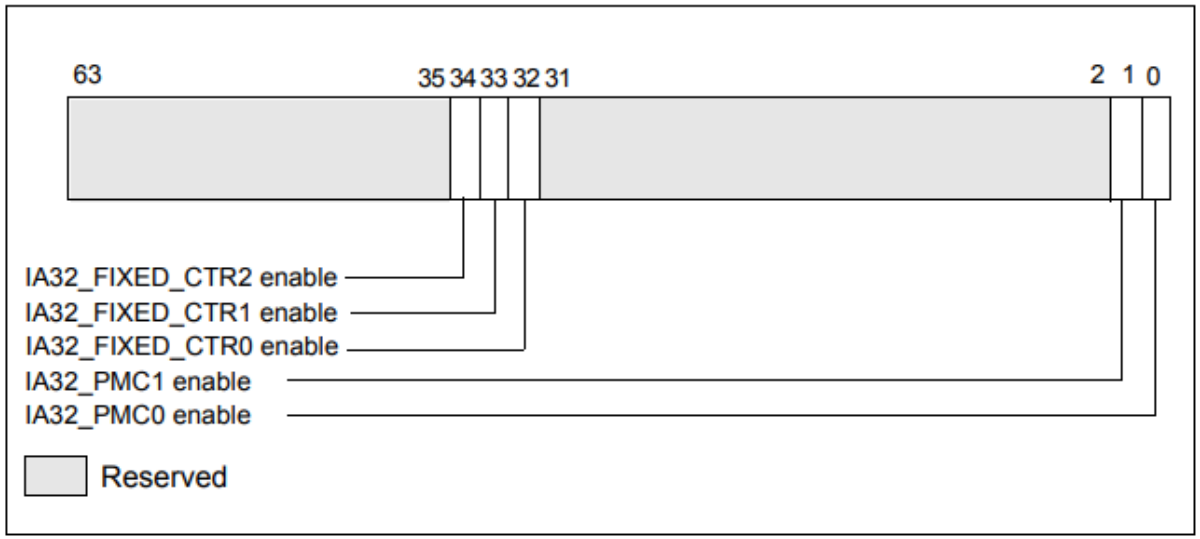


Figure 18-3. Layout of IA32_PERF_GLOBAL_CTRL MSR

The **IA32_PERF_GLOBAL_CTRL** (0x38f) MSR is an addition from *version 2* that allows enabling / disabling multiple counters with a single WRMSR instruction. What happens, in layman terms, is that the CPU performs an AND between each EANBLE bit in this register and its counterpart in the counter's original configuration register from *version 1* (which we will deal with in the next task.) If the result is 1, the counter begins to register the programmed event every clock cycle. Normally, all these bits should be set by default during the booting process but it never hurts to check. Also, note that this register exists for each logical core.

If for CR4 we had to write a kernel module, for MSRs we have user space tools that take care of this for us (rdmsr [http://manpages.ubuntu.com/manpages/trusty/man1/rdmsr.1.html] and wrmsr [http://manpages.ubuntu.com/manpages/trusty/man1/wrmsr.1.html]) by interacting with a driver called **msr** (install **msr-tools** if it's missing from your system.) But first, we must load this driver.

```
$ lsmod | grep msr
$ sudo modprobe msr
$ lsmod | grep msr
    msr                    16384  0
```

Next, let us read the value in the IA32_PERF_GLOBAL_CTRL register. If the result differs from what you see in the snippet below, overwrite the value (the **-a** flag specifies that we want the command to run on each individual logical core).

```
$ sudo rdmsr -a 0x38f
    70000000f
$ sudo wrmsr -a 0x38f 0x70000000f
```

[10p] Task D - Configure IA32_PERFEVENTSELx

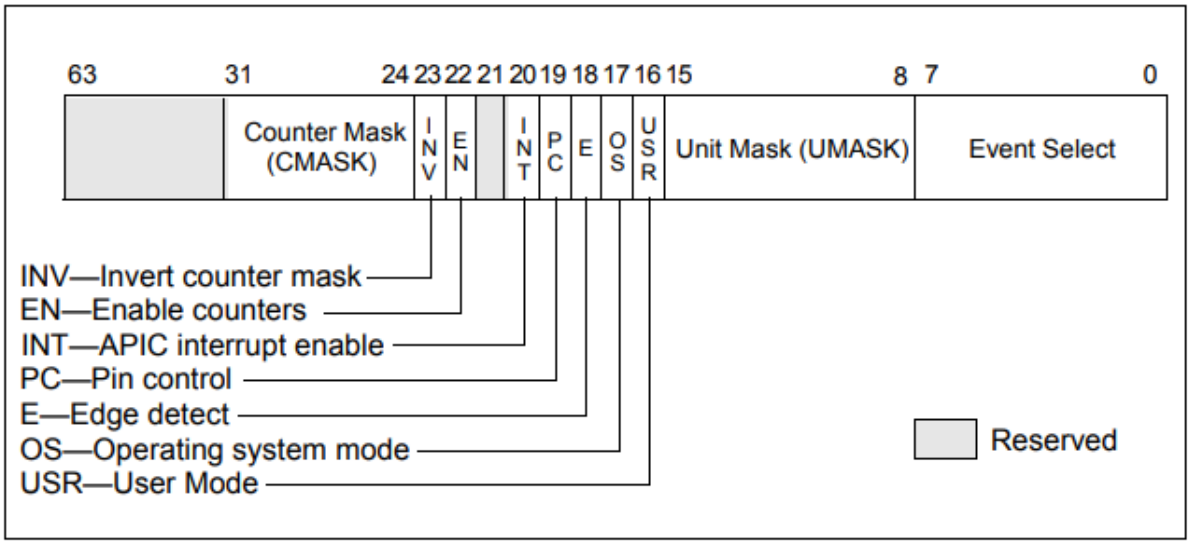


Figure 18-1. Layout of IA32_PERFEVTSELx MSRs

The **IA32_PERFEVENTSELx** are MSRs from *version 1* that are used to configure the monitored event of a certain counter, its enabled state and a few other things. We will not go into detail and instead only mention the fields that interest us right now (you can read about the rest in the Intel manual.) Note that the *x* in the MSR's name stands for the counter number. If we have 4 counters, it takes values in the 0:3 range. The one that we will configure is IA32_PERFEVENTSEL0 (0x186). If you want to configure more than one counter, note that they have consecutive register number (i.e. 0x187, 0x188, etc.).

As for the register flags, those that are not mentioned in the following list should be left cleared:

- **EN** (enable flag) = **1** starts the counter
- **USR** (user mode flag) = **1** monitors only ring3 events
- **UMASK** (unit mask) = **??** depends on the monitored event (see chapter 19.2)
- **EVSEL** (event select) = **??** depends on the monitored event (see chapter 19.2)

Before actually writing in this register, we should verify that no one is currently using it. If this is indeed the case, we might also want to clear **IA32_PMC0** (0xc1). PMC0 is the actual counter that is associated to PERFEVENTSEL0.

```
$ sudo rdmsr -a 0x186
    0
$ sudo wrmsr -a 0xc1 0x00
$ sudo wrmsr -a 0x186 0x41????
```

For the next (and *final* task) we are going to monitor the number of L2 cache misses. Look for the **L2_RQSTS.MISS** event in table 19-3 or 19-11 (depending on CPU version id) in the Intel manual and set the last two bytes (the unit mask and event select) accordingly. If the operation is successful and the counters have started, you should start seeing non-zero values in the PMC0 register, increasing in subsequent reads.

[10p] Task E - Ring3 cache performance evaluation

As of now, we should be able to modify the **CR4** register with the kernel module, enable all counters in the **IA32_PERF_GLOBAL_CTRL** across all cores and start an **L2 cache miss** counter again, across all cores. What remains is putting everything into practice.

Take *mat_mul.c*. This program may be familiar from an ASC laboratory but, in case it isn't, the gist of it is that when using the naive matrix multiplication algorithm (O(n^3)), the frequency with which each iterator varies can wildly affect the performance of the program. The reason behind this is (in)efficient use of the CPU cache. Take a look at the following snippet from the source and keep in mind that each matrix buffer is a continuous area in memory.

```
for (uint32_t i=0; i<N; ++i)            /* line   */
    for (uint32_t j=0; j<N; ++j)        /* column */
        for (uint32_t k=0; k<N; ++k)
            r[i*N + j] += m1[i*N + k] * m2[k*N + j];
```

What is the problem here? The problem is that i and k are multiplied with a large number N when updating a certain element. Thus, fast variations in these two indices will cause huge strides in accessed memory areas (larger than a cache line) and will cause unnecessary cache misses. So what are the best and worst configurations for the three fors? The best: i, k j. The worst: j, k, i. As we can see, the configurations that we will monitor in *mat_mul.c* do not coincide with the aforementioned two (so… not great, not terrible.) Even so, the difference in execution time and number of cache misses will still be significant.

Which brings us to the task at hand: using the **RDPMC** instruction, calculate the number of L2 cache misses for each of the two multiplications without performing any context switches (hint: look at gcc extended asm [https://gcc.gnu.org/onlinedocs/gcc/Extended-Asm.html] and the following macro from *mat_mul.c*).

```
#define rdpmc(ecx, eax, edx)    \
    asm volatile (              \
        "rdpmc"                 \
        : "=a"(eax),            \
          "=d"(edx)             \
        : "c"(ecx))
```

A word of caution: remember that each logical core has its own PMC0 counter, so make sure to use taskset [https://linux.die.net/man/1/taskset] in order to set the CPU affinity of the process. If you don't the process may be passed around different cores and the counter value becomes unreliable.

```
$ taskset 0x01 ./mat_mul 1024
```

NON-DEMO TASK

Include a screenshot of the last task's output (containing run time *and* cache misses for both multiplications).

05. [10p] Feedback

Please take a minute to fill in the **feedback form [https://forms.gle/k7FqUM16AcrkgKTJA]** for this lab.

NON-DEMO TASK

Include a screenshot with the successful submission response from the server (not your actual feedback).

ep/labs/01.txt · Last modified: 2020/08/18 18:27 by radu.mantu

Lab 02 - Memory Monitoring (Linux)

Objectives

- Offer an introduction to Virtual Memory.
- Get you acquainted with relevant commands and their outputs for monitoring memory related aspects.
- Introduce the concept of page de-duplication.
- Present a step-by-step guide to Intel PIN for dynamic instrumentation.

Contents

Tasks

- 01. [10p] Memory usage
- 02. [20p] Swap space
- 03. [30p] Kernel Samepage Merging
- 04. [40p] Intel PIN
- 05. [10p] Feedback

Introduction

01. Virtual Memory

Virtual memory uses a disk as an extension of RAM so that the effective size of usable memory grows correspondingly. The kernel will write the contents of a currently unused block of memory to the hard disk so that the memory can be used for another purpose. When the original contents are needed again, they are read back into memory. This is all made completely transparent to the user; programs running under Linux only see the larger amount of memory available and don't notice that parts of them reside on the disk from time to time. Of course, reading and writing the hard disk is slower (on the order of a thousand times slower) than using real memory, so the programs don't run as fast. The part of the hard disk that is used as virtual memory is called the swap space.

02. Virtual Memory Pages

Virtual memory is divided into pages. Each virtual memory page on the X86 architecture is 4KB. When the kernel writes memory to and from disk, it writes memory in pages. The kernel writes memory pages to both the swap device and the file system.

03. Kernel Memory Paging

Memory paging is a normal activity not to be confused with memory swapping. Memory paging is the process of syncing memory back to disk at normal intervals. Over time, applications will grow to consume all of memory. At some point, the kernel must scan memory and reclaim unused pages to be allocated to other applications.

04. The Page Frame Reclaim Algorithm (PFRA)

The PFRA is responsible for freeing memory. The PFRA selects which memory pages to free by page type. Page types are listed below:

- **Unreclaimable** – locked, kernel, reserved pages
- **Swappable** – anonymous memory pages
- **Syncable** – pages backed by a disk file
- **Discardable** – static pages, discarded pages

All but the "unreclaimable" pages may be reclaimed by the PFRA. There are two main functions in the PFRA. These include the kswapd kernel thread and the "Low On Memory Reclaiming" function.

05. Kswapd

The **kswapd** daemon is responsible for ensuring that memory stays free. It monitors the **pages_high** and **pages_low** watermarks in the kernel. If the amount of free memory is below **pages_low**, the **kswapd** process starts a scan to attempt to free 32 pages at a time. It repeats this process until the amount of free memory is above the **pages_high** watermark.

The **kswapd** thread performs the following actions:

- If the page is unmodified, it places the page on the free list.
- If the page is modified and backed by a file system, it writes the contents of the page to disk.
- If the page is modified and not backed up by any file system (anonymous), it writes the contents of the page to the swap device.

06. Kernel Paging with pdflush

- The **pdflush** daemon is responsible for synchronizing any pages associated with a file on a filesystem back to disk. In other words, when a file is modified in memory, the **pdflush** daemon writes it back to disk.
- The **pdflush** daemon starts synchronizing dirty pages back to the filesystem when 10% of the pages in memory are dirty. This is due to a kernel tuning parameter called **vm.dirty_background_ratio**.
- The **pdflush** daemon works independently of the PFRA under most circumstances. When the kernel invokes the LMR (Low on Memory Reclaiming) algorithm, the LMR specifically forces **pdflush** to flush dirty pages in addition to other page freeing routines.
- The **vmstat** utility reports on virtual memory usage in addition to CPU usage. The following fields in the **vmstat** output are relevant to virtual memory: **Swapd**, **Free**, **Buff**, **Cache**, **So**, **Si**, **Bo**, **Bi** (use *man vmstat* to read their description).

The following **vmstat** output demonstrates heavy utilization of virtual memory during an I/O application spike. The following observations can be made based on this output:

- A large amount of disk blocks are paged in (*bi*) from the filesystem. This is evident in the fact that the cache of data in process address spaces (*cache*) grows.
- During this period, the amount of free memory (*free*) remains steady at 17MB even though data is paging in from the disk to consume free RAM.
- To maintain the free list, **kswapd** steals memory from the read/write buffers (*buff*) and assigns it to the free list. This is evident in the gradual decrease of the buffer cache (buff).
- The **kswapd** process then writes dirty pages to the swap device (*so*). This is evident in the fact that the amount of virtual memory utilized gradually increases (*swpd*).

Conclusions:

- The less major page faults on a system, the better response times achieved as the system is leveraging memory caches over disk caches.
- Low amounts of free memory are a good sign that caches are effectively used unless there are sustained writes to the swap device and disk.
- If a system reports any sustained activity on the swap device, it means there is a memory shortage on the system.

Tasks

01. [10p] Memory usage

Open ex01.py and take a look at the code. What is the difference between the methods *do_append()* and *do_allocate()*?

Use *vmstat* to monitor the memory usage while performing the following experiments. In the main method, call:

- The *do_append* method on it's own (see *Experiment 1*).
- The *do_allocate* method on it's own (see *Experiment 2*).
- Both methods as shown in the *Experiment 3* area in the code.
- Both methods as shown in the *Experiment 4* area in the code.

Offer an interpretation for the obtained results.

NON-DEMO TASK

Explain in a paragraph how Generational Garbage Collection works.
In a python shell, get the generational thresholds and the current number of live references in each generation.
Copy+paste the code and the resulting values or add a screenshot.

02. [20p] Swap space

Before starting this task, call the assistant to show him your progress. If you manage to freeze your PC, it might prove tricky to do so afterwards.

[10p] Task A - Swap File

First, let us check what swap devices we have enabled. Check the *NAME* and *SIZE* columns of the following command:

```
$ swapon --show
```

No output means that there are no swap devices available.

If you ever installed a Linux distro, you may remember creating a separate *swap partition*. This, however, is only one method of creating swap space. The other is by adding a *swap file*. Run the following commands:

```
$ sudo swapoff -a
$ sudo dd if=/dev/zero of=/swapfile bs=1024 count=$((4 * 1024 * 1024))
$ sudo chmod 600 /swapfile
$ sudo mkswap /swapfile
$ sudo swapon /swapfile

$ swapon --show
```

Just to clarify what we did:

- disabled all swap devices
- created a 4Gb zero-initialized file
- set the permission to the file so only *root* can edit it
- created a swap area from the file using **mkswap** (works on devices too)
- activated the swap area

The new swap area is temporary and will not survive a reboot. To make it permanent, we need to register it in /etc/fstab [https://en.wikipedia.org/wiki/Fstab] by adding a line such as this:

```
/swapfile swap swap defaults 0 0
```

Now that we created a *swap file*, what are the advantages / disadvantages when compared to a *swap partition*?

- advantages:
  - easier to manage
  - similar performance
- disadvantage:
  - can be affected by disk fragmentation (not the case for a partition)

[10p] Task B - Does it work?

In one terminal run **vmstat** and look at the *swpd* and *free* columns.

```
$ vmstat -w 1
```

In another terminal, open a python shell and allocate a bit more memory than the available RAM. Identify the moment when the newly created swap space is being used.

One thing you might notice is that the value in **vmstat**'s *free* column is lower than before. This does not mean that you have less available RAM after creating the swap file. Remember using the **dd** command to create a 4GB file? A big chunk of RAM was used to buffer the data that was written to disk. If *free* drops to unacceptable levels, the kernel will make sure to reclaim some of this buffer/cache memory. To get a clear view of how much available memory you actually have, try running the following command:

```
$ free -h
```

Observe that once you close the python shell and the memory is freed, *swpd* still displays a non-zero value. Why? There simply isn't a reason to clear the data from the swap area. If you really want to clean up the used swap space, try the following:

```
$ vmstat
$ sudo swapoff -a && sudo swapon -a
$ vmstat
```

NON-DEMO TASK

Create two swap files. Set their priorities to 10 and 20, respectively.
Include the commands (copy+paste) or a screenshot of the terminal.

03. [30p] Kernel Samepage Merging

KSM [https://www.kernel.org/doc/html/latest/admin-guide/mm/ksm.html] is a page de-duplication strategy introduced in kernel version 2.6.32. In case you are wondering, it's not the same thing as the file page cache. KSM was originally developed in tandem with KVM in order to detect data pages with *exactly* the same content and make their page table entries point to the same physical address (marked Copy-On-Write.) The end goal was to allow more VMs to run on the same host. Since each page must be scanned for identical content, this solution had no chance of scaling well with the available quantity of RAM. So, the developers compromised to scan only with the private anonymous pages that were marked as likely candidates via `madvise(addr, length, MADV_MERGEABLE)`.

Download the skeleton for this task.

[15p] Task A - Check kernel support & enable ksmd

First things first, you need to verify that KSM was enabled during your kernel's compilation. For this, you need to check the Linux make config build file that is stored on your /boot partition. Hopefully, you should see something like this:

```
$ grep CONFIG_KSM /boot/config-$(uname -r)
CONFIG_KSM=y
```

If you don't have KSM enabled, you *could* recompile the kernel with the CONFIG_KSM flag and try it, but you don't have to :)

Moving forward. Next thing on the list is to check that the **ksmd** daemon is functioning. Any configuration that we'll do will be through the sysfs files in `/sys/kernel/mm/ksm`. Consequently, you should change user to root (even `sudo` should not allow you to write to these files.)

- **/.../run** : this is **1** if the daemon is active; write 1 to it if it's not
- **/.../pages_to_scan** : this is how many pages will be scanned before going to sleep; you can increase this to 1000 if you want to see faster results
- **/.../sleep_millisecs** : this is how many ms the daemon sleeps in between scans; since you've modified **pages_to_scan**, you can leave this be
- **/.../max_page_sharing** : this is the maximum number of pages that can be de-duplicated; in cases like this it's better to go big or go home; so set it to something like 1000000, just to be sure

There are a few more files in the ksm/ directory. We will still use one or two later on. But for now, configuring the previous ones should be enough. Google the rest if you're interested.

[15p] Task B - Watch the magic happen

For this step it would be better to have a few terminals open. First, let's start a `vmstat`. Keep your eyes on the active memory column when we run the sample program.

```
$ vmstat -wa -S m 1
```

Next would be a good time to introduce two more files from the ksm/ sysfs directory:

- **/.../pages_shared** : this file reports how many *physical* pages are in use at the moment
- **/.../pages_sharing** : this file reports how many *virtual* page table entries point to the aforementioned physical pages

For this experiment we will also want to monitor the number of de-duplicated virtual pages, so have at it:

```
$ watch -n 0 cat /sys/kernel/mm/ksm/pages_sharing
```

Finally, look at the provided code, compile it, and launch the program. As an argument you will need to provide the number of pages that will be allocated and initialized with the same value. Note that not all pages will be de-duplicated instantly. So keep in mind your system's RAM limitations before deciding how much you can spare (1-2GB should be ok, right?)

The result should look something like this:

```
mants@victim ~
  ↳ vmstat -wa -S m 1
procs -----------------------memory---------------------- ---swap-- -----io---- -system-- --------cpu--------
 r  b         swpd         free        inact       active   si   so    bi    bo   in   cs  us  sy  id  wa  st
 3  0            1         4626         6580        12436    0    0    42   140   75  202  33  10  56   0   0
 0  0            1         4627         6579        12436    0    0     0     0 2406 8396  10   4  86   0   0
 1  0            1         4628         6579        12436    0    0     0  5352 2431 4985   7   3  83   7   0
 2  0            1         3483         6579        13578    0    0     0     0 4657 4657  11   8  81   0   0
 0  0            1         2499         6579        14560    0    0     0     0 4963 4645   5   6  89   0   0
 2  0            1         2516         6579        14543    0    0     0     0 2743 10127   9   4  87   0   0
 1  0            1         2535         6579        14523    0    0     0     0 2030 5626   8   4  89   0   0
 0  0            1         2552         6579        14507    0    0     0    88 1923 5790   5   3  91   0   0
 0  0            1         2572         6579        14487    0    0     0     0 1812 4611   6   4  90   0   0
 1  0            1         2588         6579        14470    0    0     0     0 1462 4191   4   3  93   0   0
 0  0            1         2608         6579        14450    0    0     0     0 1713 4451   7   3  90   0   0
 0  0            1         2624         6579        14433    0    0     0     0 1393 4133   4   3  93   0   0
 0  0            1         2644         6579        14413    0    0     0    24 1746 4570   7   3  89   0   0
 0  0            1         2660         6579        14396    0    0     0     0 1392 4176   4   2  94   0   0
 0  0            1         4625         6579        12437    0    0     0     0 1852 4727   7   4  88   0   0
 0  0            1         4625         6579        12437    0    0     0     0 1392 4123   4   1  95   0   0
 0  0            1         4626         6579        12437    0    0     0    52 1713 4620   7   2  90   0   0
 1  0            1         4630         6579        12434    0    0     0     0 1735 5283   5   2  92   0   0
```

Here, we can see the active memory suddenly rising when we start the process. Over the next few seconds, as **ksmd** starts scanning pages, the active memory slowly drops. Finally, as the process terminates, all memory is reclaimed by the kernel and the active memory returns to roughly the same value as before.

If you ever want to make use of this in your own experiments, remember to adjust the configurations of **ksmd**. Waking too often or scanning to many pages at once could end up doing more harm than good. See what works for your particular system.

NON-DEMO TASK

Include a screenshot with the same output as the one in the spoiler above.
Edit the screenshot or note in writing at what point you started the application, where it reached max memory usage, the interval where KSM daemon was doing its job (in the 10s sleep interval) and where the process died.

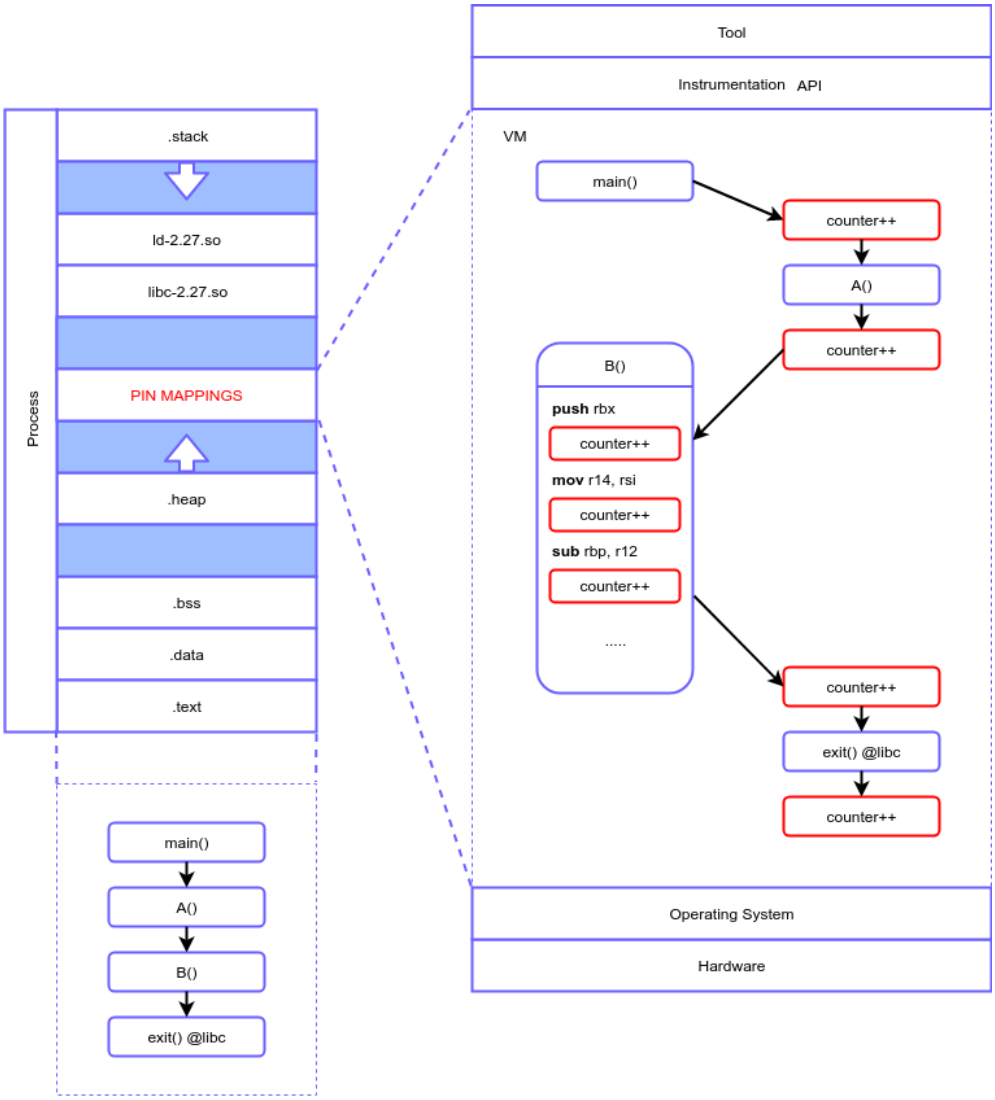
04. [40p] Intel PIN

Broadly speaking, binary analysis is of two types:

- **Static analysis** - used in an offline environment to understand how a program works without actually running it.
- **Dynamic analysis** - applied to a running process in order to highlight interesting behavior, bugs or *performance issues*.

In case you are still wondering, in this exercise we are going to look at (one of) the best dynamic analysis tools available: **Intel Pin**. Specifically, what Pin does is called program instrumentation, meaning that it inserts user-defined code at arbitrary locations in the executable. The code is inserted at runtime, meaning that Pin can attach itself to a process, just like **gdb**.

Although Pin is closed source, the concepts that serve as its fundament are described in this paper [https://web.stanford.edu/class/cs343/resources/pin.pdf]. Since we don't have time to scan through the whole material, we will offer a bird's eye view of its architecture. Just enough to get you started with the tasks.



When a process is started via Pin, the very first instruction is intercepted and new mappings are created in the virtual space of the process. These mappings contain *libraries* that Pin uses, the *tool* that the user wrote (which is compiled as a shared object) and a small *sandbox* that will act as a VM. During the execution, Pin will translate the original instructions into the sandbox on an as-needed basis and, according to the rules defined in the tool, insert arbitrary code. This code can be inserted at different levels of granularity:

- instruction
- basic block
- function
- image

The immediate advantages should be clear. Only from a performance evaluation standpoint, a few applications could be:

- obtaining metrics from programs that were not designed with this in mind
- hotpatching bugs without stopping the process
- detecting the most accessed code regions to prioritize manual optimization

Although this sounds great, we should not ignore some of the glaring disadvantages:

- overhead
  - this is highly dependent on the amount of instrumentation and the instrumented code itself
  - overall, this seems to have a bit more impact on ARM than on other architectures
- volatile
  - remember that the instrumented code shares things like the virtual memory space and file descriptors with the original process
  - while something like in-memory fuzzing is possible, the risk of breaking the process is very high
- limited use cases
  - Pin works directly on a regular executable (with native bytecode)
  - Pin will not work (as intended) on interpreted languages and variations of these

In case you are wondering what else you can do with **Intel Pin**, check out TaintInduce [https://www.comp.nus.edu.sg/~prateeks/papers/TaintInduce.pdf]. The authors of this paper wrote an architecture agnostic taint analysis tool that successfully found 24 CVEs, 17 missing or wrongly emulated instructions in unicorn [https://www.unicorn-engine.org/] and 1 mistake in the Intel Developer Manual.

For reference, use the Pin 3.13 User Guide [https://software.intel.com/sites/landingpage/pintool/docs/98189/Pin/html/] (which also contains examples) and the Pin API Documentation [https://software.intel.com/sites/landingpage/pintool/docs/81205/Pin/html/group__API__REF.html].

[5p] Task A - Setup

In this tutorial we will build a Pin tool with the goal of instrumenting any memory reads/writes. For reads, we output the source buffer state before the operation takes place. For writes, we output the destination buffer states both before and after.

Download the skeleton for this task. First thing you will need to do is run *setup.sh*. This will download the Intel Pin framework into the newly created *third_party/* directory.

Next, open *src/minspect.cpp* in an editor of your choice, but avoid modifying the code. In between tasks, we will apply diff patches to this file. This will allow us to gradually build our tool and observe its behavior at different stages during its development. However, altering the source in any significant manner may cause the patch to fail.

Let us apply the first patch before proceeding to the following task:

```
$ patch src/minspect.cpp patches/Task-A.patch
```

[10p] Task B - Instrumentation Callbacks

Looking at *main()*, most Pin API calls are self explanatory. The only one that we're interested in is the following:

```
INS_AddInstrumentFunction(ins_instrum, NULL);
```

This call instructs Pin to trap on each instruction in the binary and invoke *ins_instrum()*. However, this happens only *once* per instruction. The role of the instrumentation callback that we register is to decide if a certain instruction is of interest to us. "Of interest" can mean basically anything. We can pick and choose "interesting" instructions based on their class, registers / memory operands, functions or objects containing them, etc.

Let's say that an instruction has indeed passed our selection. Now, we can use another Pin API call to insert an *analysis routine* before or after said instruction. While the instrumentation routine will never be invoked again for that specific instruction, the analysis routine will execute seamlessly for each pass.

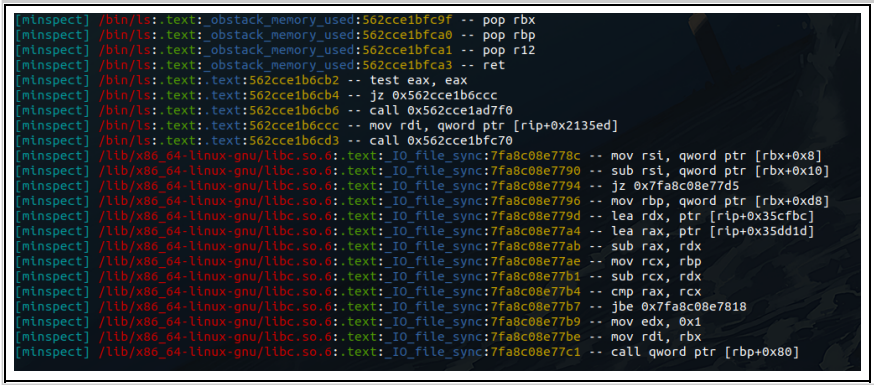
For now, let us observe only the instrumentation callback and leave the analysis routine registration for the following task. Take a look at *ins_instrum()*. Then, compile the tool and run any program you want with it. Waiting for it to finish is not really necessary. Stop it after a few seconds.

```
$ make
$ ./third_party/pin-3.13/pin -t obj-intel64/minspect.so -- ls -l 1>/dev/null
```

Just to make sure everything is clear: the default rule for *make* will generate an *obj-intel64/* directory and compile the tool as a shared object. The way to start a process with our tool's instrumentation is by calling the *pin* util. **-t** specifies the tool to be used. Everything after **--** should be the exact command that would normally be used to start the target process.

Note: here, we output information to stderr from our instrumentation callback. This is not good practice. The Pin tool and the target process share pretty much everything: file descriptors, virtual memory, etc. Normally, you will want to output these things to a log file. However, let's say we can get away with it for now, under the pretext of convenience.

Remember to apply the *Task-B.patch* before proceeding to the next task.



[10p] Task C - Analysis Callbacks (Read)

Going forward, we got rid of some of the clutter in *ins_instrum()*. As you may have noticed, the most recent addition to this routine is the *for* iterating over the memory operands of the instruction. We check whether each operand is the source of a read using INS_MemoryOperandIsRead() [https://software.intel.com/sites/landingpage/pintool/docs/81205/Pin/html/group__INS__BASIC__API__GEN__IA32.html#ga3fdb434cd56a5b72be15dd0931a2b19c]. If this check succeeds, we insert an *analysis routine* before the current instruction using INS_InsertPredicatedCall() [https://software.intel.com/sites/landingpage/pintool/docs/81205/Pin/html/group__INS__INST__API.html#ga26d02bff719bf8600421895956804252]. Let's take a closer look at how this API call works:

```
INS_InsertPredicatedCall(
    ins, IPOINT_BEFORE, (AFUNPTR) read_analysis,
    IARG_ADDRINT,       ins_addr,
    IARG_PTR,           strdup(ins_disass.c_str()),
    IARG_MEMOROP_EA,    op_idx,
    IARG_MEMORYREAD_SIZE,
    IARG_END);
```

The first three parameters are:

- `ins`: reference to the INS argument passed to the instrumentation callback by default.
- `IPOINT_BEFORE`: instructs to insert the analysis routine *before* the instruction executes (see Instrumentation arguments [https://software.intel.com/sites/landingpage/pintool/docs/97503/Pin/html/group__INST__ARGS.html] for more details.)
- `read_analysis`: the function that is to be inserted as the analysis routine.

Next, we pass the arguments for *read_analysis()*. Each argument is represented by a type macro and the actual value. When we don't have any more parameters to send, we end by specifying **IARG_END**. Here are all the arguments:

- `IARG_ADDRINT, ins_addr`: a 64-bit integer containing the absolute address of the instruction.
- `IARG_PTR, strdup(ins_disass.c_str())`: all objects in the callback's local context will be lost after we return; thus, we need to duplicate the disassembled code's string and pass a pointer to the copy.
- `IARG_MEMOROP_EA, op_idx`: effective address of a specific memory operand; so this argument is not passed by value, but in stead recalculated each time and passed to the analysis routine seamlessly.

- `IARG_MEMORYREAD_SIZE`: size in bytes of the memory read; check the documentation for some important exceptions.

Take a look at what *read_analysis()* does. Recompile the tool and run it again (just as in task B). Finally, apply *Task-C.patch* and move on to the next task.

```
[minspect] READ  | 0x00007f0cb70e6849: mov rax, qword ptr [rsp+0x28]          -- [ 00 8f 52 b7 14 a9 24 03 ]
[minspect] READ  | 0x00007f0cb70e684e: xor rax, qword ptr fs:[0x28]           -- [ 00 8f 52 b7 14 a9 24 03 ]
[minspect] READ  | 0x00007f0cb70e6861: pop rbx                               -- [ 80 56 44 b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70e6862: pop rbp                               -- [ 80 56 44 b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70e6863: ret                                   -- [ 7d 73 0d b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70d737d: mov eax, dword ptr [rbx]              -- [ 06 20 ad fb ]
[minspect] READ  | 0x00007f0cb70d7384: mov rdx, qword ptr [rbx+0x88]         -- [ b0 68 44 b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70d738b: mov r8, qword ptr fs:[0x10]           -- [ 00 d6 9a b6 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70d7394: cmp qword ptr [rdx+0x8], r8           -- [ 00 00 00 00 00 00 00 00 ]
[minspect] READ  | 0x00007f0cb70d73a1: cmp dword ptr [rip+0x372550], 0x0     -- [ 00 00 00 00 ]
[minspect] READ  | 0x00007f0cb70d73b2: cmpxchg dword ptr [rdx], esi          -- [ 00 00 00 00 ]
[minspect] READ  | 0x00007f0cb70d73cd: mov rdx, qword ptr [rbx+0x88]         -- [ b0 68 44 b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70d73d4: mov eax, dword ptr [rbx]              -- [ 06 20 ad fb ]
[minspect] READ  | 0x00007f0cb70d73da: add dword ptr [rdx+0x4], 0x1          -- [ 00 00 00 00 ]
[minspect] READ  | 0x00007f0cb70e5204: mov ebp, dword ptr [rdi+0x70]         -- [ 02 00 00 00 ]
[minspect] READ  | 0x00007f0cb70e5210: mov eax, dword ptr [rdi]              -- [ 06 20 ad fb ]
[minspect] READ  | 0x00007f0cb70e8c50: cmp qword ptr [rdi+0x60], 0x0         -- [ 00 00 00 00 00 00 00 00 ]
[minspect] READ  | 0x00007f0cb70e8c5f: mov rax, qword ptr [rdi+0x48]         -- [ 00 00 00 00 00 00 00 00 ]
[minspect] READ  | 0x00007f0cb70e8cb0: ret                                   -- [ 32 52 0e b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70e5232: test byte ptr [rbx+0x74], 0x20        -- [ 00 ]
[minspect] READ  | 0x00007f0cb70e5238: mov rbp, qword ptr [rbx+0xd8]         -- [ a0 12 44 b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70e5262: call qword ptr [rbp+0x88]             -- [ b0 38 0e b7 0c 7f 00 00 ]
[minspect] READ  | 0x00007f0cb70e38b0: mov edi, dword ptr [rdi+0x70]         -- [ 02 00 00 00 ]
```

[10p] Task D - Analysis Callbacks (Write)

For the memory write analysis routine, we need to add instrumentation both before and after each instruction. The former needs to save the original buffer state while the latter displays the information in its entirety. Assuming that there are more than one memory locations that are written to, we push the initial buffer state hexdumps to a stack. Consequently, we need to add the post-write instrumentation in reverse order to ensure that the succession of elements popped from the stack is correct. Let's take a look at the pre-write instrumentation insertion:

```
INS_InsertPredicatedCall(
    ins, IPOINT_BEFORE, (AFUNPTR) pre_write_analysis,
    IARG_CALL_ORDER,    CALL_ORDER_FIRST + op_idx + 1,
    IARG_MEMORYOP_EA,   op_idx,
    IARG_MEMORYWRITE_SIZE,
    IARG_END);
```

We notice a new set of parameters:

- `IARG_CALL_ORDER, CALL_ORDER_FIRST + op_idx + 1,`: specifies the call order when multiple analysis routines are registered; see CALL_ORDER enum [https://software.intel.com/sites/landingpage/pintool/docs/97503/Pin/html/group__INST__ARGS.html#ga3d1d5f6805cb16d00bce441290ca2212]'s documentation for details.

Recompile the tool. Test to see that the write analysis routines work properly. Apply *Task-D.patch* and let's move on to applying the finishing touches.

```
[minspect] READ  | 0x00007f33edb06384: mov rdx, qword ptr [rbx+0x88]          -- [ b0 58 e7 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb0638b: mov r8, qword ptr fs:[0x10]            -- [ 00 16 87 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb06394: cmp qword ptr [rdx+0x8], r8            -- [ 00 00 00 00 00 00 00 00 ]
[minspect] READ  | 0x00007f33edb063a1: cmp dword ptr [rip+0x372550], 0x0      -- [ 00 00 00 00 ]
[minspect] READ  | 0x00007f33edb063b2: cmpxchg dword ptr [rdx], esi           -- [ 00 00 00 00 ]
[minspect] WRITE | 0x00007f33edb063b2: cmpxchg dword ptr [rdx], esi           -- [ 00 00 00 00 ] ==> [ 01 00 00 00 ]
[minspect] READ  | 0x00007f33edb063cd: mov rdx, qword ptr [rbx+0x88]          -- [ b0 58 e7 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb063d4: mov eax, dword ptr [rbx]              -- [ 06 20 ad fb ]
[minspect] WRITE | 0x00007f33edb063d6: mov qword ptr [rdx+0x8], r8            -- [ 00 00 00 00 00 00 00 00 ] ==> [ 00 16 87 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb063da: add dword ptr [rdx+0x4], 0x1           -- [ 00 00 00 00 ]
[minspect] WRITE | 0x00007f33edb063da: add dword ptr [rdx+0x4], 0x1           -- [ 00 00 00 00 ] ==> [ 01 00 00 00 ]
[minspect] WRITE | 0x00007f33edb14200: push r12                              -- [ 80 46 e7 ed 33 7f 00 00 ] ==> [ 80 46 e7 ed 33 7f 00 00 ]
[minspect] WRITE | 0x00007f33edb14202: push rbp                              -- [ 80 46 e7 ed 33 7f 00 00 ] ==> [ 60 47 e7 ed 33 7f 00 00 ]
[minspect] WRITE | 0x00007f33edb14203: push rbx                              -- [ 60 47 e7 ed 33 7f 00 00 ] ==> [ 00 54 83 55 ad 0e 3e 6c ]
[minspect] READ  | 0x00007f33edb14204: mov ebp, dword ptr [rdi+0x70]         -- [ 02 00 00 00 ]
[minspect] READ  | 0x00007f33edb14210: mov eax, dword ptr [rdi]              -- [ 06 20 ad fb ]
[minspect] READ  | 0x00007f33edb17c50: cmp qword ptr [rdi+0x60], 0x0         -- [ 00 00 00 00 00 00 00 00 ]
[minspect] READ  | 0x00007f33edb17c5f: mov rax, qword ptr [rdi+0x48]         -- [ 00 00 00 00 00 00 00 00 ]
[minspect] READ  | 0x00007f33edb17cb0: ret                                   -- [ 32 42 b1 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb14232: test byte ptr [rbx+0x74], 0x20        -- [ 00 ]
[minspect] READ  | 0x00007f33edb14238: mov rbp, qword ptr [rbx+0xd8]         -- [ a0 02 e7 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb14262: call qword ptr [rbp+0x88]             -- [ b0 28 b1 ed 33 7f 00 00 ]
[minspect] READ  | 0x00007f33edb128b0: mov edi, dword ptr [rdi+0x70]         -- [ 02 00 00 00 ]
```

[5p] Task E - Finishing Touches

This is only a minor addition. Namely, we want to add a command line option **-i** that can be used multiple times to specify multiple image names (e.g.: ls, libc.so.6, etc.) The tool must forego instrumentation for any instruction that is not part of these objects. As such, we declare a Pin KNOB [https://software.intel.com/sites/landingpage/pintool/docs/98189/Pin/html/group__KNOB__BASIC.html]:

```
static KNOB<string> knob_img(KNOB_MODE_APPEND,    "pintool",
        "i", "", "names of objects to be instrumented for branch logging");
```

We should not use argp [https://www.gnu.org/software/libc/manual/html_node/Argp.html] or other alternatives. In stead, let Pin use its own parser for these things. `knob_img` will act as an accumulator for any argument passed with the flag **-i**. Observe it's usage in *ins_instrum()*.

Determine the shared object dependencies of your target binary of choice. Then try to recompile and rerun the Pin tool while specifying some of them as arguments.

```
$ ldd /bin/ls
        linux-vdso.so.1 (0x00007ffd0d19b000)
        libgtk3-nocsd.so.0 => /usr/lib/x86_64-linux-gnu/libgtk3-nocsd.so.0 (0x00007f32df3ad000)
        libselinux.so.1 => /lib/x86_64-linux-gnu/libselinux.so.1 (0x00007f32df185000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f32ded94000)
        libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f32deb90000)
        libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007f32de971000)
        libpcre.so.3 => /lib/x86_64-linux-gnu/libpcre.so.3 (0x00007f32de6ff000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f32df7d6000)
```

This concludes the tutorial. The resulting Pin tool can now be used as a starting point for developing a Taint analysis [https://users.ece.cmu.edu/~aavgerin/papers/Oakland10.pdf] engine. Discuss more with your lab assistant if you're interested.

NON-DEMO TASK

Patch your way through all the tasks and run the pin tool only for the base object of any binutil.
Include a screenshot of the output.

05. [10p] Feedback

Please take a minute to fill in the **feedback form [https://forms.gle/k7FqUM16AcrkgKTJA]** for this lab.

ep/labs/02.txt · Last modified: 2020/08/18 19:01 by radu.mantu

Lab 03 - I/O Monitoring (Linux)

"Every thing is a file", is a very famous **Linux** philosophy. There is a reason for this philosophy.

It's because, Linux operating system considers and works with most of its devices, by the same way a file is opened or closed.

- Block devices (Hard-disks, Compact Disk, Floppy, Flash Memory)
- Character devices or serial devices (Mouse, keyboard)
- Network Devices

Objectives

- Offer an introduction to I/O monitoring.
- Get you acquainted with a few linux standard monitoring tools and their outputs, for monitoring the impact of the I/Os on the system.
- Gives an intuition to be able to compare two relatively similar systems, but I/O different.

Contents

Tasks

- 01. [10p] Rotational delay - IOPS calculations
- 02. [10p] Iostat
- 03. [10p] Iotop
- 04. [20p] Monitor I/O with vmstat and iostat
- 05. [20p] Multitool Comparison
- 06. [30p] RAM disk
- 07. [10p] Feedback

Introduction

Disk I/O subsystems are the slowest part of any Linux system. This is mainly due to their distance from the CPU and for the old HDD the fact that disk requires physics to work (rotation and seek). If the time taken to access disk as opposed to memory was converted into days and minutes, it is the difference between 7 days and 7 minutes. As a result, it is essential that the Linux kernel minimises the amount of I/O operations it generates on a disk.

The following subsections describe the different ways the kernel processes data I/O from disk to memory and back.

01. Reading and Writing Data - Memory Pages

The Linux kernel breaks disk I/O into pages. The default page size on most Linux systems is **4K**. It reads and writes disk blocks in and out of memory in 4K page sizes. You can check the page size of your system by using the time command in verbose mode and searching for the page size:

 # getconf PAGESIZE

02. Major and Minor Page Faults

Linux, like most UNIX systems, uses a **virtual memory layer** that maps into physical address space. This mapping is **"on-demand"** in the sense that when a process starts, the kernel only maps what is required. When an application starts, the kernel searches the CPU caches and then physical memory. If

the data does not exist in either, the kernel issues a **Major Page Fault** (MPF). A MPF is a request to the disk subsystem to retrieve pages of the disk and buffer them in RAM.

Once memory pages are mapped into the buffer cache, the kernel will attempt to use these pages resulting in a **Minor Page Fault** (MnPF). A MnPF saves the kernel time by reusing a page in memory as opposed to placing it back on the disk.

To find out how many MPF and MnPF occurred when an application starts, the time command can be used:

```
# /usr/bin/time –v evolution
```

As an alternative, a more elegant solution for a specific pid is:

```
# ps -o min_flt,maj_flt ${pid}
```

03. The File Buffer Cache

The **file buffer cache** is used by the kernel to **minimise MPFs and maximise MnPFs**. As a system generates I/O over time, this buffer cache will continue to grow as the system will leave these pages in memory until memory gets low and the kernel needs to "**free**" some of these pages for other uses. The result is that many system administrators see low amounts of free memory and become concerned when in reality, the system is just making good use of its caches 😉

04. Types of Memory Pages

There are **3** types of memory pages in the Linux kernel:

- **Read Pages** – Pages of data read in via disk (MPF) that are read only and backed on disk. These pages exist in the Buffer Cache and include **static files**, **binaries**, and **libraries** that do not change. The Kernel will continue to page these into memory as it needs them. If the system becomes short on memory, the kernel will "steal" these pages and place them back on the free list causing an application to have to MPF to bring them back in.
- **Dirty Pages** – Pages of data that have been modified by the kernel while in memory. These pages need to be synced back to disk at some point by the pdflush daemon. In the event of a memory shortage, kswapd (along with pdflush) will write these pages to disk in order to make room in memory.
- **Anonymous Pages** – Pages of data that do belong to a process, but do not have any file or backing store associated with them. They can't be synchronised back to disk. In the event of a memory shortage, kswapd writes these to the swap device as temporary storage until more RAM is free ("swapping" pages).

05. Writing Data Pages Back to Disk

Applications themselves may choose to write **dirty pages** back to disk immediately using the **fsync()** or **sync()** system calls. These system calls issue a direct request to the **I/O scheduler**. If an application does not invoke these system calls, the pdflush kernel daemon runs at periodic intervals and writes pages back to disk.

Monitoring I/O

Certain conditions occur on a system that may create I/O bottlenecks. These conditions may be identified by using a standard set of system monitoring tools. These tools include **top**, **vmstat**, **iostat**, and **sar**. There are some similarities between the outputs of these commands, but for the most part, each offers a unique set of output that provides a different aspect on performance. The following subsections describe conditions that cause **I/O bottlenecks**.

Calculating IOs Per Second

Every I/O request to a disk takes a certain amount of time. This is due primarily to the fact that a *disk must spin* and *a head must seek*. The spinning of a disk is often referred to as "**rotational delay**" (RD 😎) and the moving of the head as a "**disk seek**" (DS). The time it takes for each I/O request is calculated by adding DS and RD. A disk's RD is fixed based on the RPM of the drive. An RD is considered half a revolution around a disk.

Each time an application issues an I/O, it takes an average of 8MS to service that I/O on a 10K RPM disk. Since this is a fixed time, it is imperative that the disk be as efficient as possible with the time it will spend reading and writing to the disk. The amount of I/O requests is often measured in I/Os Per Second (IOPS). The 10K RPM disk has the ability to push 120 to 150 (burst) IOPS. To measure the effectiveness of IOPS, divide the amount of IOPS by the amount of data read or written for each I/O.

Random vs Sequential I/O

The relevance of KB per I/O depends on the workload of the system. There are two different types of workload categories on a system: sequential and random.

Sequential I/O - The **iostat** command provides information on IOPS and the amount of data processed during each I/O. Use the **–x** switch with **iostat** (*iostat –x 1*). **Sequential workloads** require large amounts of data to be read sequentially and at once. These include applications such as enterprise databases executing large queries and streaming media services capturing data. With sequential workloads, the KB per I/O ratio should be high. Sequential workload performance relies on the ability to move large amounts of data as fast as possible. If each I/O costs time, it is imperative to get as much data out of that I/O as possible.

Random I/O - Random access workloads do not depend as much on size of data. They depend primarily on the amount of IOPS a disk can push. Web and mail servers are examples of random access workloads. The I/O requests are rather small. Random access workload relies on how many requests can be processed at once. Therefore, the amount of IOPS the disk can push becomes crucial.

When Virtual Memory Kills I/O

If the system does not have enough **RAM** to accommodate all requests, it must start to use the **SWAP** device. As file system I/Os, writes to the SWAP device are just as costly. If the system is extremely deprived of RAM, it is possible that it will create a paging storm to the SWAP disk. If the SWAP device is on the same file system as the data trying to be accessed, the system will enter into contention for the I/O paths. This will cause a complete **performance breakdown** on the system. If pages can't be read or written to disk, they will stay in RAM longer. If they stay in RAM longer, the kernel will need to free the RAM. The problem is that the I/O channels are so clogged that nothing can be done. This inevitably leads to a kernel panic and crash of the system.

The following **vmstat** output demonstrates a system under memory distress. It is writing data out to the swap device:

```
 procs -----------memory---------- ---swap-- -----io---- --system-- ----cpu----
 r  b    swpd   free  buff  cache   si   so    bi    bo   in cs   us sy id wa
17  0     1250   3248 45820 1488472   30 132   992     0 2437 7657 23 50  0 23
11  0     1376   3256 45820 1488888   57 245   416     0 2391 7173 10 90  0 0
12  0     1582   1688 45828 1490228   63 131  1348    76 2432 7315 10 90  0 10
12  2     3981   1848 45468 1489824  185 56   2300    68 2478 9149 15 12  0 73
14  2    10385 2400 44484 1489732     0 87   1112    20 2515 11620 0 12  0 88
14  2    12671 2280 43644 1488816    76 51   1812   204 2546 11407 20 45 0 35
```

The previous output demonstrates a large amount of read requests into memory (**bi**). The requests are so many that the system is short on memory (**free**). This is causing the system to send blocks to the swap device (**so**) and the size of swap keeps growing (**swpd**). Also notice a large percentage of WIO time (**wa**). This indicates that the CPU is starting to slow down because of I/O requests. Furthermore, **id** represents the time spent idle and it is included in **wa**

To see the effect the swapping to disk is having on the system, check the swap partition on the drive using **iostat**.

```
# iostat -x 1

avg-cpu: %user %nice %sys %idle
         0.00  0.00 100.00 0.00

Device: rrqm/s wrqm/s   r/s w/s rsec/s wsec/s    rkB/s    wkB/s   avgrq-sz  avgqu-sz   await svctm %util
/dev/sda  0.00  1766.67 4866.67 1700.00 38933.33 31200.00 19466.67 15600.00 10.68    6526.67 100.56 5.08
3333.33
/dev/sda1 0.00 933.33   0.00    0.00     0.00  7733.33     0.00  3866.67  0.00 20.00 2145.07 7.37 200.00
/dev/sda2 0.00 0.00  4833.33    0.00 38666.67   533.33  19333.33  266.67  8.11 373.33 8.07   6.90  87.00
/dev/sda3 0.00 833.33  33.33 1700.00   266.67 22933.33    133.33 11466.67   13.38   6133.33 358.46 11.35
1966.67
```

Both the swap device (*/dev/sda1*) and the file system device (*/dev/sda3*) are contending for I/O. Both have high amounts of write requests per second (*w/s*) and high wait time (*await*) to low service time ratios (*svctm*). This indicates that there is **contention** between the two partitions, causing both to **underperform**.

Takeaways

- Any time the **CPU is waiting** on I/O, the **disks are overloaded**.
- Calculate the amount of **IOPS** your disks can sustain.
- Determine whether your applications require **random** or **sequential** disk access.
- Monitor slow disks by comparing **wait times** and **service times**.
- Monitor the swap and file system partitions to make sure that **virtual memory** is not contending for **filesystem I/O**.

Tasks

For every task/subtask marked with "!!" you should provide proof of the requirements mentioned in that manner.

01. [10p] Rotational delay - IOPS calculations

Every disk in your storage system has a maximum theoretical IOPS value that is based on a formula. Disk performance and IOPS is based on three key factors:

- **Rotational speed**. Measured in RPM, mostly 7,200, 10,000 or 15,000 RPM. A higher rotational speed is associated with a higher-performing disk.
- **Average latency**. The time it takes for the sector of the disk being accessed to rotate into position under a read/write head.
- **Average seek time**. The time (in ms) it takes for the hard drive's read/write head to position itself over the track being read or written.
- **Average IOPS**: Divide 1 by the sum of the average latency in ms and the average seek time in ms (1 / (average latency in ms + average seek time in ms).

To calculate the **IOPS range** divide 1 by the sum of the average latency in ms and the average seek time in ms. The formula is:

```
average IOPS = 1 / (average latency in ms + average seek time in ms).
```

Let's calculate the Rotational Delay - RD for a 10K RPM drive:

- Divide 10000 RPM by 60 seconds: **10000/60 = 166 RPS**
- Convert 1 of 166 to decimal: **1/166 = 0.006 seconds per Rotation**
- Multiply the seconds per rotation by 1000 milliseconds (6 MS per rotation).
- Divide the total in half (RD is considered half a revolution around a disk): **6/2 = 3 MS**
- Add an average of 3 MS for seek time: **3 MS + 3 MS = 6 MS**
- Add 2 MS for latency (internal transfer): **6 MS + 2 MS = 8 MS**
- Divide 1000 MS by 8 MS per I/O: **1000/8 = 125 IOPS**

[10p] Task A - Calculate rotational delay

NON-DEMO TASK

Add in your archive the operations and the result you obtained. (Screenshot, picture of calculations made by hand on paper)

Calculate the rotational delay (RD) for a 5400 RPM drive.

02. [10p] Iostat

Parameteres for iostat:

- -x for extended statistics
- -d to display device stastistics only
- -m for displaying r/w in MB/s

```
$ iostat -xdm
```

Use iostat with -p for specific device statistics:

```
$ iostat -xdm -p sda
```

[10p] Task A - Monitoring the behaviour

- Run *iostat -x 1 5*.
- Considering the last two outputs provided by the previous command, calculate **the efficiency of IOPS** for each of them. Does the amount of data written per I/O **increase** or **decrease**?

NON-DEMO TASK

Add in your archive screenshot or pictures of the operations and the result you obtained, also showing the output of iostat from which you took the values.

How to do:

- Divide the kilobytes read (*rkB/s*) and written (*wkB/s*) per second by the reads per second (*r/s*) and the writes per second (*w/s*).
- If you happen to have quite a few loop devices [https://en.wikipedia.org/wiki/Loop_device] in your **iostat** output, find out what they are exactly:

```
$ df -kh /dev/loop*
```

03. [10p] Iotop

Iotop is an utility similar to top command, that interfaces with the kernel to provide per-thread/process I/O usage statistics.

```
Debian/Ubuntu Linux install iotop
$ sudo apt-get install iotop

How to use iotop command
$ sudo iotop OR $ iotop
```

Supported options by iotop command:

Options	Description
–version	show program’s version number and exit
-h, –help	show this help message and exit

-o, –only	only show processes or threads actually doing I/O
-b, –batch	non-interactive mode
-n NUM, –iter=NUM	number of iterations before ending [infinite]
-d SEC, –delay=SEC	delay between iterations [1 second]
-p PID, –pid=PID	processes/threads to monitor [all]
-u USER, –user=USER	users to monitor [all]
-P, –processes	only show processes, not all threads
-a, –accumulated	show accumulated I/O instead of bandwidth
-k, –kilobytes	use kilobytes instead of a human friendly unit
-t, –time	add a timestamp on each line (implies –batch)
-q, –quiet	suppress some lines of header (implies –batch)

[10p] Task A - Monitoring the behaviour

- Run iotop (install it if you do not already have it) in a separate shell showing only processes or threads actually doing I/O.
- Inspect the script code (dummy.sh) to see what it does.
- Monitor the behaviour of the system with iotop while running the script.
- Identify the PID and PPID of the process running the dummy script and kill the process using command line from another shell (sending SIGINT signal to both parent & child processes).
- Hint - How to get parent PID of a given process in GNU/Linux from command line? [https://superuser.com/questions/150117/how-to-get-parent-pid-of-a-given-process-in-gnu-linux-from-command-line]

NON-DEMO TASK

Provide a screenshot in which it shows the iotop with only the active processes and one of them being the running script. Then another screenshot after you succeeded to kill it.

04. [20p] Monitor I/O with vmstat and iostat

We said in the beginning that the disk I/O subsystems are the slowest part of any system. This is why the I/O monitoring is so important, maximizing the performance of the slowest part of a system resulting in an improvement of the performance of the entire system.

[10p] Task A - Script

Write a script that reads the data into memory and generates a text file 500 times larger, by concatenating the contents of the following novel olivertwist.txt to itself.

[10p] Task B - Monitoring behaviour

Now we want to analyze what is happening with the I/O subsystem during an expensive operation. Monitor the behavior of the system while running your script using **vmstat** and **iostat**.

Understanding vmstat IO section:

- **bi** - column reports the number of blocks received (or "blocks in") from a disk per second.
- **bo** - column reports the number of blocks sent ("blocks out") to a disk per second.

NON-DEMO TASK

Add a picture with the script alongside the terminal with the vmstat/iostat output while the script is running.

05. [20p] Multitool Comparison

Now we will see in a slightly different approach, as for more special situations, a classic tool is not enough as long as we want to have a more detailed analysis of the behavior of I/O mechanisms.

[10p] Task A - Different tool, same I/O

An example would be to create an infinite loop that **copies endlessly** (simulation of a demanding process for a long time) a **large file** (use one of the files obtained previously).

Put the command below in a script:

```
$ while true; do cp original_file1 copied_file2; done
```

Use iostat, iotop and another special monitoring tools for I/O to investigate the state of the system (choose from here: https://www.golinuxcloud.com/monitor-disk-io-performance-statistics-linux/ [https://www.golinuxcloud.com/monitor-disk-io-performance-statistics-linux/])

NON-DEMO TASK

Provide screenshots of the I/O system under the different tools you chose.

[10p] Task B - Plot of Comparison

Plot a graph with the results obtained with iostat, iotop and one of the previously chosen tools. Interpret the graph and the values obtained using those tools.

- basic plot
- the values used should represent the same metric (eg: kb written per second)
- you can take the values manually or automatically
- standardize the values (kb/s)
- preferably use the matplotlib module in python

Fill the data you obtained into the python3 script in plot.zip.

Make sure you have **python3** and **python3-matplotlib** installed.

NON-DEMO TASK

Put the plot in your archive/pdf.

06. [30p] RAM disk

Linux allows you to use part of your RAM as a block device, viewing it as a hard disk partition. The advantage of using a RAM disk is the **extremely low latency** (even when compared to SSDs). The disadvantage is that all contents will be lost after a reboot.

There are two main types of RAM disks:

- **ramfs** - cannot be limited in size and will continue to grow until you run out of RAM. Its size can not be determined precisely with tools like **df**. Instead, you have to estimate it by looking at the "cached" entry from **free**'s output.
- **tmpfs** - newer than **ramfs**. Can set a size limit. Behaves exactly like a hard disk partition but can't be monitored through conventional means (i.e. **iostat**). Size can be precisely estimated using **df**.

[15p] Task A - Create RAM Disk

Before getting started, let's find out the file system that our root partition uses. Run the following command (T - print file system type, h - human readable):

```
$ df -Th
```

The result should look like this:

```
Filesystem     Type      Size  Used Avail Use% Mounted on
udev           devtmpfs  1.1G     0  1.1G   0% /dev
tmpfs          tmpfs     214M  3.8M  210M   2% /run
/dev/sda1      ext4      218G  4.1G  202G   2% / <- root partition
tmpfs          tmpfs     1.1G  252K  1.1G   1% /dev/shm
tmpfs          tmpfs     5.0M  4.0K  5.0M   1% /run/lock
tmpfs          tmpfs     1.1G     0  1.1G   0% /sys/fs/cgroup
/dev/sda2      ext4      923M   73M  787M   9% /boot
/dev/sda4      ext4      266G   62M  253G   1% /home
```

From the results, we will assume in the following commands that the file system is **ext4**. If it's not your case, just replace with what you have:

```
$ sudo mkdir /mnt/ramdisk
$ sudo mount -t tmpfs -o size=1G ext4 /mnt/ramdisk
```

If you want the RAM disk to persist after a reboot, you can add the following line to */etc/fstab*. Remember that its contents will still be lost.

```
tmpfs     /mnt/ramdisk     tmpfs     rw,nodev,nosuid,size=1G     0  0
```

That's it. We just created a 1Gb **tmpfs** ramdisk with an **ext4** file system and mounted it at */mnt/ramdisk*. Use **df** again to check this yourself.

[15p] Task B - Pipe View & RAM Disk

As we mentioned before, you can't get I/O statistics regarding **tmpfs** since it is not a real partition. One solution to this problem is using **pv** to monitor the progress of data transfer through a pipe. This is a valid approach only if we consider the disk I/O being the bottleneck.

Next, we will generate 512Mb of random data and place it in */mnt/ramdisk/file* first and then in */home/student/file*. The transfer is done using **dd** with 2048-byte blocks.

```
$ pv /dev/urandom | dd of=/mnt/ramdisk/rand  bs=2048 count=$((512 * 1024 * 1024 / 2048))
$ pv /dev/urandom | dd of=/home/student/rand bs=2048 count=$((512 * 1024 * 1024 / 2048))
```

Look at the elapsed time and average transfer speed. What conclusion can you draw?

NON-DEMO TASK

Put one ss with the tmpfs partition in df output and one ss of the both pv commands and write your conclusion.

07. [10p] Feedback

Please take a minute to fill in the **feedback form [https://forms.gle/k7FqUM16AcrkgKTJA]** for this lab.

NON-DEMO TASK

If you provided feedback, please put a screenshot of the submitted form.

References

- These examples are from Darren Hoch’s Linux System and Performance Monitoring [http://ufsdump.org/papers/oscon2009-linux-monitoring.pdf].

ep/labs/03.txt · Last modified: 2020/10/25 15:57 by cristian.marin0805

Lab 04 - Networking Monitoring (Linux)

Why is **Networking Important**?

Having a well-established network has become an important part of our lives. The easiest way to expand your network is to build on the relationships with people you know; family, friends, classmates and colleagues. We are all expanding our networks daily.

What to install before lab for non-demo tasks:

- arp-scan
- tcpdump
- wireshark (it is used for 5 and it can also be used for 3)
- sysdig, smartsniff, tcpflow or others tcpdump alternatives (for 3 if you don't want to use wireshark)
- nethogs

Objectives

- Offer an introduction to Network monitoring.
- Get you acquainted with a few Linux standard monitoring tools and their outputs, for monitoring the impact of the Network on the system.
- Provides a set of insights related to understanding networks and connection behavior.

Contents

Tasks

- 01. [10p] Local Network Scan
- 02. [20p] Traffic monitoring - Tcpdump
- 03. [15p] Transfer layer analysis
- 04. [15p] Monitoring Bandwidth Used by Processes
- 05. [30p] NetfilterQueue
- 06. [10p] Feedback

Introduction

01. Ethernet Configuration Settings

Unless explicitly changed, all Ethernet networks are auto negotiated for speed. The benefit of this is largely historical when there were multiple devices on a network at **different speeds and duplexes**.

Most enterprise Ethernet networks run at either 100 or 1000BaseTX. Use **ethtool** to ensure that a specific system is synced at this speed.

In the following example, a system with a 100BaseTX card is running auto negotiated in *10BaseT*.

```
# ethtool eth0
Settings for eth0:
        Supported ports: [ TP MII ]
        Supported link modes:   10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
        Supports auto-negotiation: Yes
        Advertised link modes:  10baseT/Half 10baseT/Full
                                100baseT/Half 100baseT/Full
        Advertised auto-negotiation: Yes
        Speed: 10Mb/s
        Duplex: Half
        Port: MII
        PHYAD: 32
        Transceiver: internal
        Auto-negotiation: on
        Supports Wake-on: pumbg
        Wake-on: d
        Current message level: 0x00000007 (7)
        Link detected: yes
```

The following command can be used to force the card into 100BaseTX:

```
# ethtool -s eth0 speed 100 duplex full autoneg off
```

02. Monitoring Network Throughput

It is impossible to control or tune the switches, wires, and routers that sit in between two host systems. The best way to test **network throughput** is to send **traffic** between two systems and measure statistics like **latency** and **speed**.

Using iptraf for Local Throughput

The **iptraf** utility (http://iptraf.seul.org [http://iptraf.seul.org]) provides a **dashboard** of **throughput** per Ethernet interface. (Use: *# iptraf –d eth0*)

Using netperf for Endpoint Throughput

Unlike **iptraf** which is a **passive interface** that monitors traffic, the **netperf** utility enables a system administrator to perform **controlled tests** of **network throughput**. This is extremely helpful in determining the throughput from a client workstation to a heavily utilised server such as a file or web server. The **netperf** utility runs in a **client/server** mode.

To perform a basic controlled throughput test, the **netperf** server must be running on the server system (*server# netserver*).

There are multiple tests that the **netperf** utility may perform. The most basic test is a standard throughput test. The following test initiated from the client performs a 30 second test of TCP based throughput on a LAN. The output shows that the throughput on the network is around 89 mbps. The server (192.168.1.215) is on the same LAN. This is exceptional performance for a 100 mbps network.

```
client# netperf -H 192.168.1.215 -l 30
TCP STREAM TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to
192.168.1.230 (192.168.1.230) port 0 AF_INET
Recv   Send    Send
Socket Socket  Message  Elapsed
Size   Size    Size     Time     Throughput
bytes  bytes   bytes    secs.    10^6bits/sec

 87380  16384  16384    30.02      89.46
```

Another useful test using **netperf** is to monitor the amount of **TCP request and response** transactions taking place per second. The test accomplishes this by creating a single TCP connection and then sending multiple request/response sequences over that connection (ack packets back and forth with a byte size of 1). This behavior is similar to applications such as RDBMS executing multiple transactions or mail servers piping multiple messages over one connection.

The following example simulates TCP request/response over the duration of 30 seconds.

```
client# netperf -t TCP_RR -H 192.168.1.230 -l 30
TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET
to 192.168.1.230 (192.168.1.230) port 0 AF_INET
Local /Remote
Socket Size   Request  Resp.   Elapsed  Trans.
Send   Recv   Size     Size    Time     Rate
bytes  Bytes  bytes    bytes   secs.    per sec

16384  87380  1        1       30.00    4453.80
16384  87380
```

In the previous output, the network supported a transaction rate of 4453 psh/ack per second using 1 byte payloads. This is somewhat unrealistic due to the fact that most requests, especially responses, are greater than 1 byte.

In a more realistic example, a **netperf** uses a **default size** of **2K** for requests and **32K** for responses.

```
client# netperf -t TCP_RR -H 192.168.1.230 -l 30 -- -r 2048,32768
TCP REQUEST/RESPONSE TEST from 0.0.0.0 (0.0.0.0) port 0 AF_INET to
192.168.1.230 (192.168.1.230) port 0 AF_INET
Local /Remote
Socket Size   Request  Resp.   Elapsed  Trans.
Send   Recv   Size     Size    Time     Rate
bytes  Bytes  bytes    bytes   secs.    per sec

16384  87380  2048     32768   30.00     222.37
16384  87380
```

The transaction rate reduces significantly to 222 transactions per second.

Using iperf to Measure Network Efficiency

The **iperf** tool is similar to the **netperf** tool in that it checks connections between two endpoints. The difference with **iperf** is that it has more in-depth checks around TCP/UDP efficiency such as window sizes and QoS settings. The tool is designed for administrators who specifically want to **tune TCP/IP stacks** and then test the effectiveness of those stacks. The **iperf** tool is a single binary that can run in either server or client mode. The tool runs on port **5001** by default. In addition to TCP tests, **iperf** also has UDP tests to measure packet loss and jitter.

03. Individual Connections with tcptrace

The **tcptrace** utility provides detailed TCP based information about specific connections. The utility uses **libpcap** based files to perform an analysis of specific TCP sessions. The utility provides information that is at times difficult to catch in a TCP stream. This information includes:

- **TCP Retransmissions** – the amount of packets that needed to be sent again and the total data size
- **TCP Window Sizes** – identify slow connections with small window sizes
- **Total throughput** of the connection
- **Connection duration**

For more information refer to pages 34-37 from Darren Hoch's Linux System and Performance Monitoring [http://ufsdump.org/papers/oscon2009-linux-monitoring.pdf].

04. TCP and UDP measurements

"Time remaining: 12 Hours! What's wrong with the network?"

- This issue is all too common and it has nothing to do with the network.

TCP measurements: throughput, bandwidth

- **Capacity**: link speed
  - Narrow link: link with the lowest capacity along a path
  - Capacity of the end-to-end path: capacity of the narrow link
- **Utilized bandwidth**: current traffic load
- **Available bandwidth**: capacity – utilized bandwidth
  - Tight link: link with the least available bandwidth in a path
- **Achievable bandwidth**: includes protocol and host issues
- Many things can **limit TCP throughput**:
  - Loss
  - Congestion
  - Buffer Starvation
  - Out of order delivery

TCP performance: window size

- In data transmission, TCP sends a certain amount of data and then pauses;
- To ensure **proper delivery of data**, it doesn't send more until it receives an acknowledgment from the remote host;

TCP performance: Bandwith Delay Product (BDP)

- The further away the two hosts, the longer it takes for the sender to receive the acknowledgment from the remote host, **reducing overall throughput**.
- To overcome BDP, we send more data at a time ⇒ **we adjust the TCP Window**. Telling TCP to send more data per flow than the default parameters.

To get **full TCP performance** the TCP window needs to be large enough to accommodate the Bandwidth Delay Product.

TCP performance: parallel streams - read/write buffer size

- TCP breaks the stream into pieces transparently
- **Longer writes often improve performance**
  - Let TCP "do it's thing"
  - Fewer system calls
- How?
  - -l <size> (lower case ell)

    - Example –l 128K
- **UDP doesn’t break up writes, don’t exceed Path MTU**
- The –P option sets the number of streams to use

UDP measurements

- Loss
- Jitter
- Out of order delivery
- Use -b to specify target bandwidth (default is 1M)

Takeaways

- Check to make sure all **Ethernet interfaces** are running at proper rates.
- Check **total throughput** per **network interface** and be sure it is inline with network speeds.
- Monitor **network traffic** types to ensure that the **appropriate traffic** has precedence on the system.

Tasks

01. [10p] Local Network Scan

One of the first things that you have to do when trying to guide yourself in a network is to determine what devices you can contact. **arp-scan** is a tool that helps detect devices in a local network by sending ARP echo requests for IP addresses in an arbitrary range.

[5p] Task A - Using arp-scan

Analyse the output of the following command:

```
$ sudo arp-scan --interface eth0 --localnet
```

- **--interface** : specifies the interface used (if absent, uses the lowest numbered configured interface)
- **--localnet** : targets that are scanned are automatically determined from the interface's IP address and its network mask

[5p] Task B - ARP vs ICMP echo request

Investigate localnet-ping.sh and use the script to ping each host detected by **arp-scan**.

NON-DEMO TASK

Explain in a paragraph how the script works and why some devices detected by arp-scan couldn't be reached? (at least in theory)

02. [20p] Traffic monitoring - Tcpdump

In most of the situations presented in all the laboratories we have already gone through, we have seen numerous tools whose output helps us to understand the behavior of the system we are analyzing. Next, we choose the most used Linux utility for **analyzing transferred packets** in a conversation between two or more systems.

Tcpdump captures and prints out a **description** of the contents of **packets** on a **network interface**. Tcpdump utilises the ***libpcap*** library for packet capturing. The packet details can either be displayed on the screen or saved to files.

Supported options by tcpdump command:

Options	**Description**
–version	print the tcpdump and libpcap version strings and exit
-h, –help	print the tcpdump and libpcap version strings, print a usage message, and exit
-B buffer_size	set the operating system capture buffer size to *buffer_size*, in units of KiB
-c count	exit after receiving *count* packets
-D	print the list of the network interfaces on which tcpdump can capture packets
-i interface	report the results of compiling a filter expression on *interface*
-n	don't convert addresses (host addresses, port numbers) to names
-s snaplen	truncate *snaplen* bytes of data from each packet rather than the default

-t	don't print a timestamp on each dump line
-v	produce more verbose output
-w file	write the raw packets to *file* rather than parsing and printing them out
-r file	read packets from file
-A	print each packet in ASCII

- Check if **tcpdump** is installed and which **version** is installed.
- Check out the **network interfaces** available on your system.
- After starting a capture on all interfaces, you can always stop it using ***control + c***.

[10p] Task A - Understanding traffic

a) Start a capture that stops by itself after getting 10 packets on all interfaces.

b) Have a look at the output. You can notice that **host names** are used instead of **IP addresses**, and commonly known port are replaced with application names. Use a command to display the **IP addresses** and port numbers instead of these names.

Tcpdump triggers itself DNS traffic as it captures, if it is ran without the *-n* option. The utility will trigger reverse or PTR DNS lookups to find hostnames for IP addresses as it captures them. So, from now on, use *-n*.

- What is the capture size?

What does this mean? It means that tcpdump will keep all those bytes for analysis. We don't need all this information for now, so change the capture size to 96 bytes. The Ethernet, IP and TCP headers are the in the first 64 bytes of the packets, so capturing 96 bytes per packet is more than enough to capture these headers.

c) Do the capture again with the output limitation.

The **TCP flags** are **SYN**, **ACK**, **RESET**, **FIN**, **URGENT** and **PUSH**. All flags are represented by the first letter, with the exception of ACK which is represented by a **dot**.

d) Start a new capture only on the interface that connects you to the internet, without printing the timestamp on each dump line. Open a separate terminal and try to connect through ssh somewhere. Spot the **3-way handshake** in the capture.

e) Repeat what you did for the previous task, but add *-S* to your tcpdump command. Figure out what has changed, and why.

Check out the **window size** in the previous capture. Since window scaling is enabled, that is not the *actual* window size. Notice the window scaling factor (***wscale***) in the 3-way handshake output. The scaling factor translates in multiplying the receive window by 2 to the power of wscale. So the *real* window size is the window value shown in the capture, multiplied by 2 to the power of wscale.

The **length field** stands for packet length, and represents the number of bytes in the layer 4 headers, and it matches with the sequence numbers **(packet_length = larger_seq_no - smaller_seq_no)**.

[10p] Task B - DNS capture

If we are the victims of a possible cyber attack (DNS hijacking), the DNS request packages are investigated.

We will simulate the monitoring of all DNS packages.

a) Capture an output for a DNS request.

b) Save a capture to a file. Use the appropriate options so that:

- it displays the number of packets captured
- the capture stops after 30 packets

c) Read the contents of the capture file.

NON-DEMO TASK

Using filters helps you view just the types of traffic that you are interested in and ignore the rest. Create short captures of up to 5 packets for the following cases:

- Capture traffic just from the IP 8.8.8.8
- Capture traffic having the source IP 8.8.8.8
- Capture traffic to or from your PC on port 80
- Capture traffic to or from your PC on port 80 or port 443

Include the commands or a screenshot of the terminal in the submission.

03. [15p] Transfer layer analysis

For this task, we want to analyze the network for the network traffic while using Proxychains and observe the effect on the performance.

Proxying refers to the technique of bouncing your Internet traffic through multiple machines to hide the identity of the original machine, or to overcome network restrictions. **ProxyChains** is a tool that hackers often use to accomplish this goal, it forces any TCP connection made by any given application to go through proxies.

[15p] Task A - Proxychains performance

First of all, we want to set up proxychains. For this, you can follow this link [https://linuxhint.com/proxychains-tutorial/] and use any proxy server. After this, use a tool (not **tcpdump**) to analyze the effects produced by proxying in comparison with normal traffic.

Some tcpdump alternatives:

- wireshark
- sysdig
- smartsniff
- tcpflow

NON-DEMO TASK

Add 2 screenshots of the output generated using the selected tool, one for the normal traffic and one using the proxy server.

Besides the advantages of proxying(hide the identity, overcome network restrictions), there are also disadvantages. Write in one paragraph some of them.

04. [15p] Monitoring Bandwidth Used by Processes

Nethogs is a small 'net top' tool that shows the bandwidth used by individual processes and sorts the list putting the most intensive processes on top. Nethogs returns the PID, user and the path of the program.

[10p] Task A - Monitoring the behaviour

Open a data streaming website (example: youtube.com) and start downloading/playing content. Use nethogs (sudo apt-get install nethogs) to find the process that uses **most of the bandwidth** and kill it.

NON-DEMO TASK

Use nethogs to find the process that uses **most of the bandwidth** on the primary interface(eth0) and set the refresh rate to 5 seconds. Add the command and a picture with the output generated by nethogs.

05. [30p] NetfilterQueue

A tool that you have previously used is **iptables**. This user space tool interacts with the packet filtering framework that is implemented in the kernel (i.e. netfilter [https://www.netfilter.org/]) in order to allow you to specify your firewall rules.

Netfilter is a framework for packet mangling, outside the normal Berkeley socket interface. It has four parts. Firstly, each protocol defines “hooks” (IPv4 defines 5) which are well-defined points in a packet’s traversal of that protocol stack. At each of these points, the protocol will call the netfilter framework with the packet and the hook number.

In this exercise, we introduce **Netfilter Queues**, an **iptables** target (also part of the netfilter project). What's interesting about this tool is that the hook that it inserts in the network protocol stack does not immediately decide what happens to the packet, based only on the usual vectors (protocol, src/dst IP, src/dst port, etc). Instead, it sends the packet to a user space process for analysis. All packets are buffered and the user space process can retrieve them, evaluate their contents and decide whether to **ACCEPT, DROP or RETURN** them. The messages do not need to be evaluated in the order that they were received. Moreover, the process can even alter the packets and ask the kernel to use the modified version (this happens to be very useful when implementing MitM [https://en.wikipedia.org/wiki/Man-in-the-middle_attack] attacks).

Next, we will look at two examples of **Netfilter Queue** analysis programs that also alter the traffic. First, download the demo scripts. Also, make sure that you install all dependencies before starting the tasks:

```
$ sudo apt update
$ sudo apt install -y python3-pip wireshark openssh-server libnetfilter-queue1 libnetfilter-queue-dev nfqueue-bindings-python python3-scapy
$ pip3 install NetfilterQueue
```

[15p] Task A - DNS hijack

In this task we will intercept **all DNS responses** and alter the returned IP address for a certain domain name. Before proceeding with the following commands, make sure you have a ssh server running on your machine (we will mess around with that later on). The main goal is to understand how it all works, so make sure you read the script.

If you feel that you need a better understanding of the DNS message format, check out Let's hand write DNS messages [https://routley.io/posts/hand-writing-dns-messages/].

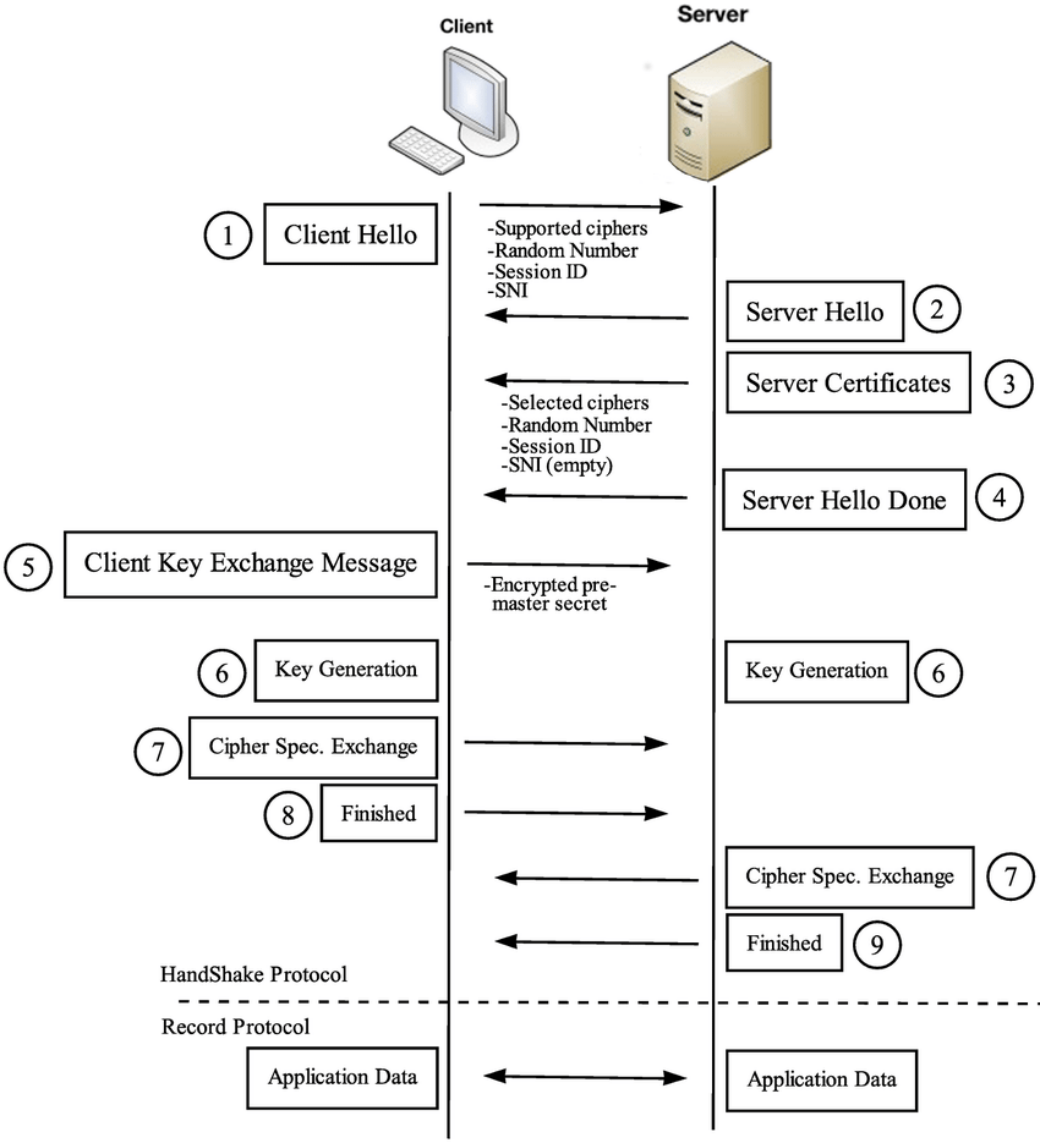
```
$ dig +short fep.grid.pub.ro
$ sudo iptables -I INPUT -p udp --sport 53 -j NFQUEUE --queue-num 1
$ sudo ./mitm-dns_hijack.py fep.grid.pub.ro. 127.0.0.1

$ dig +short fep.grid.pub.ro
$ ssh student@fep.grid.pub.ro
    Password: student
$ sudo iptables -D INPUT 1
```

Let's take a look at what just happened:

- first, we use **dig** to obtain the IP address of *fep.grid.pub.ro* and it should be *141.85.241.99* or something along those lines
- next, we add an **iptables** rule: *redirect all UDP incoming traffic that originated from port 53 to queue number 1 for validation in userspace*
- then, we run the python3 script that will subscribe to queue number 1 and will replace all DNS reponses to fep's domain with 127.0.0.1
- after checking the IP again with **dig**, we notice that the script worked
- what may be a possible implication? Well… let's try connecting to *fep* via **ssh**
- finally, we stop the script and delete the **iptables** rule:

[15p] Task B - TLS downgrade & Wireshark

TLS is a cryptographic protocol meant to provide secure communication over a network (e.g. *https*). All sessions that use this protocol start with a *TLS handshake* (see figure below). When the client contacts the server, it sends a *Client Hello*. This message includes a list of supported encryption algorithms. The server receives this message, chooses the best encryption algorithm (that it also known) and responds with a *Server Hello*, containing the chosen algorithm.



So we know how **Netfilter Queues** and **TLS** work. In this task we will use **wireshark** to detect abnormal traffic. This time, our script will intercept all *Client Hello* messages and replace the supported cipher suite list with a single (weaker) item that the server will be forced to select. Let's try to connect to *ocw.cs.pub.ro* and see what cipher suite it normally chooses:

```
$ echo | openssl s_client -connect ocw.cs.pub.ro:443
```

The answer should be *ECDHE-RSA-AES256-GCM-SHA384*. Now, make sure you have **wireshark** installed and get a network capture of this unaltered handshake. Save it for later.

Next, set up the **iptables** rule and run the process:

```
$ sudo iptables -I OUTPUT -p tcp --dport 443 -j NFQUEUE --queue-num 1 --queue-bypass
$ sudo ./mitm-tls_downgrade.py

$ sudo iptables -D OUTPUT 1
```

Try to capture the *Client Hello* once again with **wireshark**.

NON-DEMO TASK

Add in the submission the capture of the unaltered handshake and the capture after running the script.

Place the two captures side by side and identify the cipher suites lists. Explain briefly which algorithm did our script force the server to accept.

06. [10p] Feedback

Please take a minute to fill in the **feedback form [https://forms.gle/k7FqUM16AcrkgKTJA]** for this lab.

NON-DEMO TASK

If you provided feedback, please put a screenshot of the submitted form.

ep/labs/04.txt · Last modified: 2020/11/04 22:56 by gheorghe.petre2608

EP - Python Scientific Computing & Basic Plotting (Numpy & Matplotlib)

In this lab, we will study a new library in python that offers fast, memory efficient manipulation of vectors, matrices and tensors: **numpy**. We will also study basic plotting of data using the most popular data visualization libraries in the python ecosystem: **matplotlib**.

```
# Some IPython magic
# Put these at the top of every notebook, to get automatic reloading and inline plotting
%reload_ext autoreload
%autoreload 2

import numpy as np
```

Python scientific computing ecosystem

For scientific computing we need an environment that is easy to use, and provides a couple of tools like manipulating data and visualizing results. Python is very easy to use, but the downside is that it's not fast at numerical computing. Luckily, we have very eficient libraries for all our use-cases.

Libraries

Core computing libraries

- `numpy` and `scipy`: scientific computing
- `matplotlib`: plotting library

Machine Learning

- `sklearn`: machine learning toolkit
- `tensorflow`: deep learning framework developed by google
- `keras`: deep learning framework on top of `tensorflow` for easier implementation
- `pytorch`: deep learning framework developed by facebook

Statistics and data analysis

- `pandas`: very popular data analysis library
- `statsmodels`: statistics

We also have advanced interactive environments:

- Ipython: advanced python console

- Jupyter: notebooks in the browser

There are many more scientific libraries available.

Check out these cheetsheets for fast reference to the common libraries:

Cheat sheets:

- python
- numpy
- matplotlib
- sklearn
- pandas

Other:

- Probabilities & Stats Refresher
- Algebra

numpy

`numpy` works with tensors of data, the main data structure is `numpy.array` or `numpy.ndarray`.

Why it is useful: Memory-efficient container that provides fast numerical operations.

Speed test

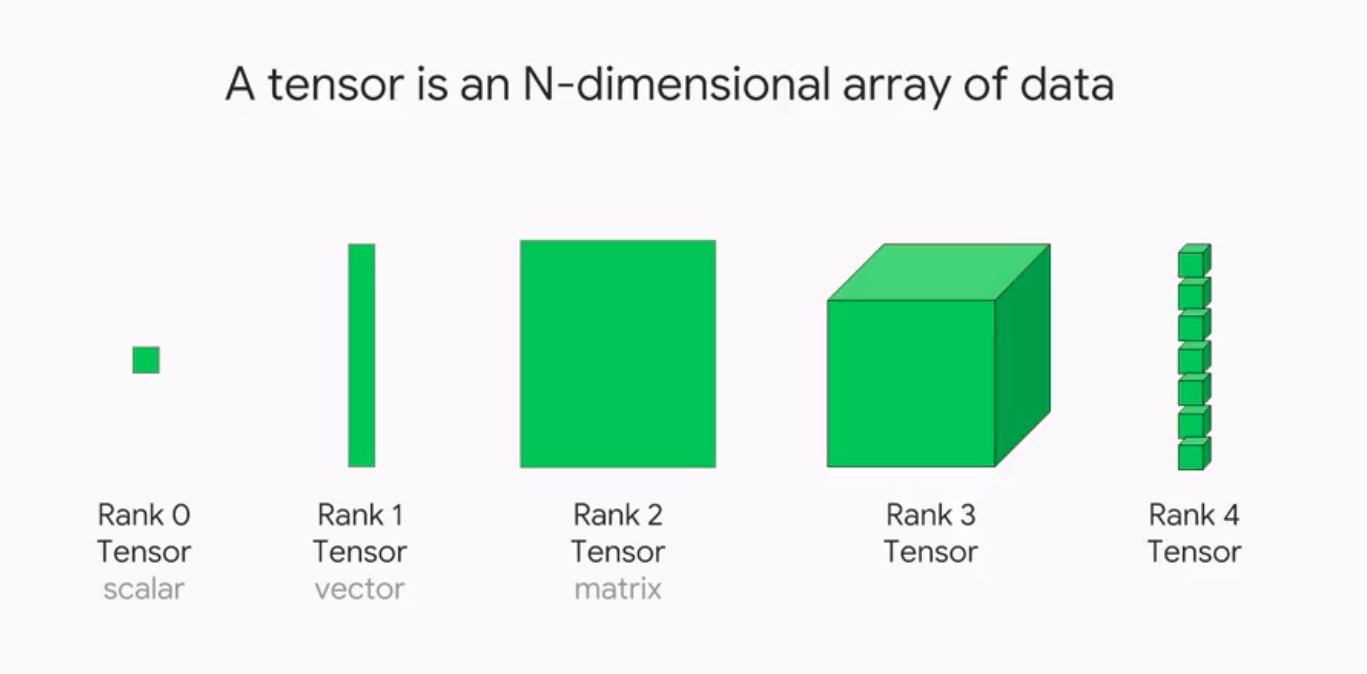
```
%timeit [x**2 for x in range(1000)]
```

```
1000 loops, best of 3: 270 µs per loop
```

```
%timeit np.arange(1000)**2
```

```
The slowest run took 1010.07 times longer than the fastest. This could mean that an i
100000 loops, best of 3: 3.72 µs per loop
```

It is clear that math operations using numpy arrays are far more efficient computation-wise than using plain python lists. More than that, numpy arrays offer a rich API for ***tensor*** manipulation.



Working with numpy arrays

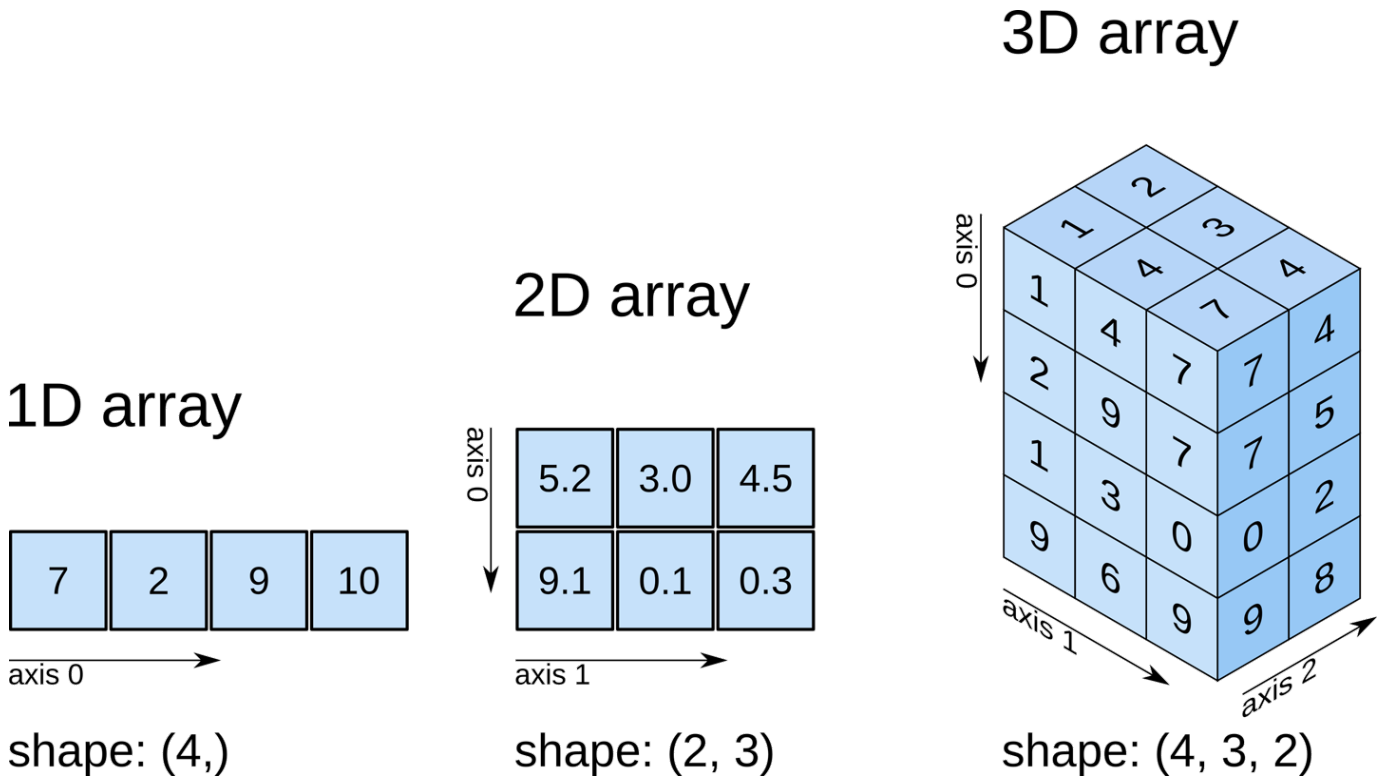
a few important attributes of the `array` structure:

- `ndarray.ndim` - number of axes (tensor rank)
- `ndarray.shape` - tuple, gives dimensions of each axes.
- `ndarray.size` - size of the array (product of elements of `ndarray.shape`)
- `ndarray.dtype` - data type

```
a = np.array([[2,7,5],[0,-1,2]])
print(a)
print("Dim:  ",a.ndim)
print("Shape:",a.shape)
print("Size: ",a.size)
print("Type: ",a.dtype)
```

```
[[ 2  7  5]
 [ 0 -1  2]]
Dim:   2
Shape: (2, 3)
Size:  6
Type:  int64
```

Shape Manipulation




```
x = np.array([1,1,2])
x.shape
```

```
(3,)
```

```
x.reshape(1,3)
```

```
array([[1, 1, 2]])
```

```
x.reshape(1,4) # this will fail because dimensions don't match
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
<ipython-input-7-72e7449de1fa> in <module>()
----> 1 x.reshape(1,4) # this will fail because dimensions don't match

ValueError: cannot reshape array of size 3 into shape (1,4)
```

SEARCH STACK OVERFLOW

```
x.reshape(1,-1)
```

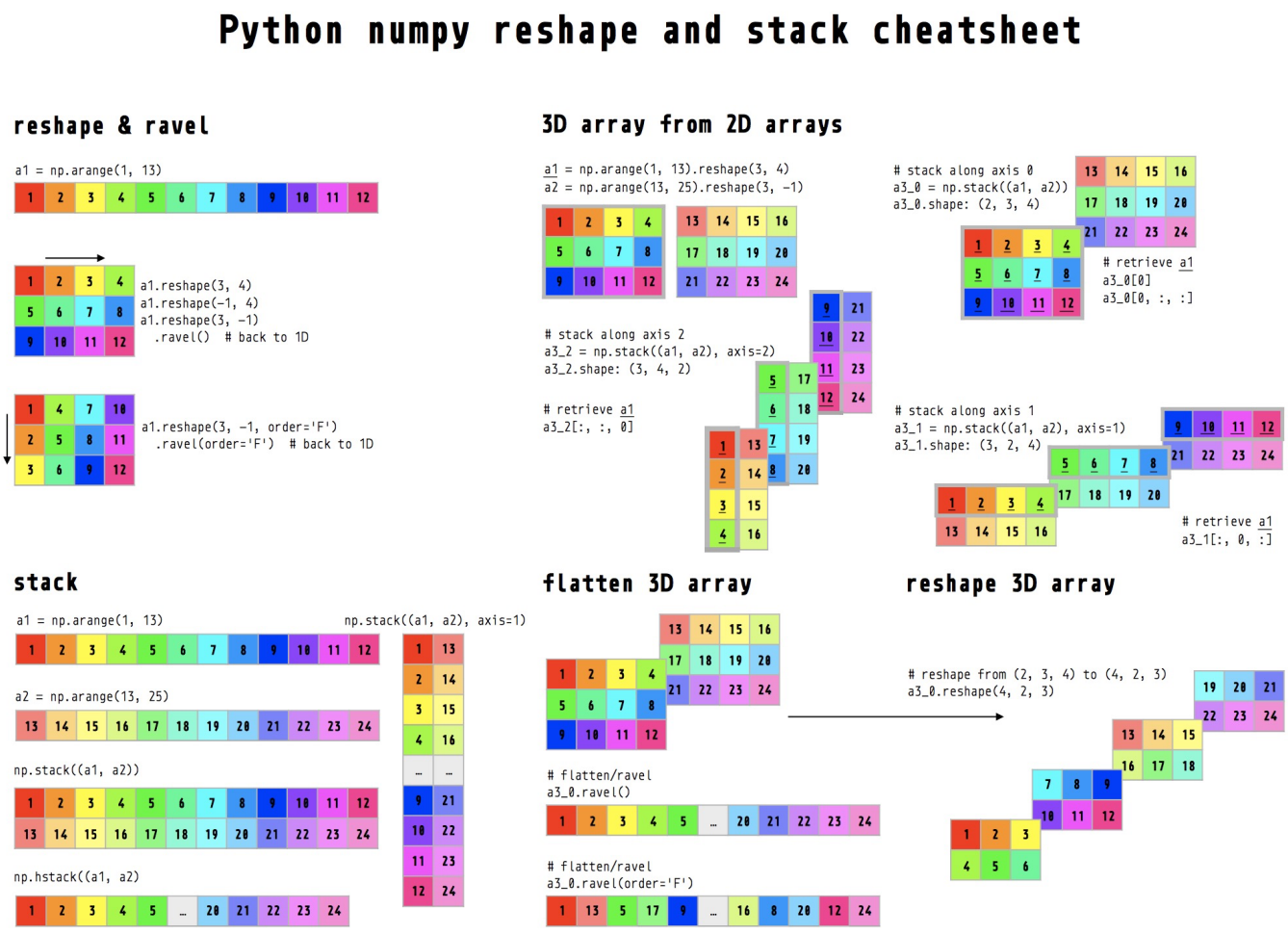
```
a = np.floor(10*np.random.random((3,4))) # some random matrix
a
```

```
a.shape # a tuple that describes the shape of the object
```

```
a.ravel() # flattened list
```

```
a.T # transpose
```

```
a.reshape(2,1,-1) # -1 means "as much as you need", such that the dimensions maches the si
```



Initializing

- `np.zeros(shape)`
- `np.ones(shape)`
- `np.empty(shape)`
- `np.eye(rows, cols)`

Those functions take as input the shape of an array.

Sequences

- `np.arange` - like Python's `range()`, but returns a `np.array`
- `np.linspace` - n equidistant points in interval [a,b]

```
np.arange(5)
```

```
array([0, 1, 2, 3, 4])
```

```
np.linspace(1, 2, 5)
```

```
array([1.  , 1.25, 1.5 , 1.75, 2.  ])
```

Math operators

Math operators are applied elementwise. (`*` is never the dot product for numpy arrays!)

```
a = np.ones((2,3), dtype=int)
a *= 3
a
```

```
array([[3, 3, 3],
       [3, 3, 3]])
```

```
a = a - 1
a
```

```
array([[1, 1, 1],
       [1, 1, 1]])
```

```
a = a + np.ones_like(a) / 5
a
```

```
array([[1.2, 1.2, 1.2],
       [1.2, 1.2, 1.2]])
```

```
print("Pointwise multiplication:\n", a * np.ones_like(a) * 2 )
print("Dot Product (matrix multiplication)\n", np.dot(a.T, np.ones_like(a) * 2))
```

```
Pointwise multiplication:
 [[9.6 9.6 9.6]
 [9.6 9.6 9.6]]
Dot Product (matrix multiplication)
 [[19.2 19.2 19.2]
 [19.2 19.2 19.2]
 [19.2 19.2 19.2]]
```

Linear Algebra

numpy has implementations of some of the most common algebraic operations. The most common one, the dot product is implemented both as a method of the array object and as a numpy function.

Matrix multiplication can be performed either using the dot product or `np.matmul`. Using functions from numpy you can

- find eigenvalues and eigenvectors,
- compute some decompositions (QR, SVD),
- compute determinant of a matrix,
- norm of a vector,
- solve a linear system
- invert a matrix

See the documentation

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. Almost all python plotting frameworks make use of matplotlib behind the scenes.

Plotting data is an effective way to convey the message hidden in the data. A good plot is worth a thousand words, and it can be difficult to unambiguously transmit your intended message. Different types of plots that represent the same data (i.e. scatter plots, linear plots, bar plots, pie charts etc.) can be perceived differently depending on who's looking.

Check out this article for some tips on better plotting:

Ten Simple Rules for Better Figures

NB: Do note that, regardless of the type of plot and data, ***all*** plots must have properly annotated axis ticks, axis labels, a title and a caption.

For a more comprehensive tutorial on matplotlib, check out this resource:

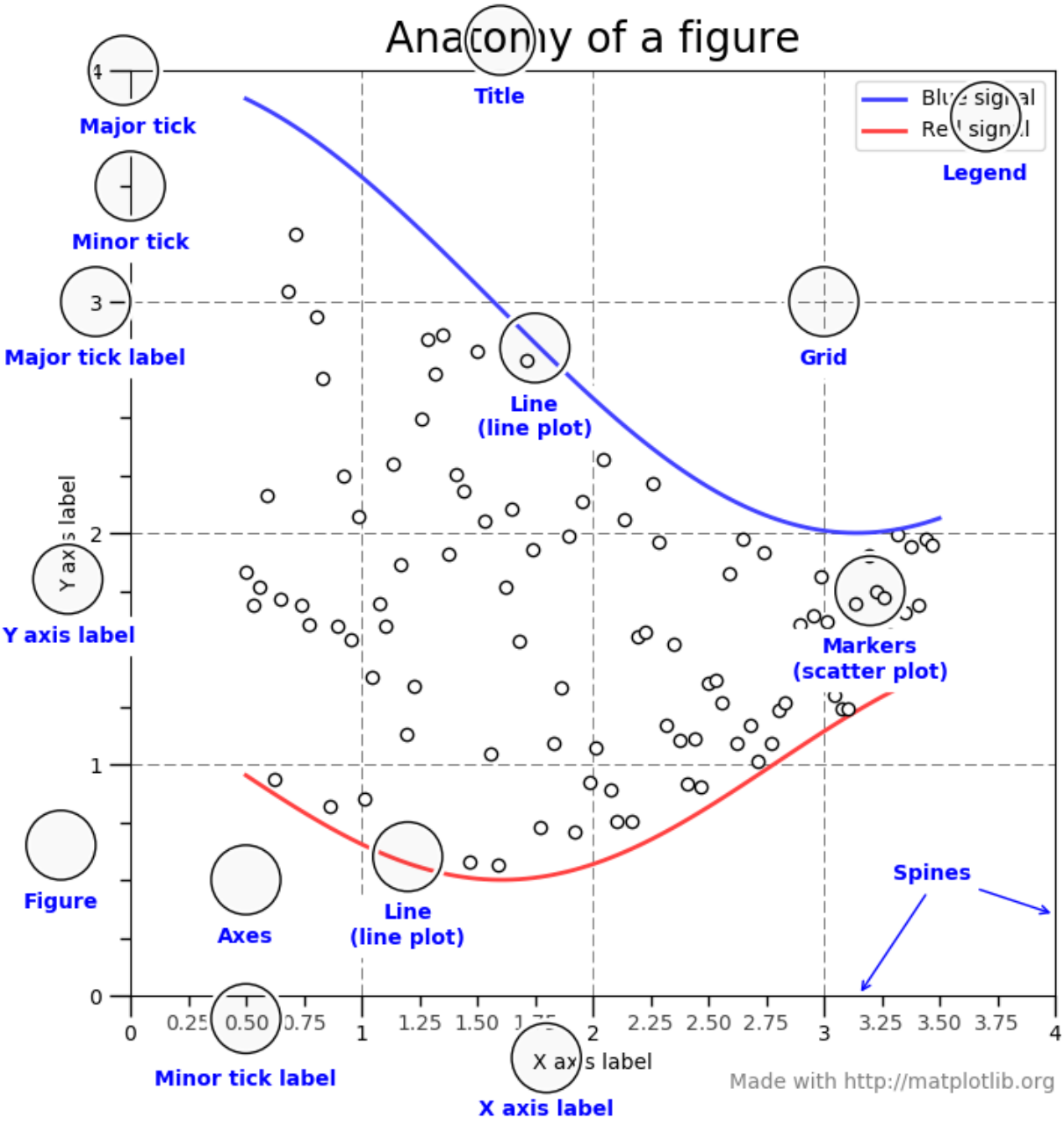
Matplotlib Tutorial – A Complete Guide to Python Plot w/ Examples

```
import matplotlib.pyplot as plt
%matplotlib inline
```

The elements of a plot

The figure and the axes

The figure is the entire image, the individual plots are called "axes". Here's a description from matplotlib's documentation.



Scatter & Line Plots

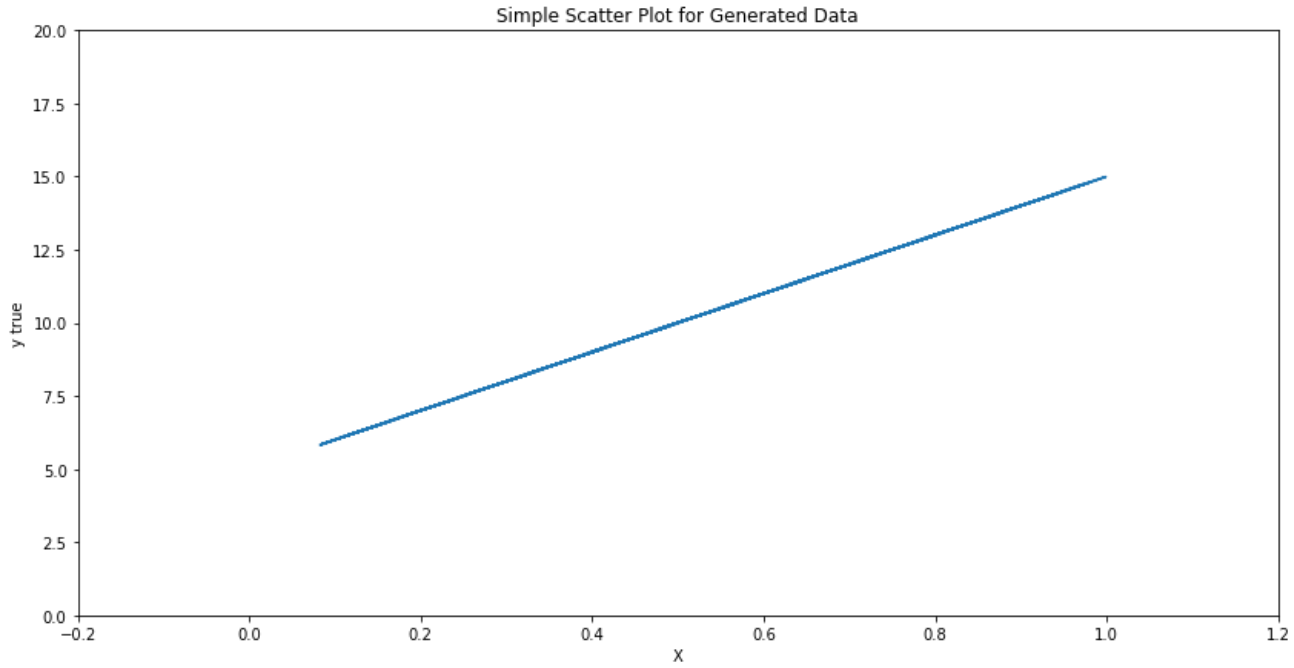
Scatter plots are used to plot data points on horizontal and vertical axis in the attempt to show how much one variable is affected by another. Each row in the data table is represented by a marker the position depends on its values in the columns set on the X and Y axes. A third variable can be set to correspond to the color or size of the markers, thus adding yet another dimension to the plot.

We will plot a simple regression line with generated data.

```
M = 10
N = 5
```

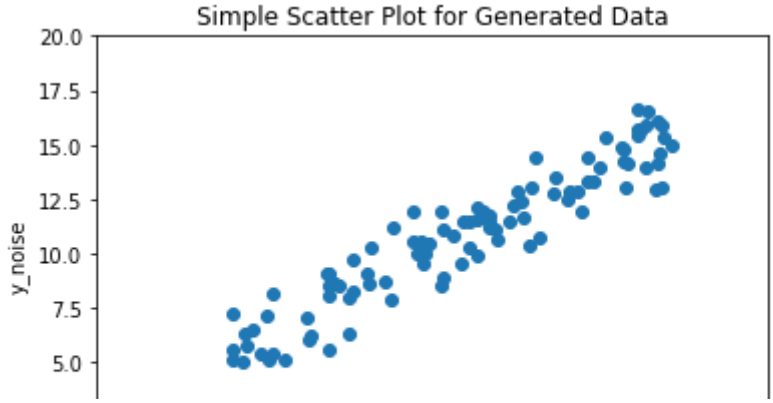
```
X = np.random.random(size = 100)
y_true = M * X + N

y_noise = y_true + np.random.normal(size = 100)
```

```
plt.plot(X, y_true)
plt.xlim(-0.2, 1.2)
plt.ylim(0, 20)

plt.xlabel("X")
plt.ylabel("y true")

plt.title("Simple Scatter Plot for Generated Data")
plt.gcf().set_size_inches(14, 7)
```




```
plt.scatter(X, y_noise)
plt.xlim(-0.2, 1.2)
plt.ylim(0, 20)

plt.xlabel("X")
plt.ylabel("y_noise")

plt.title("Simple Scatter Plot for Generated Data")
```

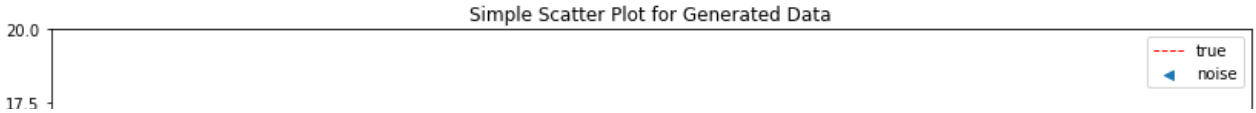
```
Text(0.5, 1.0, 'Simple Scatter Plot for Generated Data')
```



We can plot the two variables one on top of the other, for a better visualization. Make sure to add a legend, and different coloring, such that it is clear which part of the plot belongs to which set of variables.

Matplotlib also offers a wide range of **markers** to better differentiate between multiple data categories. Check https://matplotlib.org/api/markers_api.html for a full list.

```
plt.scatter(X, y_noise, label = 'noise', marker = '<')
plt.plot(X, y_true, color = 'r', label = 'true', linestyle = 'dashed', linewidth = 1)

plt.xlim(-0.2, 1.2)
plt.ylim(0, 20)

plt.xlabel("X")
plt.ylabel("t")

plt.title("Simple Scatter Plot for Generated Data")
plt.legend()
plt.gcf().set_size_inches(14, 7)
```

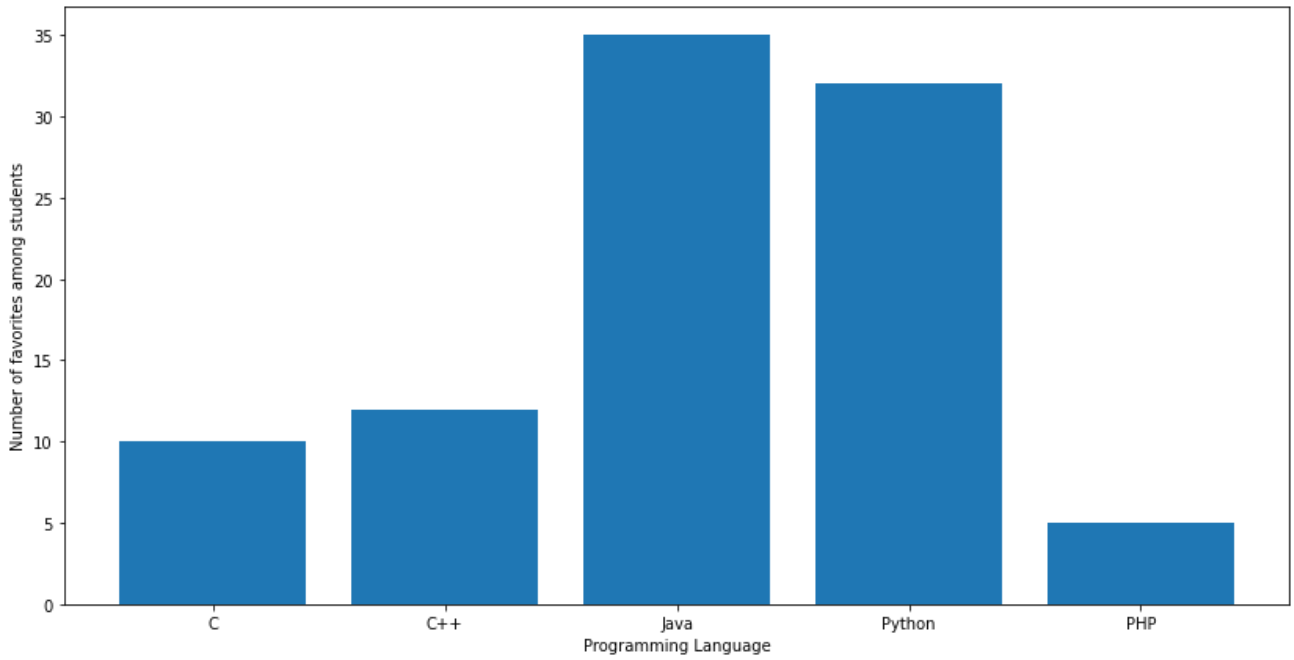


Bar Plots

A bar chart or bar graph is a chart or graph that presents categorical data with rectangular bars with heights or lengths proportional to the values that they represent. The bars can be plotted vertically or horizontally.

A bar graph shows comparisons among discrete categories. One axis of the chart shows the specific categories being compared, and the other axis represents a measured value.

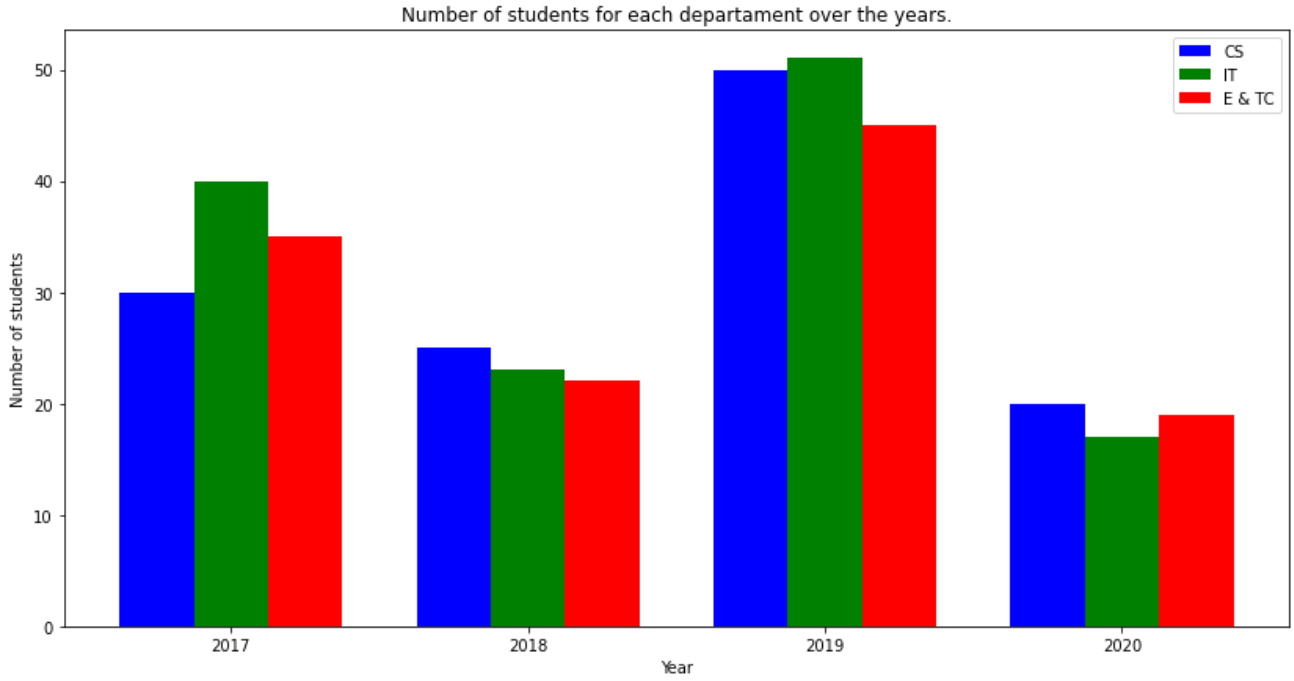
```
languages = ['C', 'C++', 'Java', 'Python', 'PHP']
students = [10, 12, 35, 32, 5]

ax = plt.subplot()
ax.bar(languages, students)

plt.xlabel('Programming Language')
plt.ylabel("Number of favorites among students")
plt.gcf().set_size_inches(14, 7)
```



We can make multiple bars for each category. It makes it easier for comparing different quantities with a common category.

```
data = [
```

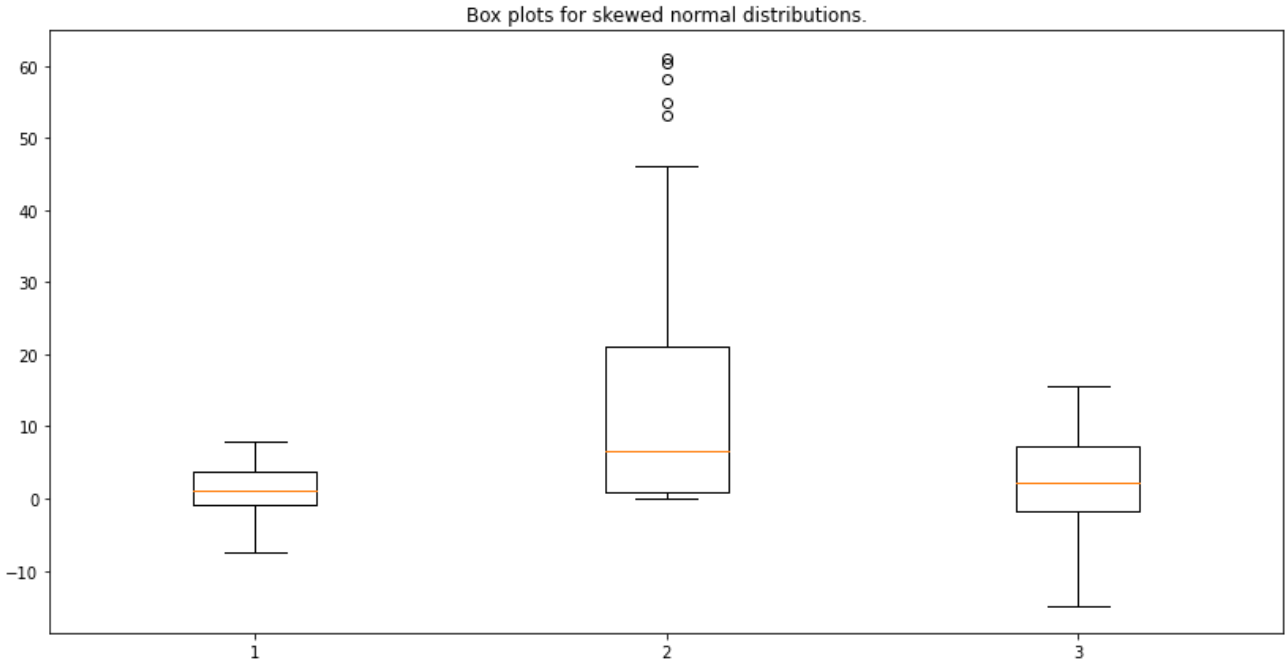
```
  [30, 25, 50, 20],
  [40, 23, 51, 17],
  [35, 22, 45, 19]
]

X = np.arange(4)
ax = plt.subplot()

ax.bar(X + 0.00, data[0], color = 'b', width = 0.25, label = 'CS')
ax.bar(X + 0.25, data[1], color = 'g', width = 0.25, label = 'IT')
ax.bar(X + 0.50, data[2], color = 'r', width = 0.25, label = 'E & TC')

ax.set_xticks(X + 0.25)
ax.set_xticklabels(X + 2017)

plt.xlabel("Year")
plt.ylabel("Number of students")
plt.title("Number of students for each departament over the years.")
ax.legend()
plt.gcf().set_size_inches(14, 7)
```



Box Plots

A boxplot is a standardized way of displaying the distribution of data based on a five number summary (“minimum”, first quartile (Q1), median, third quartile (Q3), and “maximum”). It can tell you about your outliers and what their values are. It can also tell you if your data is symmetrical, how tightly your data is grouped, and if and how your data is skewed.

```
data = np.random.normal(size = 100, scale = 3.0, loc = 1.0)

ax = plt.subplot()
ax.boxplot([data, data ** 2, data * 2])

plt.title("Box plots for skewed normal distributions.")
plt.gcf().set_size_inches(14, 7)
```



Exercises

After you finish your lab exercises, you should export this notebook as **pdf** and upload it to Moodle. (i.e. **File -> Print**, Destination: Save as PDF).

1. System of linear equations

Using numpy, solve the following linear equation:

$$\begin{bmatrix} 1 & 2 & 3.3 & 2 \\ 3 & 3.6 & 7 & 0 \\ 1 & 3 & -1 & 12 \\ 2 & 11 & 4 & 16 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 10 \\ 15 \\ 3 \\ -1 \end{bmatrix}$$

```
# Your code here
```

2. Simple numpy exercises

1. Given an matrix, calculate the sum for each row
2. Given an vector, normalize the vector using the l_{10} norm

```
M = np.array([
    [1, 2, 3],
    [9, 3, 10],
    [3, 5, 0],
    [1, 7, -3],
    [0, -2, 3],

])

v = np.array([100, 2302, 2, -10, 134, -1])

# Your code here
```

3. Harder numpy exercises

Using the matrix and vector defined above:

1. Subtract the mean of each column in a given matrix.
2. Get the 3rd largest value in an array.

```
# Your code here
```

4. Plot some functions

Using numpy and matplotlib, plot the following functions (you can choose the colors and styling, interval of interest and other parameters):

$f_1(x) = max(0, x)$

$f_2(x) = \frac{e^{2x}-1}{e^{2x}+1}$

$f_3(x) = \frac{1}{1+e^{-x}}$

$f_4(x) = \frac{1}{\mu^2+1} * e^{-\frac{(x-\mu)^2}{b^2}}$

$f_5(x) = 0.5x(1 + tanh(0.797885x + 0.035677x^3))$

Note that all plots should have relevant ticks, labels and a title.

```
# Your code here
```

5. Prettify Plot

Given the below plot, make it look presentable. Add **square** markers to scatter plots, **dashdotted** lines, add relevant axis **limits** and **ticks**, a **legend** and a **title**.

NB: A plot should be self-contained. Any reader should understand it without have to refer to its text description.

```
# Data Synthetization
x1 = np.linspace(0.0, 5.0)
x2 = np.linspace(0.0, 2.0)

y1 = np.cos(2 * np.pi * x1) * np.exp(-x1)
y2 = np.cos(2 * np.pi * x2)
y3 = np.cos(1 * np.pi * x1) * np.exp(-0.5 * x1) + 0.3 * np.random.random(size = x1.shape)

# Plotting code
# TODO make this pretty.

fig, (ax1, ax2) = plt.subplots(2, 1)

ax1.plot(x1, y1)
ax1.scatter(x1, y3)
ax1.get_xaxis().set_ticks([])
ax1.get_yaxis().set_ticks([])

ax2.plot(x2, y2)
ax2.get_xaxis().set_ticks([])
ax2.get_yaxis().set_ticks([])

plt.gcf().set_size_inches(14, 7)
```

6. Plotting a dataset

Plot different aspects of the Wine Dataset from sklearn. These data are the results of a chemical analysis of wines grown in the same region in Italy but derived from three different cultivars.

6.1 Create a plot for visualizing the distribution of one features (e.g. alchohol level / malic acid / ash etc.) for each class of wine.

6.2 Create 3 subplots highlighting different aspects of the dataset. Choose the appropriate plot types (scatter plot, line plot, bar plot, box plot etc.). You can choose various combinations of columns to gain a better understanding of this dataset.

Be creative! You can color your points using the `target` array. Remember to always have labels on your axes, appropriate ticks, a legend where necessary, and a plot title.

```
from sklearn.datasets import load_wine

wine_dataset = load_wine()

data = wine_dataset['data']
target = wine_dataset['target']

column_names = wine_dataset['feature_names']

# TODO your code here
```

EP - Data Manipulation & Advanced Plotting (pandas, seaborn & 3D Plotting)

In this lab, we will study data manipulation and visualization using **pandas**, and explore the high level API of **seaborn** for generating visually appealing plots. We will also take a look at 3D plotting using **mplot3d**.

```
# Some IPython magic
# Put these at the top of every notebook, to get automatic reloading and inline plotting
%reload_ext autoreload
%autoreload 2

import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
```

Check out these cheetsheets for fast reference to the common libraries:

Cheat sheets:

- python
- numpy
- matplotlib
- sklearn
- pandas

Other:

- Probabilities & Stats Refresher
- Algebra

Pandas Crash Course

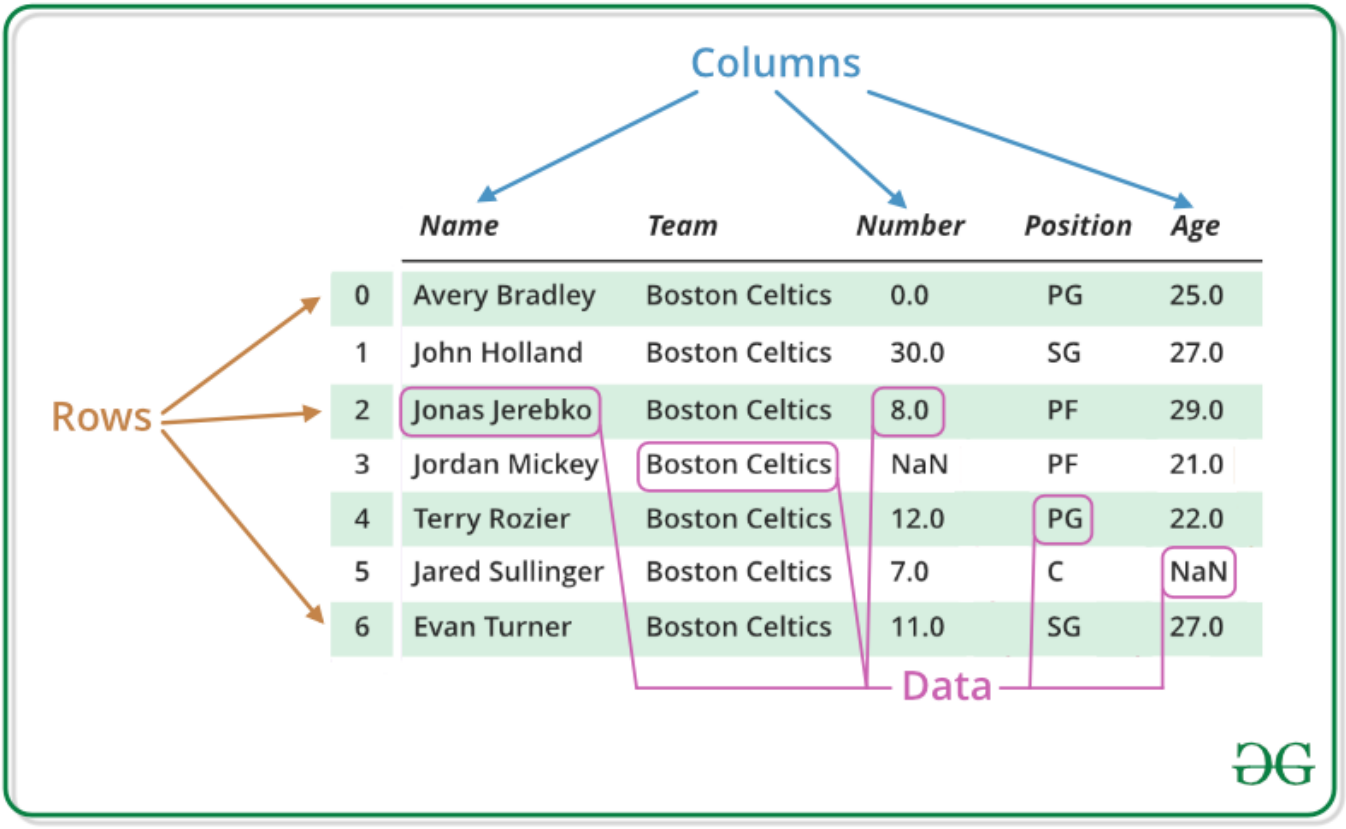
Pandas is a high-level data manipulation tool. It is built on the Numpy package and its key data structure is called the DataFrame. DataFrames allow you to store and manipulate tabular data in rows of observations and columns of variables.

Check this official guide for a started in pandas:

10 minutes to pandas

```
import pandas as pd
```

Pandas DataFrame is two-dimensional size-mutable, potentially heterogeneous tabular data structure with labeled axes (rows and columns). A Data frame is a two-dimensional data structure, i.e., data is aligned in a tabular fashion in rows and columns. Pandas DataFrame consists of three principal components, the data, rows, and columns.



Let's load a publicly available *.csv* dataset into a pandas ***DataFrame***. We will use the popular *iris* dataset.

```
file_name = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"

df = pd.read_csv(file_name)
df.head(n = 10)
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa

A dataframe's **.describe()** method offers descriptive statistics which include those that summarize the central tendency, dispersion and shape of a dataset’s distribution, excluding NaN values.

```
df.describe()
```

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.054000	3.758667	1.198667
std	0.828066	0.433594	1.764420	0.763161
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Let's see some ways in which we can access the DataFrames' data. Each column of a pandas `DataFrame` is a pandas `Series`.

```
df['petal_width']
```

```
0      0.2
1      0.2
2      0.2
3      0.2
4      0.2
      ...
145    2.3
146    1.9
147    2.0
148    2.3
149    1.8
Name: petal_width, Length: 150, dtype: float64
```

We can do any vectorized operation on a `Series`. Moreover, a pandas `Series` allows us to do conditional selection of rows in a `DataFrame`.

```
setosas = df[df['species'] == 'setosa']

setosas.head() # only setosa species selected
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

We can add a new column to a pandas `DataFrame`, simply by specifying its name and its contents.

NB: the data added to the new column must be the same length as the rest of the `DataFrame`.

```
df['sepal_area'] = df['sepal_length'] * df['sepal_width'] # adding new columns
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	5.1	3.5	1.4	0.2	setosa	17.85
1	4.9	3.0	1.4	0.2	setosa	14.70
2	4.7	3.2	1.3	0.2	setosa	15.04
3	4.6	3.1	1.5	0.2	setosa	14.26
4	5.0	3.6	1.4	0.2	setosa	18.00

We can work with `Series` as we work with numpy arrays. We perform Min-Max normalization on the `petal_length` column.

```
# Min-Max Normalization
df['petal_length'] = (df['petal_length'] - df['petal_length'].min()) / (df['petal_length']
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	5.1	3.5	0.067797	0.2	setosa	17.85
1	4.9	3.0	0.067797	0.2	setosa	14.70
2	4.7	3.2	0.050847	0.2	setosa	15.04
3	4.6	3.1	0.084746	0.2	setosa	14.26
4	5.0	3.6	0.067797	0.2	setosa	18.00

We can also use the `.apply()` method on either a `Series` or a `DataFrame` to modify its contents, or create a new column.

```
def capitalize(col):
```

```
  return col.capitalize()

df['species'] = df['species'].apply(capitalize)
df.head()
```

	sepal_length	sepal_width	petal_length	petal_width	species	sepal_area
0	5.1	3.5	0.067797	0.2	Setosa	17.85
1	4.9	3.0	0.067797	0.2	Setosa	14.70
2	4.7	3.2	0.050847	0.2	Setosa	15.04
3	4.6	3.1	0.084746	0.2	Setosa	14.26
4	5.0	3.6	0.067797	0.2	Setosa	18.00

A `DataFrame` also has a `groupby` method, that allows us to work on groupings of rows.

```
df.groupby('species').mean()
```

species	sepal_length	sepal_width	petal_length	petal_width	sepal_area
Setosa	5.006	3.418	0.078644	0.244	17.2088
Versicolor	5.936	2.770	0.552542	1.326	16.5262
Virginica	6.588	2.974	0.771525	2.026	19.6846

We can also iterate through each group. A group is another `DataFrame`.

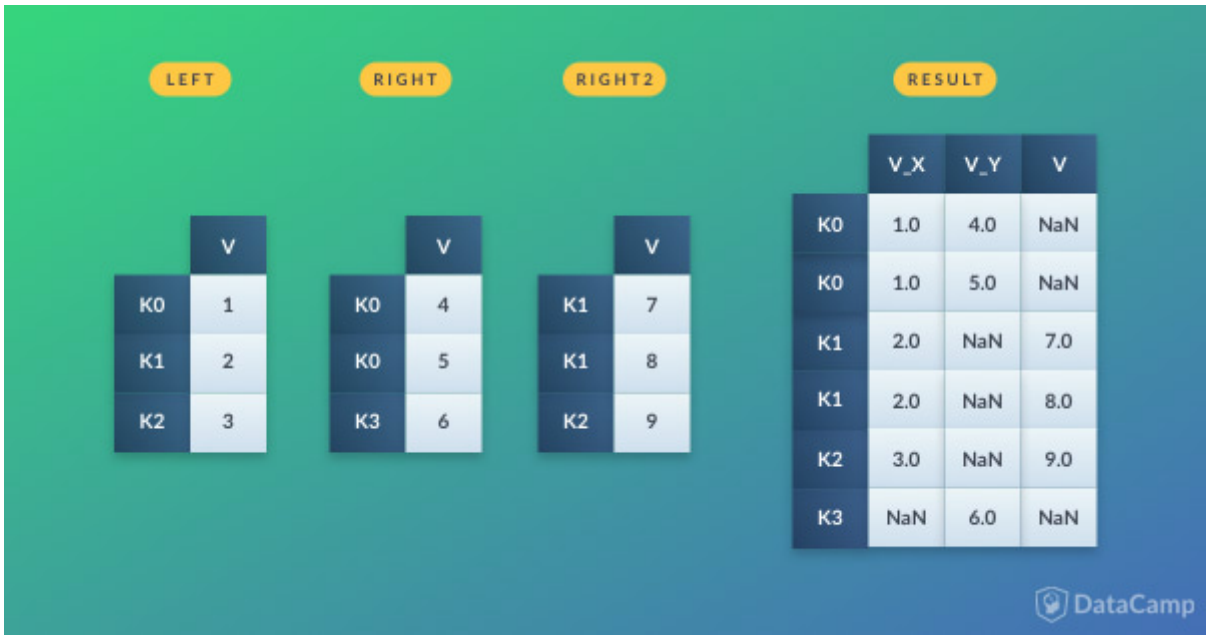
```
for name, group in df.groupby('species'):
  print("Group:", name)
  print(group.head())
  print("-----")
```

```
Group: Setosa
   sepal_length  sepal_width  petal_length  petal_width species  sepal_area
0           5.1          3.5      0.067797          0.2  Setosa       17.85
1           4.9          3.0      0.067797          0.2  Setosa       14.70
2           4.7          3.2      0.050847          0.2  Setosa       15.04
3           4.6          3.1      0.084746          0.2  Setosa       14.26
4           5.0          3.6      0.067797          0.2  Setosa       18.00
-----
Group: Versicolor
    sepal_length  sepal_width  ...     species  sepal_area
50           7.0          3.2  ...  Versicolor       22.40
51           6.4          3.2  ...  Versicolor       20.48
52           6.9          3.1  ...  Versicolor       21.39
53           5.5          2.3  ...  Versicolor       12.65
54           6.5          2.8  ...  Versicolor       18.20

[5 rows x 6 columns]
-----
```

```
Group: Virginica
     sepal_length  sepal_width  ...    species  sepal_area
100           6.3          3.3  ...  Virginica       20.79
101           5.8          2.7  ...  Virginica       15.66
102           7.1          3.0  ...  Virginica       21.30
103           6.3          2.9  ...  Virginica       18.27
104           6.5          3.0  ...  Virginica       19.50

[5 rows x 6 columns]
-----
```

Joins

Pandas allows for joining two or more `DataFrames` together using a common key. We can also do vertical or horizontal concatenation .




```
df1 = pd.DataFrame({'A': ['A0', 'A1', 'A2', 'A3'],
                    'B': ['B0', 'B1', 'B2', 'B3'],
                    'C': ['C0', 'C1', 'C2', 'C3'],
                    'D': ['D0', 'D1', 'D2', 'D3']})
df2 = pd.DataFrame({'A': ['A0', 'A1', 'A4', 'A5'],
                    'B': ['B4', 'B5', 'B6', 'B7'],
                    'C': ['C4', 'C5', 'C6', 'C7'],
                    'D': ['D4', 'D5', 'D6', 'D7']})
df3 = pd.DataFrame({'A': ['A8', 'A9', 'A10', 'A11'],
                    'B': ['B8', 'B9', 'B10', 'B11'],
                    'C': ['C8', 'C9', 'C10', 'C11'],
                    'D': ['D8', 'D9', 'D10', 'D11']})

pd.concat([df1, df2, df3]).reset_index(drop = True)
```

	A	B	C	D
0	A0	B0	C0	D0
1	A1	B1	C1	D1
2	A2	B2	C2	D2
3	A3	B3	C3	D3
4	A0	B4	C4	D4
5	A1	B5	C5	D5
6	A4	B6	C6	D6
7	A5	B7	C7	D7
8	A8	B8	C8	D8

```
pd.merge(df1, df2, on = 'A', how = 'left')
```

	A	B_x	C_x	D_x	B_y	C_y	D_y
0	A0	B0	C0	D0	B4	C4	D4
1	A1	B1	C1	D1	B5	C5	D5
2	A2	B2	C2	D2	NaN	NaN	NaN
3	A3	B3	C3	D3	NaN	NaN	NaN

Saving `DataFrames`

Pandas offers a multitude of methods for saving `DataFrames`.

```
df.to_csv('out.csv', index = False) # saves it locally, check out the files in the right t
```

```
df.to_json()
```

```
'{"sepal_length":{"0":5.1,"1":4.9,"2":4.7,"3":4.6,"4":5.0,"5":5.4,"6":4.6,"7":5.0,"8"
4.8,"13":4.3,"14":5.8,"15":5.7,"16":5.4,"17":5.1,"18":5.7,"19":5.1,"20":5.4,"21":5.1,
0,"26":5.0,"27":5.2,"28":5.2,"29":4.7,"30":4.8,"31":5.4,"32":5.2,"33":5.5,"34":4.9,"3
9":5.1,"40":5.0,"41":4.5,"42":4.4,"43":5.0,"44":5.1,"45":4.8,"46":5.1,"47":4.6,"48":5
9,"53":5.5,"54":6.5,"55":5.7,"56":6.3,"57":4.9,"58":6.6,"59":5.2,"60":5.0,"61":5.9,"6
6":5.6,"67":5.8,"68":6.2,"69":5.6,"70":5.9,"71":6.1,"72":6.3,"73":6.1,"74":6.4,"75":6
7,"80":5.5,"81":5.5,"82":5.8,"83":6.0,"84":5.4,"85":6.0,"86":6.7,"87":6.3,"88":5.6,"8
```

```
print(df.head().to_markdown())
```

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	0.0677966	0.2	Setosa
1	4.9	3	0.0677966	0.2	Setosa
2	4.7	3.2	0.0508475	0.2	Setosa
3	4.6	3.1	0.0847458	0.2	Setosa
4	5	3.6	0.0677966	0.2	Setosa

```
print(df.head().to_latex())
```

```
\begin{tabular}{lrrrrlr}
\toprule
{} &  sepal\_length &  sepal\_width &  petal\_length &  petal\_width & species &  sep
\midrule
0 &           5.1 &          3.5 &      0.067797 &          0.2 &  Setosa &       17
1 &           4.9 &          3.0 &      0.067797 &          0.2 &  Setosa &       14
2 &           4.7 &          3.2 &      0.050847 &          0.2 &  Setosa &       15
3 &           4.6 &          3.1 &      0.084746 &          0.2 &  Setosa &       14
4 &           5.0 &          3.6 &      0.067797 &          0.2 &  Setosa &       18
\bottomrule
\end{tabular}
```

```
print(df.head(n = 3).to_html())
```

```
<table border="1" class="dataframe">
  <thead>
    <tr style="text-align: right;">
      <th></th>
      <th>sepal_length</th>
      <th>sepal_width</th>
      <th>petal_length</th>
      <th>petal_width</th>
      <th>species</th>
      <th>sepal_area</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <th>0</th>
      <td>5.1</td>
      <td>3.5</td>
      <td>0.067797</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>17.85</td>
    </tr>
    <tr>
      <th>1</th>
      <td>4.9</td>
      <td>3.0</td>
      <td>0.067797</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>14.70</td>
    </tr>
    <tr>
      <th>2</th>
      <td>4.7</td>
      <td>3.2</td>
      <td>0.050847</td>
      <td>0.2</td>
      <td>Setosa</td>
      <td>15.04</td>
    </tr>
```

```
  </tbody>
</table>
```

```
df.to_sql(name = '<table_name>', con = '<connection>') # insert into a sql database, works
# check out https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.DataFrame.to
```

```
---------------------------------------------------------------------------
ArgumentError                             Traceback (most recent call last)
<ipython-input-62-6d777a02bc3c> in <module>()
----> 1 df.to_sql(name = '<table_name>', con = '<connection>') # insert into a sql da
      2 # check out https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas

 7 frames
/usr/local/lib/python3.6/dist-packages/sqlalchemy/engine/url.py in _parse_rfc1738_arg
    289     else:
    290         raise exc.ArgumentError(
--> 291             "Could not parse rfc1738 URL from string '%s'" % name
    292         )
    293

ArgumentError: Could not parse rfc1738 URL from string '<connection>'
```

SEARCH STACK OVERFLOW

Pandas Plotting

Pandas offers a convenient API for plotting data directly from a DataFrame. Of course, the plotting API is build upon `matplotlib` as a low level backend. We can use that to manipulate plots as in the previous lab. Check out the official documentation for visualization:

Pandas Plotting Docs

For a quick reference, check the official cookbook.

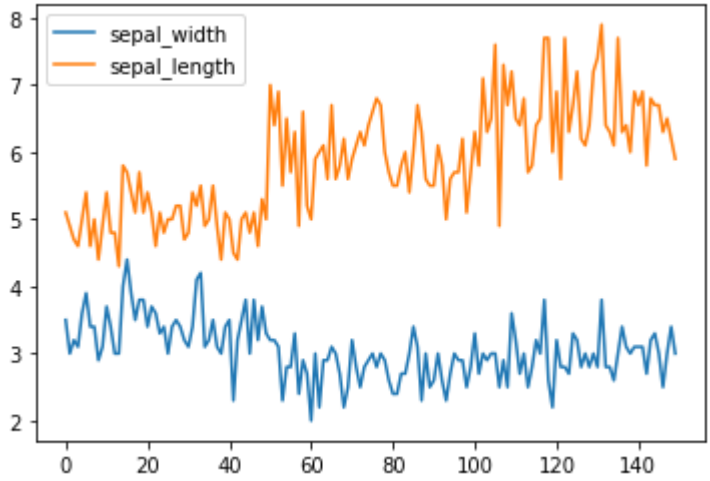
Pandas Plotting Cookbook

We will use the high level plotting API to visualize the Iris Dataset.

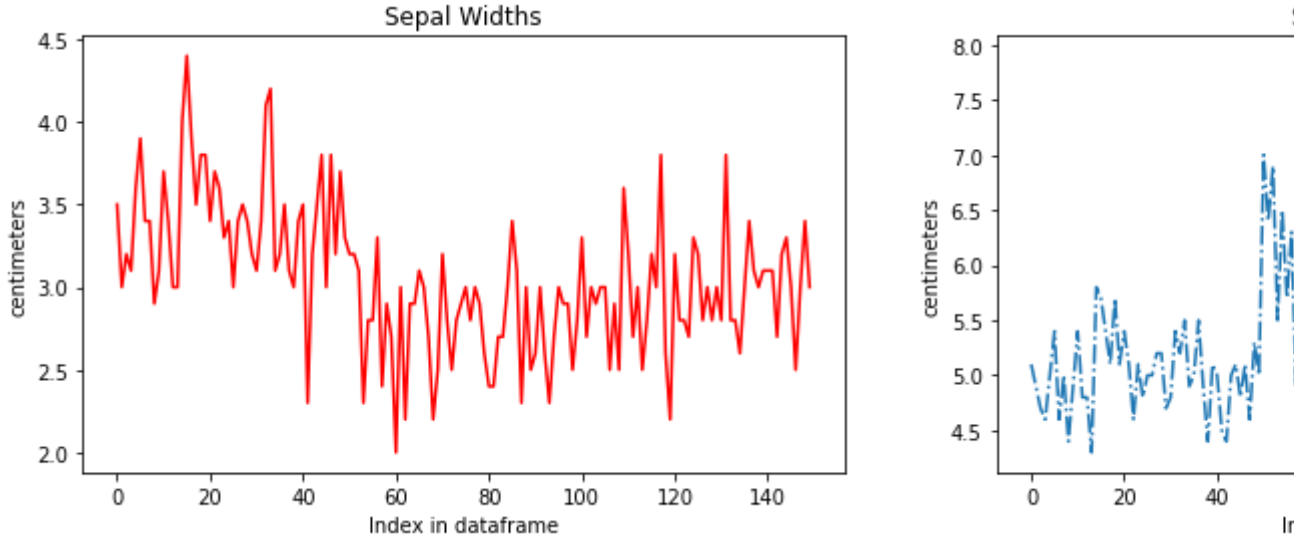
```
df['sepal_length'].plot()
```

```
df[['sepal_width', 'sepal_length']].plot()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd73f126630>
```

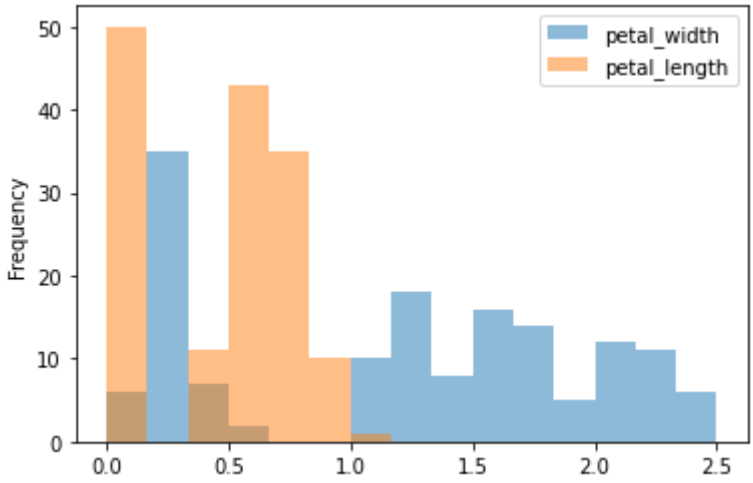



```
fig, ax = plt.subplots(1, 2)

df['sepal_width'].plot(ax = ax[0], color = 'r')
df['sepal_length'].plot(ax = ax[1], linestyle = '-.')

ax[0].set_title('Sepal Widths')
ax[1].set_title('Sepal Lengths')

ax[0].set_xlabel('Index in dataframe')
ax[1].set_xlabel('Index in dataframe')

ax[0].set_ylabel('centimeters')
ax[1].set_ylabel('centimeters')


fig.set_size_inches(15, 4)
```



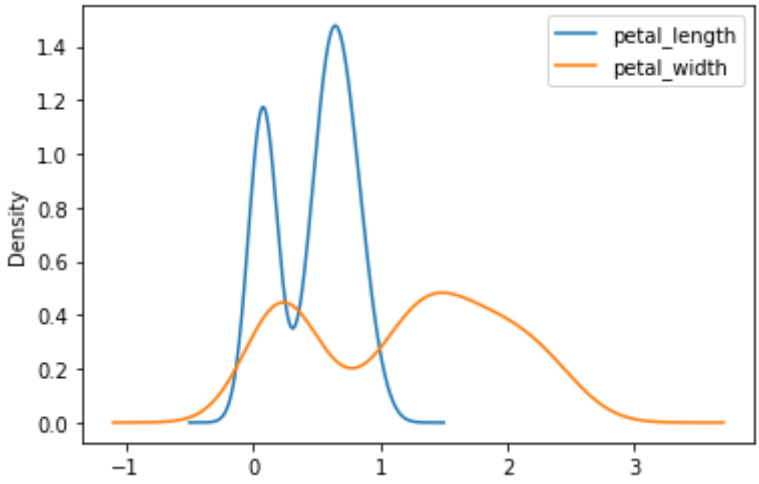

```
df[['petal_width', 'petal_length']].plot.hist(alpha = 0.5, bins = 15)
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fe648bce278>
```




```
df[['petal_length', 'petal_width']].plot.kde()
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fe648ab5748>
```



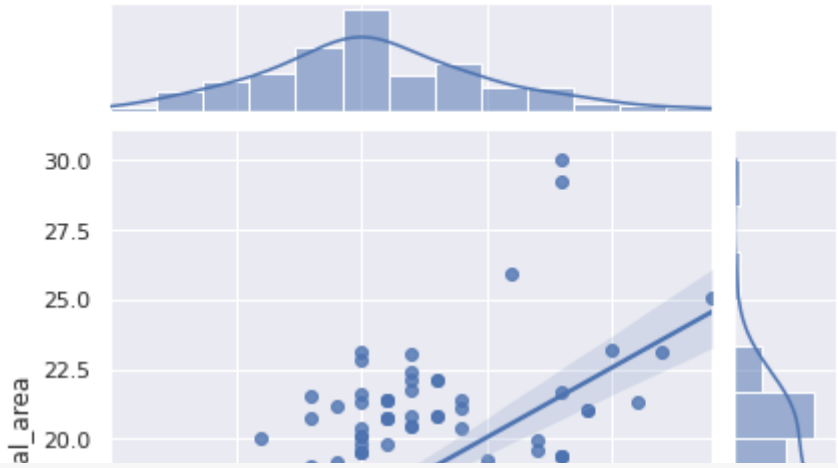
Cool Plots using Seaborn & Pandas

Check out seaborn for more awesome plots.

```
import seaborn as sns

sns.set_theme()
```

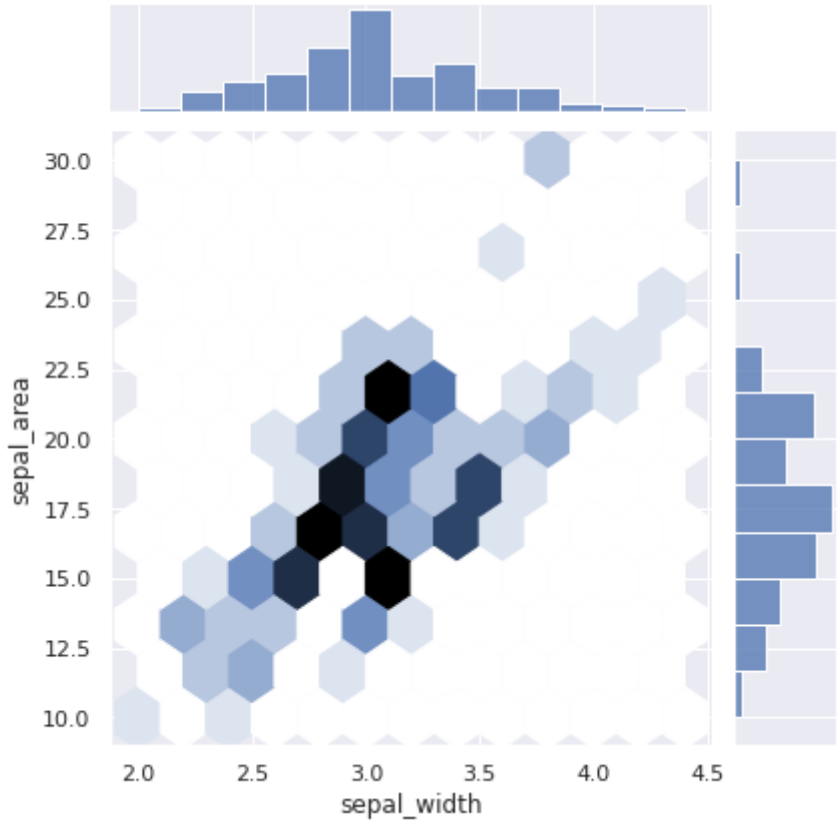
```
sns.jointplot(x = 'sepal_width', y = 'sepal_area', data = df, kind = 'reg')
```

```
<seaborn.axisgrid.JointGrid at 0x7fe64892e940>
```



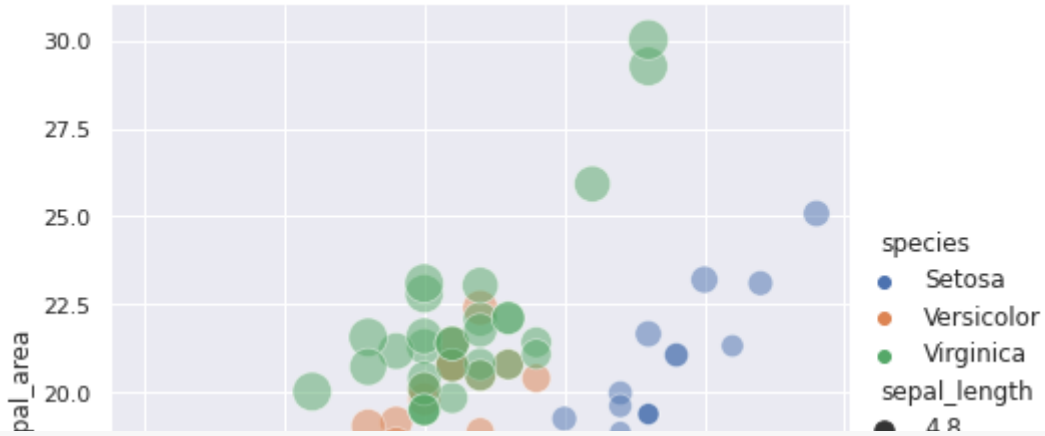

```
sns.jointplot(x = 'sepal_width', y = 'sepal_area', data = df, kind = 'hex')
```

```
<seaborn.axisgrid.JointGrid at 0x7fe63c004828>
```

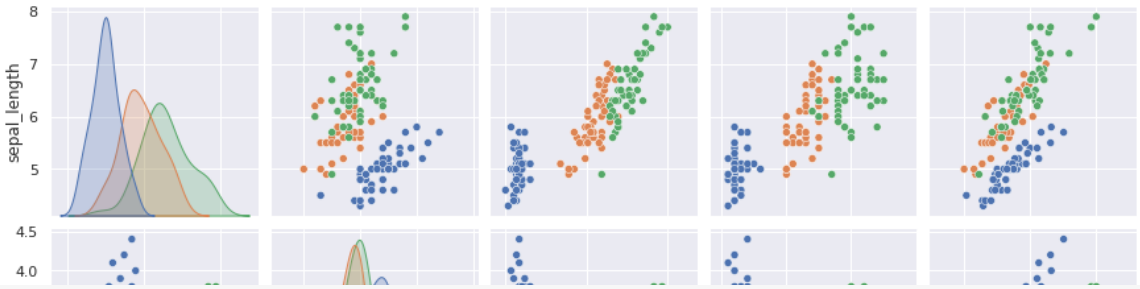



```
sns.relplot(x="sepal_width", y="sepal_area", hue="species", size="sepal_length", sizes=(40
```

```
<seaborn.axisgrid.FacetGrid at 0x7fe63982bfd0>
```

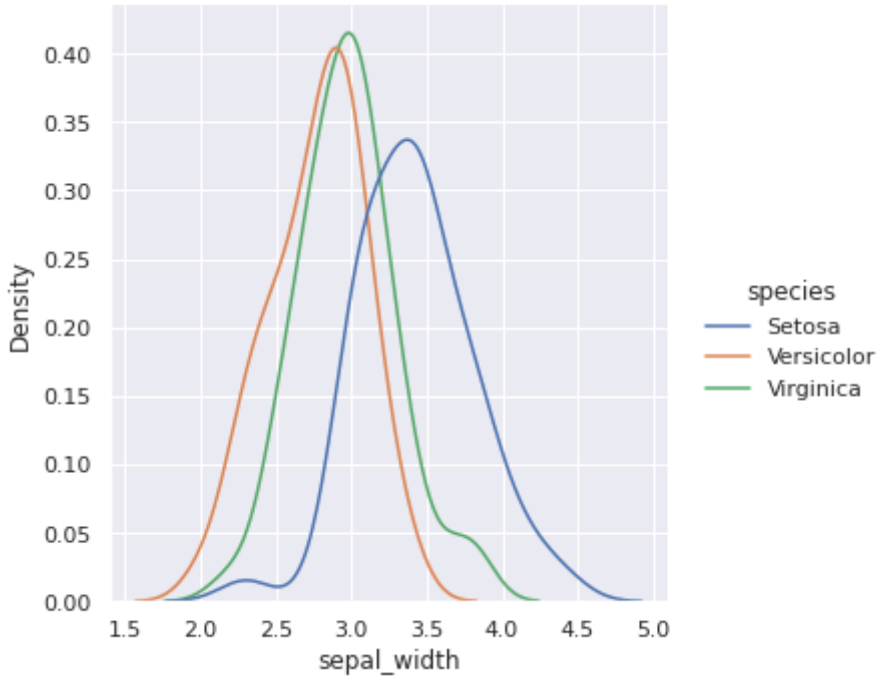



```
grid = sns.pairplot(df, hue = 'species')
```




```
sns.displot(data = df, x = 'sepal_width', kind = 'kde', hue = 'species')
```

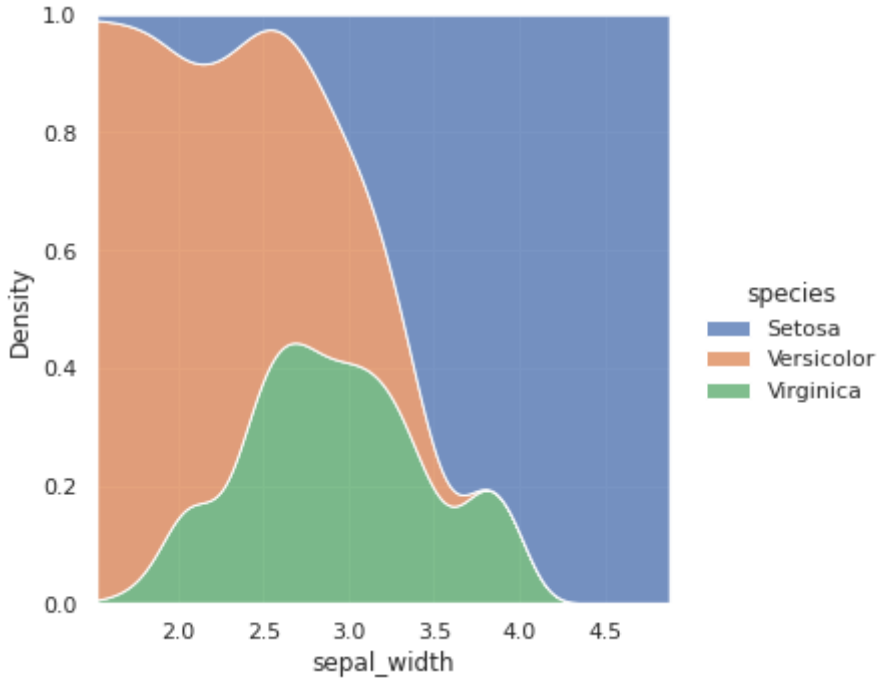
```
<seaborn.axisgrid.FacetGrid at 0x7fe63b4fbc50>
```



sepal_length	sepal_width	petal_length	petal_width	sepal_area

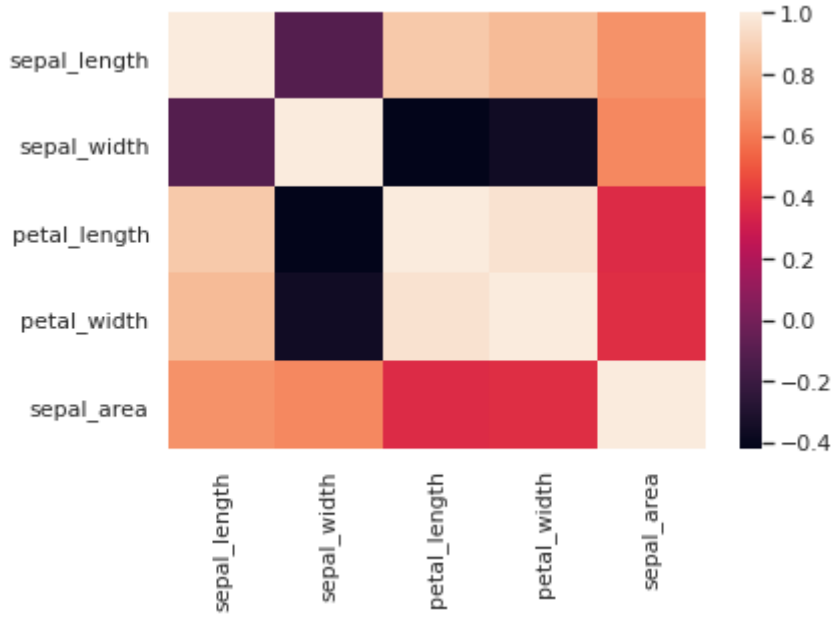
```
sns.displot(data = df, x = 'sepal_width', kind = 'kde', hue = 'species', multiple = 'fill'
```

```
<seaborn.axisgrid.FacetGrid at 0x7fe63b50fe48>
```



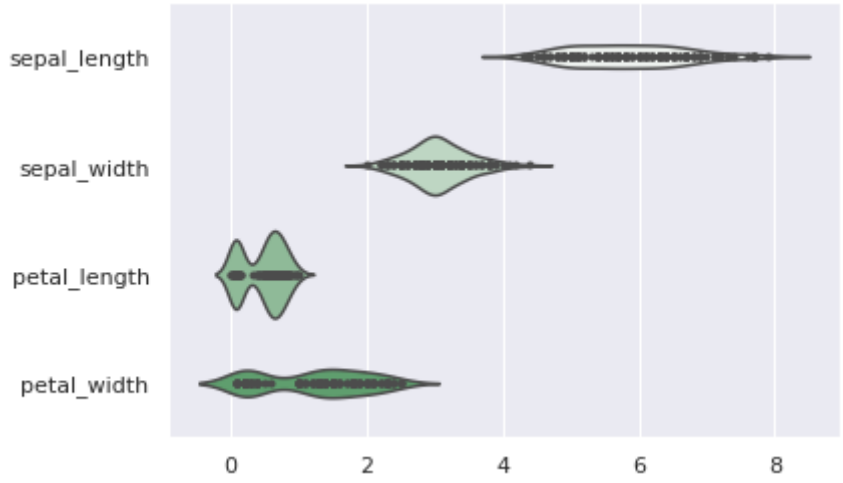

```
sns.heatmap(df.corr())
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd739b23e80>
```




```
sns.violinplot(data=df[['sepal_length', 'sepal_width', 'petal_length', 'petal_width']], pa
```

```
<matplotlib.axes._subplots.AxesSubplot at 0x7fd7396e0cc0>
```

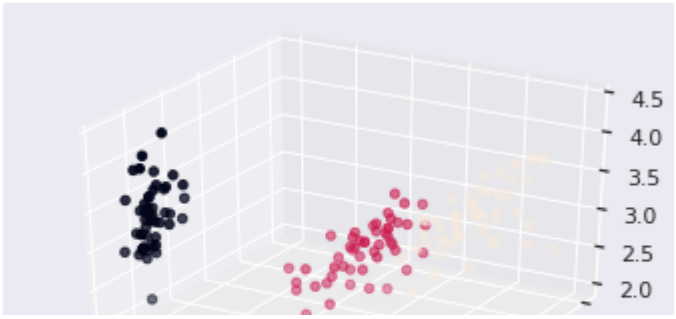


3D Plotting

```
from mpl_toolkits.mplot3d import Axes3D
```

```
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

ax.scatter(df['petal_length'], df['petal_width'], df['sepal_width'], zdir='z', s=20, c=df[
```

```
<mpl_toolkits.mplot3d.art3d.Path3DCollection at 0x7fe6396f3898>
```



Exercises

After you finish your lab exercises, you should export this notebook as **pdf** and upload it to Moodle. (i.e. **File -> Print**, Destintation: Save as PDF).

In this lab, we will look at COVID data for Romania. The data was retrieved from https://graphs.ro/.

```
import requests
import json

covid_data = json.loads(requests.get('https://www.graphs.ro/json.php').content)

covid_df = pd.DataFrame(covid_data['covid_romania'])
covid_df.head()

covid_county_data_dfs = []

for i, row in covid_df.iterrows():
  try: # some days have no county information
    county_df = pd.DataFrame(row['county_data'])
  except:
    continue

  county_df['reporting_date'] = row['reporting_date']
  covid_county_data_dfs.append(county_df)


county_df = pd.concat(covid_county_data_dfs)
covid_df = covid_df.drop(['sourceUrl', 'county_data'], axis = 1)
```

```
covid_df.head()
```

	reporting_date	total_cases	new_cases_today	total_tests	new_tests_today	total
0	2020-11-17	373474	8262	3733410.0	31082.0	

```
county_df.head()
```

	county_id	county_name	county_population	total_cases	reporting_date
0	AB	Alba	323778	8050	2020-11-17
1	AR	Arad	415732	8967	2020-11-17
2	AG	Arges	574920	10279	2020-11-17
3	BC	Bacau	580912	10840	2020-11-17
4	BH	Bihor	559992	11269	2020-11-17

1. Basic Visualizations

Make 4 subplots. Using pandas as seaborn, plot the number of new cases in a day, the number of recovered patients in day, number of tests in a day, and the number of deaths in day. We are trying to explore the evolution of COVID from the start of the pandemic until today.

NB: Make sure to add proper labels, title, axes and legend where necessary.

```
# TODO your code here
```

2. Positive testing percentage

Using `pandas`, create a new column that computes the percentage of positive tests in a given day. This new column should be the number of infected people in a day over the number of tests per day.

Plot the evolution of positive tests across time. Compare this to the number of hospitalized patients.

In a different plot, visualize the correlation between positive tests and the number of intensive care patients.

NB: Make sure to add proper labels, title, axes and legend where necessary.

```
# TODO your code here
```

3. County Information

Select at least 10 counties of your choosing, and plot the evolution of cases across time.

Plot the number of positive cases on 1000 persons for each of your selected counties. Plot a horizontal line at the 3 / 1000 mark. When the number of positive cases per 1000 persons exceeds 3 / 1000, color your points in a different color from that point onwards.

NB: Make sure to add proper labels, title, axes and legend where necessary.

BONUS

Further expore the dataset, and come up with interesting visualizations of the COVID evolution in Romania.

```
# TODO your code here (maybe)
```

Lab 08 - I/O Monitoring (Windows)

You can download the **Windows 10 VM** via ep_win10_vm.7z.torrent.txt. DokuWiki is not configured to accept *.torrent* files so remove the *.txt* extension. After that, you know what to do…

Alternatively, you can install the following on your own Windows machine:

- **ADK [https://go.microsoft.com/fwlink/?linkid=2120254]** - make sure to check ***Windows Performance Analyser*** and ***Windows Performance Recorder***.
- **Visual Studio Community Edition [https://visualstudio.microsoft.com/downloads/]** - select *C++ development*.
- **Sysinternals suite [https://docs.microsoft.com/en-us/sysinternals/downloads/sysinternals-suite]**

Objectives

- How to determine the root sources of having intensive disk usage/RAM/CPU and intensive networking using **Task Manager** (Processes, Performance, App history, Start-up, Users, Details, Services).
- Analyzing performance issues due to intensive disk use using **Windows Performance Recorder**, **Process Monitor** and **Process Explorer**.
- Monitor the disk activity, identify who is generating it and how to figure out the issue by looking at the pdbs and the code.

Introduction

Since computers started to surface, for many people it was a mystery what was happening behind the screen and it seemed magical when it wasn't working and even more magical when it was working. Since Linux is open-source, all sorts of tools appeared over time to analyse problems when they came up. On Windows on the other hand, the system being closed made it harder for tools to appear.

The first tools were provided by Sysinternals. These were written by Mike Rusinovich, who chose to make public tools such as “File monitor” and “Registry monitor”, which were later combined into “Process monitor”. The tools were so good that even Microsoft's support teams were using them. Seeing their usefulness and appreciating the know-how of their operating system, Microsoft decided to buy Sysinternals, so now the original website redirects to https://technet.microsoft.com/en-us/sysinternals [https://technet.microsoft.com/en-us/sysinternals] (outside Romania it probably redirects to a different link due to localization reasons that consider the language of the country where redirection is made).

On this website can be found some of the tools that will be used in this tutorial - Process Monitor, Process Explorer, VMMap, Autoruns. Starting with Windows 7, Microsoft has begun to invest more and more in the performance of the system and in ways to monitor the system's performance. Some tools already existed since Windows 2000, but they were only used internally.

Tutorials & Tasks

Task Manager

Shows the process name responsible for constant disk thrashing either by reads or writes.

To start Task manager use the shortcut: ***Ctrl + Shift + Esc***.

Tabs description:

- **Processes tab** shows all the running processes and their current resource usage in terms of CPU, Memory, Disk and Network.
- **Performance tab** shows the usage level of the computer's main resources in the last 60 seconds.
- The **App history tab** was first added to Windows 8, and it shows the resource consumption of metro applications. Metro applications are touch-screen-friendly applications written especially for Microsoft's WinRT programming interfaces.
- The **Start-up tab** shows all the applications that start at start-up, (or at least in Microsoft's vision - this will be further detailed in the Autoruns section), and their impact on the boot time. It is helpful to check this tab in case your computer takes a long to to start up.
- **Users tab** shows the resource consumption of every logged in user.
- **Details tab** shows details for each process - pid, status, the user under which it runs. Right-clicking the column headers bar, offers the possibility to add or remove columns. The following columns: Handles, Threads, Image Path Name and Command Line are very useful: the first one (Handles) when investigating a handle leak, the second one (Threads) in the case of investigating processes that create too many threads, the third one (Image Path Name) to find out the path from where the process was started, and the last one (Command Line) to find out the parameters with which it was started.
- **Services tab** shows the service status. A Windows service can be considered similar to a Linux daemon: a process without a visual interface, offering services to user-created processes.

Conclusion:

- Task Manager can be used to identify which process uses a lot of RAM, CPU, accesses the disk many times or generates a lot of traffic on the network at a certain moment (Services tab). However, it does not offer information if in the long run, that same process is the one that generated the slowdown of the system. It does offer some information for longer periods of time, in the Start-up tab, which shows what process had higher impact at start-up, but does not specify the area that was impacted (disk space, RAM, CPU).
- You can sort by I/O read or I/O Writes, but no option to sort the results by Total I/O (combined Read & Write).

To overcome Task manager’s limitation, and to perform a thorough analysis, use the excellent Resource Monitor (Resmon) utility, which is built-in to Windows.

Here https://drive.google.com/file/d/1z1J6lgoYfBOZF7acEzR8gEq1MH1OZgaf/view [https://drive.google.com/file/d/1z1J6lgoYfBOZF7acEzR8gEq1MH1OZgaf/view] you have a visual representation of the previous mentioned steps.

01. [20p] Task Manager

1. Watch the video and go through the tutorial
2. Which program is constantly reading or writing to your hard disk?

NON-DEMO TASK

How to:

- Open Task Manager, and select the Details tab.
- Right-click on the column header (Name, PID, Status etc) and click Select Columns.
- Enable the following checkboxes and click OK.

I/O read bytes is the number of bytes read in input/output operations generated by a process, including file, network, and device I/Os. Whereas I/O write bytes is the number of bytes written in input/output operations by a process, including file, network, and device I/Os. I/O Read Bytes & I/O Write Bytes directed to CONSOLE (console input object) handles are not counted.

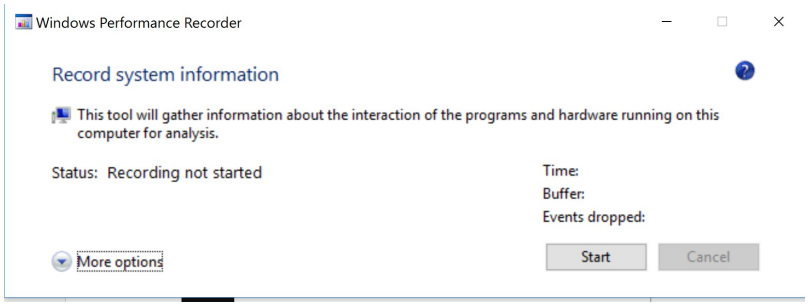
- Next, sort the listings by I/O Read bytes and see which application is generating the maximum I/O (in bytes/sec). Similarly, sort by I/O Write bytes to see which program is writing to the hard disk continuously.
- Once you identify the program, decide if you need the program or not. Leave it as it is if the I/O operations are justified. Else, remove the program or consult its documentation to tweak the settings if any. For instance, one of your browser extensions may cause high disk or CPU usage. You need to isolate the extension, add-on or the browser's feature causing the trouble.

Windows Performance Recorder

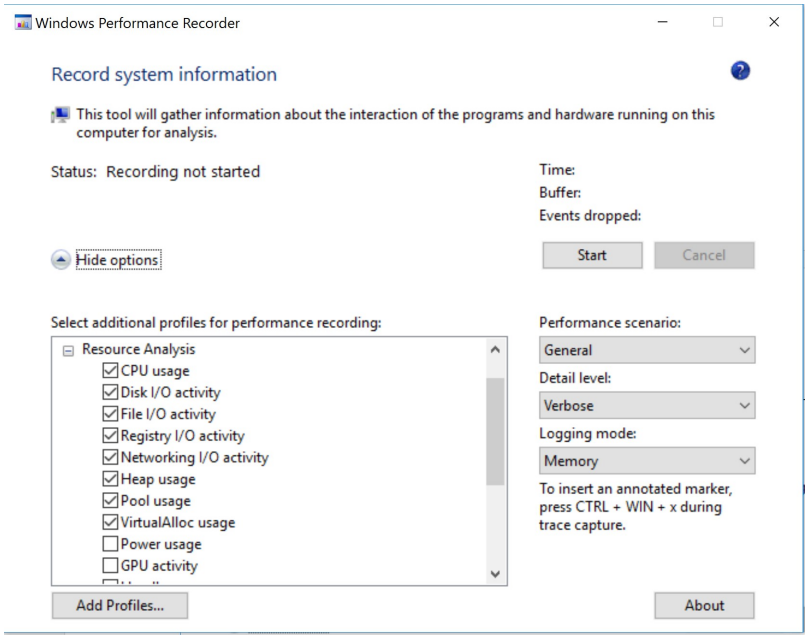
Datafile: Logs.exe and GoodLog.exe

Installing Windows ADK will install Windows Performance Recorder. Check by clicking the windows button and typing "windows performance recorder".

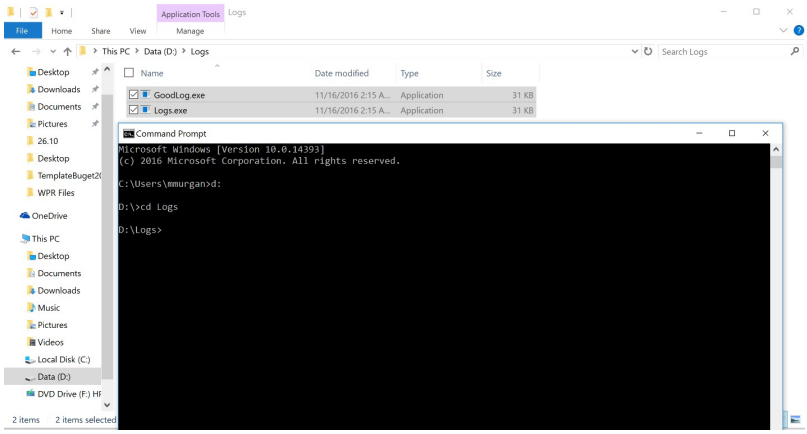
Start **Windows Performance Recorder** by pressing Enter. You will see the following:



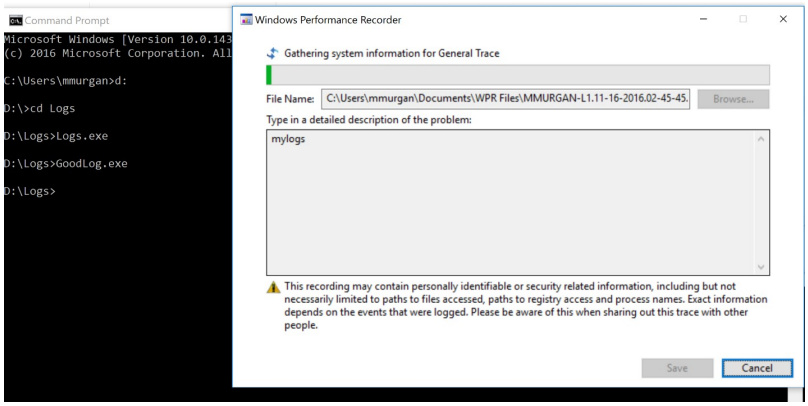
Click the **More options** button to get the list shown in the screenshot right below.



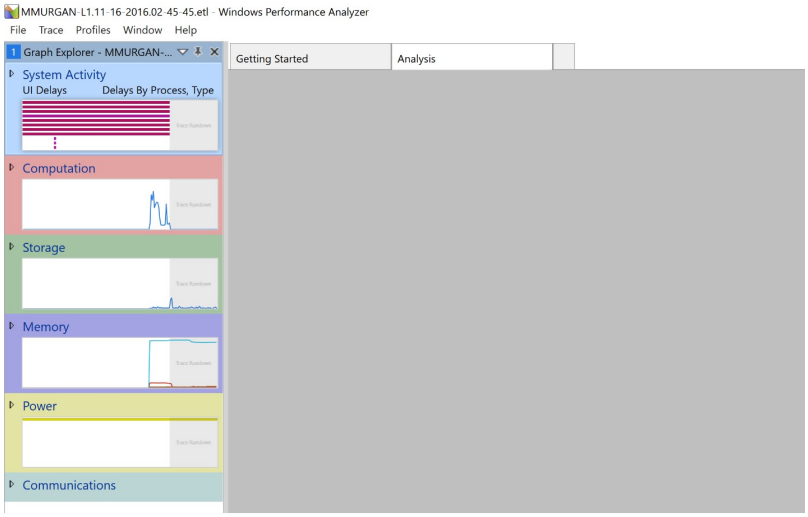
Make sure that you select the same check boxes as in the screenshot, but do not click start just yet. Create a new directory and copy the **Logs.exe** and **GoodLog.exe** files into this directory. The behaviour of these two executables is similar to logging applications that write logs to the disk. Open a terminal and change the path to the directory where you copied the files.



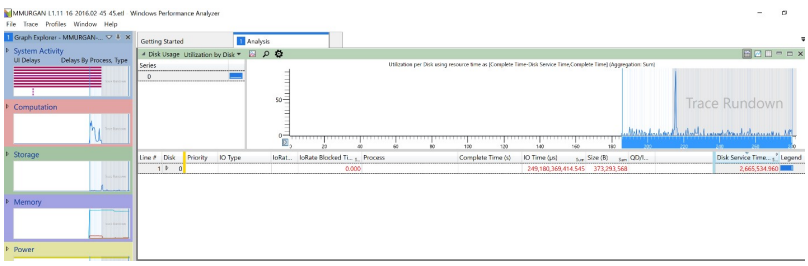
Start **Windows Performance Recorder** and right after run GoodLog.exe and then Logs.exe. Once the two applications finish running, click the Save button in Windows Performance Recorder.



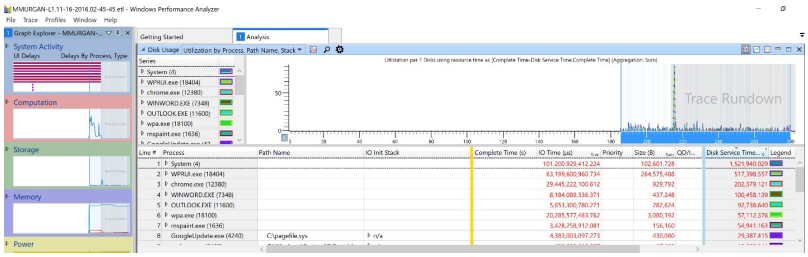
After the capture is saved, the Open option will become available in **Windows Performance Analyzer**. When clicking the Open button it should open a window such as the one below.



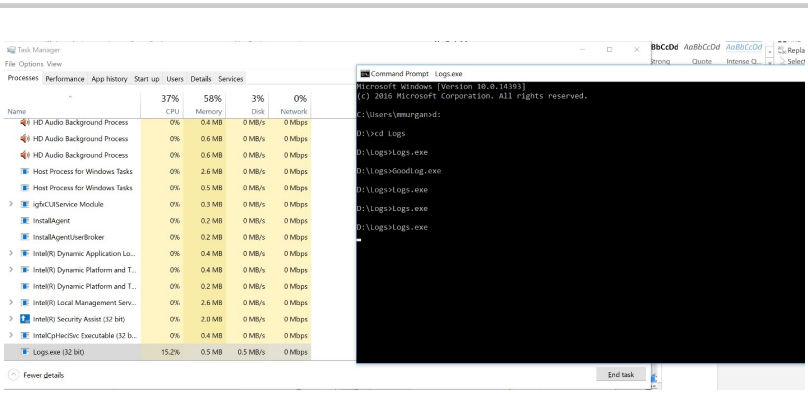
Double clicking on Storage should display the following window. Analyse the resources.



In the upper-left corner of the newly opened window it can select Disk Usage, Utilization by Disk. Click on Utilization by Disk and select: Utilization by Process, Path Name, and Stack. This will generate the following output.



The graph looks interesting in Task Manager. Processes can be selected for observing their activity on the disk. It can be noticed that our processes are not shown. Run Logs.exe again while keeping Task Manager on.



Conclusions:

- This shows that there is activity on the disk. The question is why doesn't Windows Performance Analyzer show it. The way Windows Performance Recorder records activity is based on events generated by the Windows kernel. It registers to track the events, listens to them, and during the recording period it constantly samples which process uses which resource at the time of sampling.
- It sums up the number of time that a process was caught doing something. In our case, the two processes want to write to the disk, but they are not the ones that get to do the actual writing. They tell the system that they want to write, and the System process schedules the writing. The reason for this is targeting a more efficient disk writing, as the System process is trying to minimise the impact to the disk. This is why our process's writing is passed over to the System process.

Here https://drive.google.com/file/d/1DTEnxhv9Tb5TORz1RFT7-v2ojLPW5l7A/view [https://drive.google.com/file/d/1DTEnxhv9Tb5TORz1RFT7-v2ojLPW5l7A/view] you have a visual representation as well.

02. [20p] WPR and WPA

1. Watch the video and go through the tutorial

Process Monitor

Process Monitor is an excellent troubleshooting tool from Windows Sysinternals that displays the files and registry keys that applications access in real-time. The results can be saved to a log file, which you can send it to an expert for analyzing a problem and troubleshooting it.

How to Use Process Monitor to Track Registry and File System Changes?

Step 1: Running Process Monitor & Configuring Filters
1. Download Process Monitor from Windows Sysinternals site.
2. Extract the zip file contents to a folder of your choice.
3. Run the Process Monitor application.
4. Include the processes that you want to track the activity on. For this example, you want to include Notepad.exe in the (Include) Filters.
5. Click Add, and click OK.
6. From the Options menu, click Select Columns.
7. Under "Event Details", enable Sequence Number, and click OK.

You can add multiple entries as well, in case if you want to track few more processes along with Notepad.exe. To keep this example simpler, let's only track Notepad.exe. (You'll now see the Process Monitor main window tracking the list of registry and file accesses by processes real-time, as and whey they occur.)

Step 2: Capturing Events
8. Open Notepad.
9. Switch to Process Monitor window.
10. Enable the "Capture" mode (if it's not already ON). You can see the status of the "Capture" mode via the Process Monitor toolbar.
11. The highlighted button above is the "Capture" button, which is current disabled. You need to click that button (or use Ctrl + E key sequence) to enable capturing of events.
12. Cleanup the existing events list using Ctrl + X key sequence (Important) and start afresh.
13. Now switch to Notepad and try to reproduce the problem.
14. To reproduce the problem (for this example), try writing to HOSTS file (C:\Windows\System32\Drivers\Etc\HOSTS) and saving it. Windows offers to save the file (by showing the Save As dialog) with a different name, or in a different location. So, what happens under the hood when you save to HOSTS file? Process Monitor shows that exactly.
15. Switch to Process Monitor window, and turn off Capturing (Ctrl + E) as soon as you reproduce the problem. Important Note: Don't take much time to reproduce the problem after enabling capturing. Similarly turn off capturing as soon as you finish reproducing the problem. This is to prevent Process Monitor from recording other unneeded data (which makes analysis part more difficult). You need to do all that as quickly as you can.

The log file above tells us that Notepad encountered an ACCESS DENIED error when writing to the HOSTS file. The solution would be to simply run Notepad elevated (right-click and choose "Run as Administrator") to be able to write to HOSTS file successfully.

Step 3: Saving the Output
16. In the Process Monitor window, select the File menu and click Save.
17. Select Native Process Monitor Format (PML), mention the output file name and Path, save the file.
18. Right-click on the Logfile.PML file, click Send To, and choose Compressed (zipped) folder. This compresses the file by ~90%. Look at the graphic below. You certainly want to zip the log file before sending it to someone.

You can take also take a look at this video here:
https://drive.google.com/file/d/1ZYdtOq7QsY0nfYDS3e4foPFRVPgO4Qdb/view
[https://drive.google.com/file/d/1ZYdtOq7QsY0nfYDS3e4foPFRVPgO4Qdb/view].

03. [30p] Process Monitor

Download the archive logs-final.7z and check if you have "Process Monitor" installed (Windows 10).

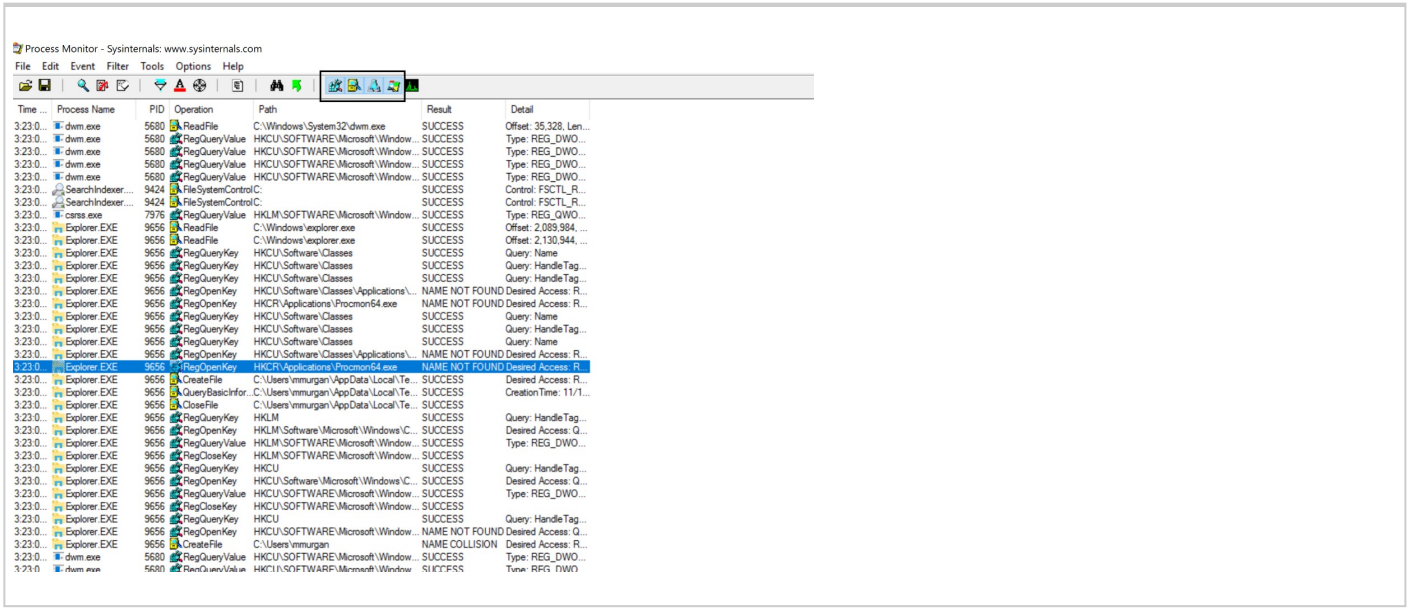
logs-final.7z: **parola**

HandleLeak.7z: **parola7**

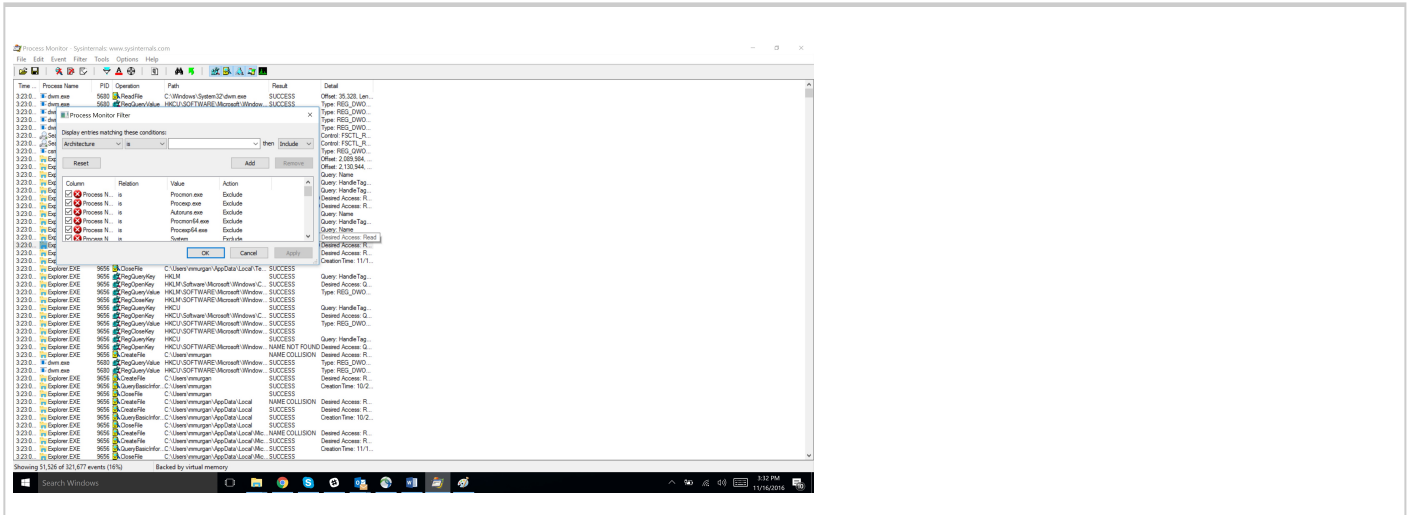
Task.7z: **parola17**

[10p] Task A - Checking logging file

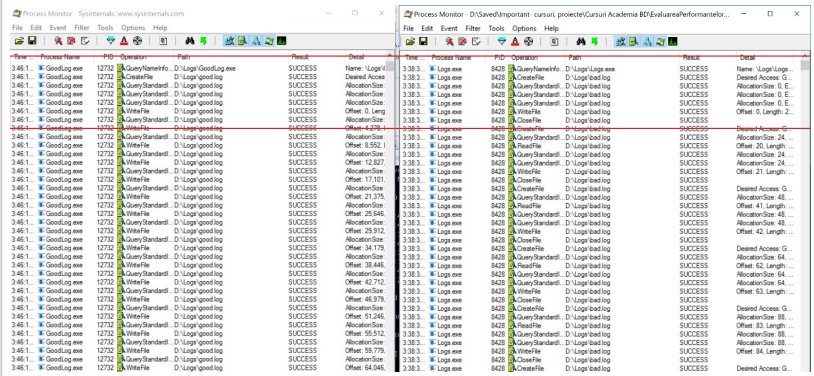
- Looking at the logs created by the two apps in Process Monitor - bad.log, good.log - they are identical, but Logs.exe has a significantly longer running time compared to GoodLog.exe. Start Process Monitor.



- If the 4 buttons in the black area on the upper part of the window are selected, Process Monitor will display the activity (in this order) for: registry, files, networking, process and thread activity. By unchecking them, the corresponding events will be no longer displayed. In the menu bar there is the Filter field. If selected, it will trigger a dropdown menu that contains another Filterfield. If this second Filter field is selected, it will open the window shown below. Replicate this on your computer.

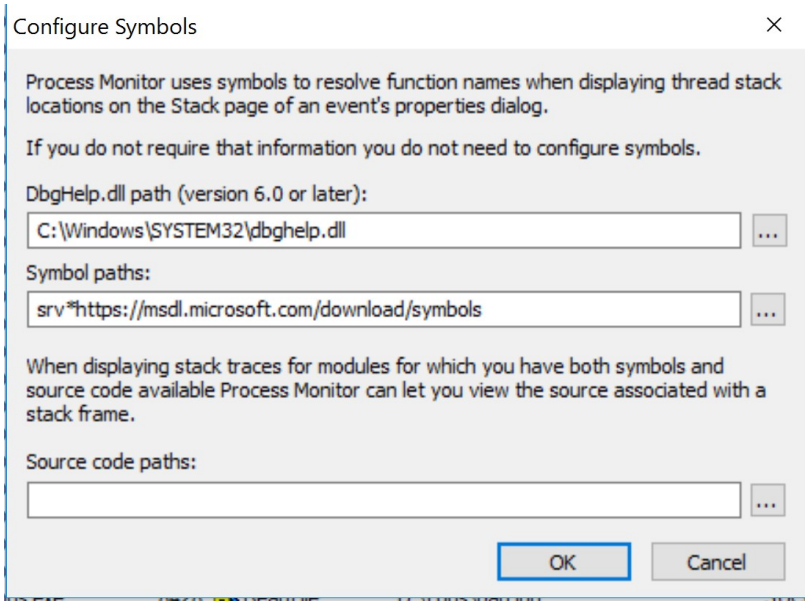


- From the two dropdown menus in the upper part of the context window, select ”Process Name” instead of ”Architecture” and ”is” instead of ”contains”. In the text filed add Logs.exe, click the Add button and then the OKbutton. Open the terminal and run Logs.exe. After the program is done running, save the Process Monitor capture. Use Ctrl + X to reset all the events captured in Process Monitor. Go to Filter → Filter area, double-click on the filter that was just added and change Logs.exe with GoodLog.exe, then click Add and Ok. Start GoodLog.exe and save the capture once the program finishes running. Scroll down in the two capture-logs until you notice the activity for bad.log respectively good.log.

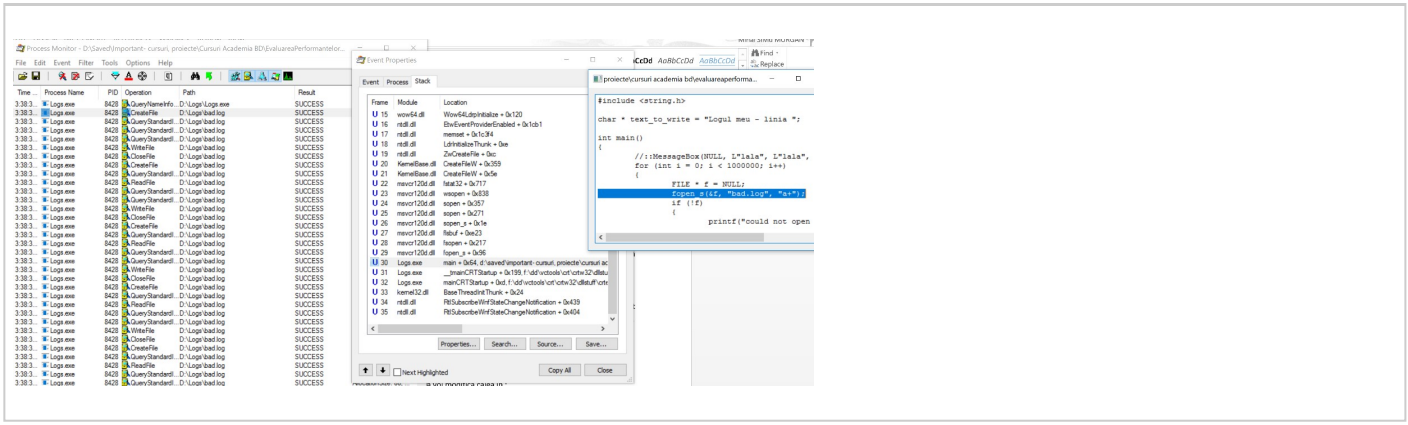


- Notice the difference. On the left-hand side it is shown the faster logging process, and on the right-hand side the slower one. Look in the red highlighted area to see the difference. On the left-hand side the logging file is opened, followed by continuous writing, while on the right-hand side the file is opened and closed for every writing operation which explains the significant slowdown.

- To recap, Task Manager shows what processes use the disk intensively at the current time, Windows Performance Recorder / Windows Performance Analyzer show who used the disc during a longer time period, although they were showing the activity as belonging to the System process instead of our process. Using Process Monitor we could identify our processes' entire activity and we could determine why one is slower than the other. But what if we could find out which line in the code causes the problem? Go back to Process Monitor. Use the window of the badly written logging program (Logs.exe). Go to Options → Configure Symbols, which will open the window shown below.



- In the log (D:\Logs\bad.log) go to CreateFile. Double-click to open the Event Properties window. Choose the Stacktab, scroll down and you can notice that in the main function of main_bad_log.cpp, at line 12 the opening takes place. Click the ”Source” button to view the source code containing the issue.



[20p] Task B - Investigating a handle leak

NON-DEMO TASK

In logs-final.7z you have another example of two executables: **good.exe** and **bad.exe**. Both have the same outcome, the only difference being their running time (one of them is significantly slower). **Identify the problem**.

How to:

- A handle leak consists of a process that opens files and does not close them. On modern computers if this action is performed millions of times, the system may become unresponsive and will either experience an overall slowdown or the application that causes this will eventually crash. You may think that millions of handles are impossible to reach, so it is not worth paying attention to this problem. However, imagine that there are services running on servers for years. As an example, having a handle leak every 2 seconds amounts for over 10 million handle leaks in a year. How should such problems be investigated?
- **Hint:** Open up a terminal and run HandleLeak.exe. Check out the ”Details” tab in Task Manager after adding the ”Handles” column.

04. [30p] Process Explorer

NON-DEMO TASK

- It can be noticed that the number of handles keeps growing. This is clearly a problem, but how do we investigate it?

How to:

- Run it as administrator. It is similar to Task Manager. Select the process that you are interested in, namely HandleLeak, and press “Ctrl + H”.
- “Ctrl + H” opens a window under the ”Process” section that displays all open handles along with information about them. Thus it will display file handles, registry handles, threads handles, and so on. There is another view (Ctrl + D) that displays all the loaded dlls.
- So it can be noticed that the leaks are on the following file: D:\Logs\HandleLeak\leak.txt. This is very useful information, but it would be better to find out who is responsible for the leak in code. Run Process Monitor with a filter on HandleLeak.exe and to notice the stack where the leakage is happens.

05. [10p] Feedback

NON-DEMO TASK

- Please take a minute to fill in the **feedback form [https://docs.google.com/forms/d/e/1FAIpQLSfsMBl2EFu10jJG2qHEiSsR-qYr3wkzQPfDwjhChKnjRtDT_w/viewform]** for this lab.

POWERED BY Bitdefender®

ep/labs/08.txt · Last modified: 2020/12/02 18:12 by radu.mantu

Lab 09 - Memory, CPU and Network Monitoring (Windows)

You can download the **Windows 10 VM** via ep_win10_vm.7z.torrent.txt. DokuWiki is not configured to accept *.torrent* files so remove the *.txt* extension. After that, you know what to do…

Alternatively, you can install the following on your own Windows machine:

- **ADK [https://go.microsoft.com/fwlink/?linkid=2120254]** - make sure to check ***Windows Performance Analyser*** and ***Windows Performance Recorder***.
- **Visual Studio Community Edition [https://visualstudio.microsoft.com/downloads/]** - select *C++ development*.
- **Sysinternals suite [https://docs.microsoft.com/en-us/sysinternals/downloads/sysinternals-suite]**

Objectives

- How to **find the root cause if the system runs out of memory** with TaskManager, Windows Performance Recorder, Windows Performance Analyzer, Visual Studio, VMMap.
- **Monitoring the CPU usage** with TaskManager, Windows Performance Recorder, Windows Performance Analyzer.
- **Check the amount of network traffic generated by processes** with TaskManager, Windows Performance Recorder, Microsoft Network Monitoring, Wireshark.

Introduction

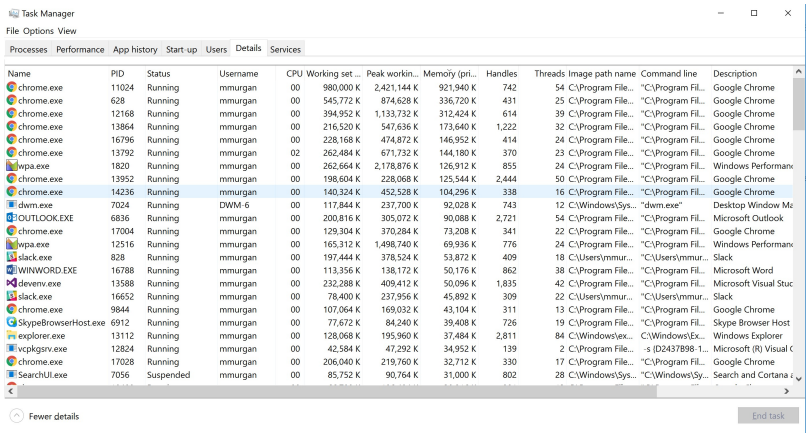
- The Windows operating system comes with plenty of built-in tools to analyze resource usage. The most prominent one is probably the Windows Task Manager, as it highlights resource usage of individual processes, and gives admins and users options to kill any misbehaving ones.
- The Performance Monitor and Resource Monitor are two additional tools that admins and experienced Windows users may use to analyze performance or resources related issues on Windows PCs.

Tutorials/Exercises

The password for log2.zip and build.zip is: *parola*

01. [30p] RAM Monitoring

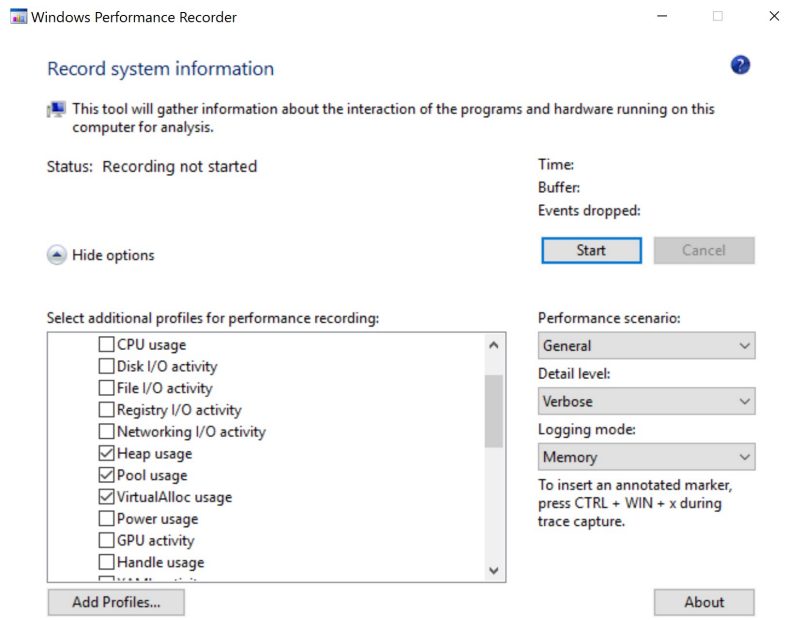
The memory usage by processes is dynamic, thus it is possible to have memory allocation spikes. **Task Manager** shows the amount of memory allocated to a process. To see this information, check the **Peak Working Set** value in the Details tab. It can be noticed that sometimes **Peak Working Set** can be significantly larger than **Working Set**.



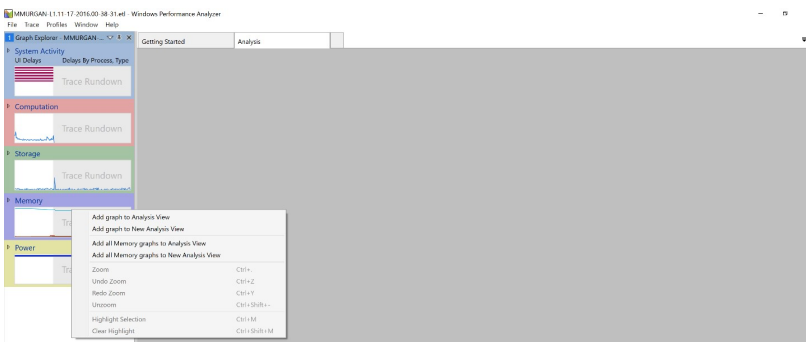
[15p] Task A - Identify the problem

- What can you do if the system runs out of memory, but you do not know what causes this?

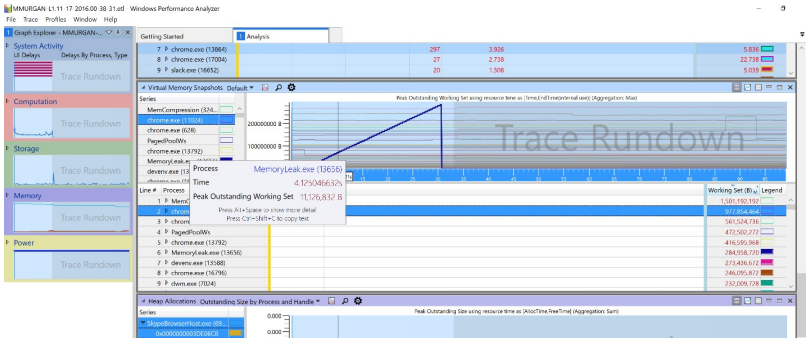
Imagine a situation when a system encounters issues if some conditions are met, and these conditions can be reproduced. To find out the source of this problem, start **Windows Performance Recorder** configured as shown below.



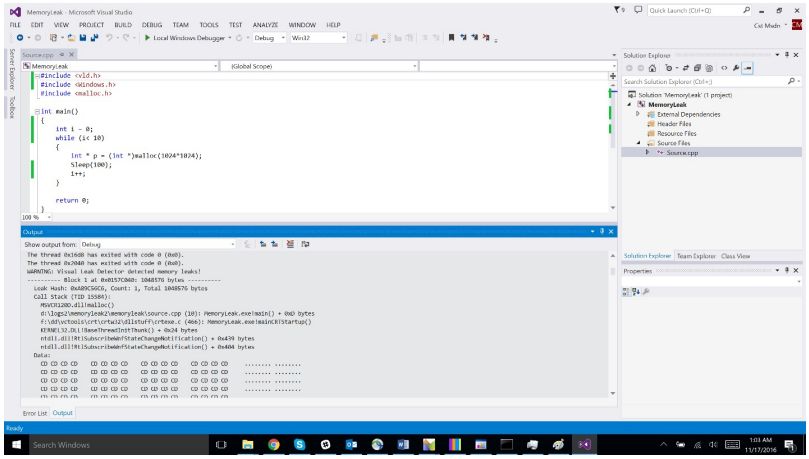
Run in parallel a program that allocates 1MB of memory every 100 milliseconds for a while and then stops. After the program stops running, save the capture and open it in **Windows Performance Analyzer**. You should get the following.



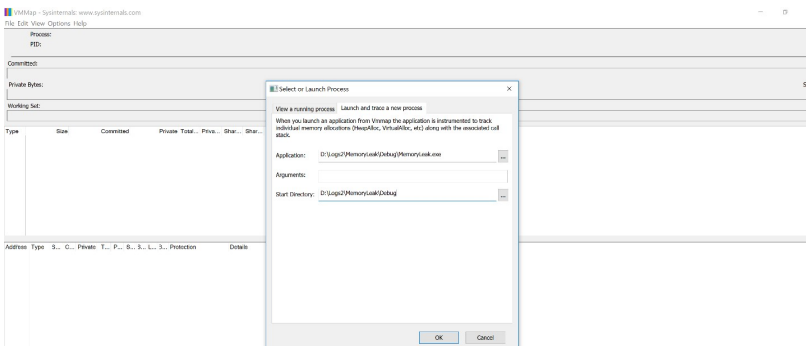
Click **Add all Memory graphs to Analysis View** and scroll down to the **Virtual Memory Snapshot** graph which shows the memory usage for all processes.



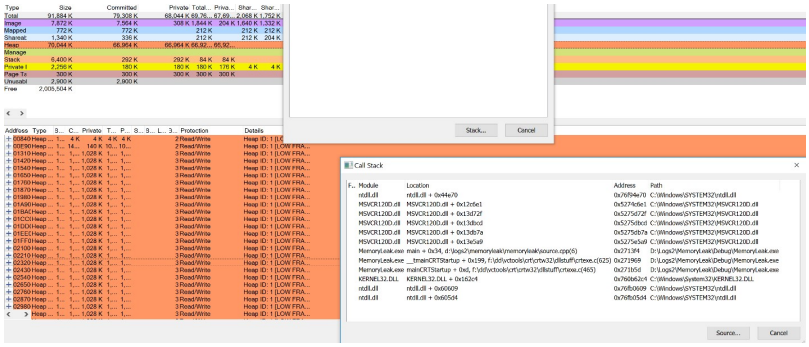
After installing, it requires including the *vld.h* file. When writing the code, the following functions need to be overwritten: malloc, free, new, and delete. This allows each memory allocation and deallocation to be tracked. All the detected leakages (having an allocation that is not followed by a deallocation) will be saved in a log file that can be viewed after the program stops running. In the bottom part of the screenshot shown below, it can be noticed where the allocation took place and that it is not followed by a deallocation.



However, it might be the case that this is not a memory leak, and somewhere at the end of the program the memory gets unallocated. If this is the case, how can you determine which part of the program is responsible for generating the spike? This requires using another tool from SysInternals, VMMap [https://technet.microsoft.com/en-us/sysinternals/vmmap.aspx]. With this tool you can view a process's memory allocations and usage. Use it to run the program that allocates 1 MB of memory every 100 milliseconds.



Prior to starting the tool, go to Options, Configure Symbols and set the paths to the program's, to the Microsoft Symbol Server, and to the program's source files. Start the tool, select **Launch and trace a new process**, select the process, select the directory where it will run, and let it run. You will see something similar to the screenshot below. To view the latest memory allocations, you need to double-click **Heap** in the upper-part of the screenshot, and hit F5 (refresh) from time to time. In the bottom part of the screenshot, you can view the memory allocations. If you click one and press **Heap Allocations** you can see the stack where the allocation occurred. By pressing the "Source" button, you can view the actual code for the allocation.



[15p] Task B - Conclusions

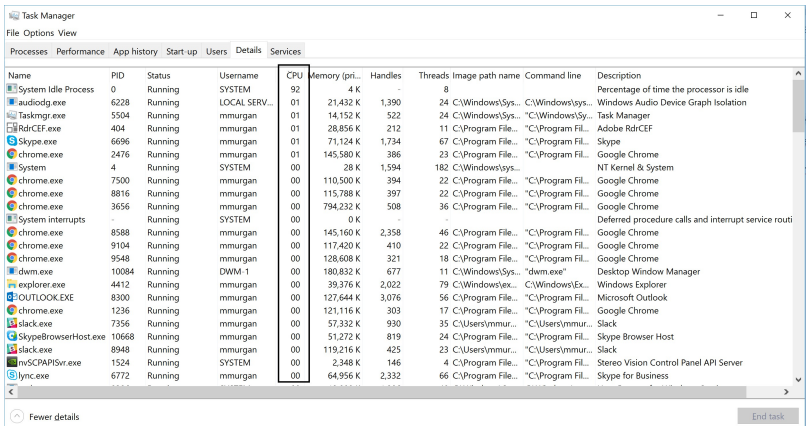
NON-DEMO TASK

- Provide screenshots with your output and add your explanation.

02. [30p] CPU Monitoring

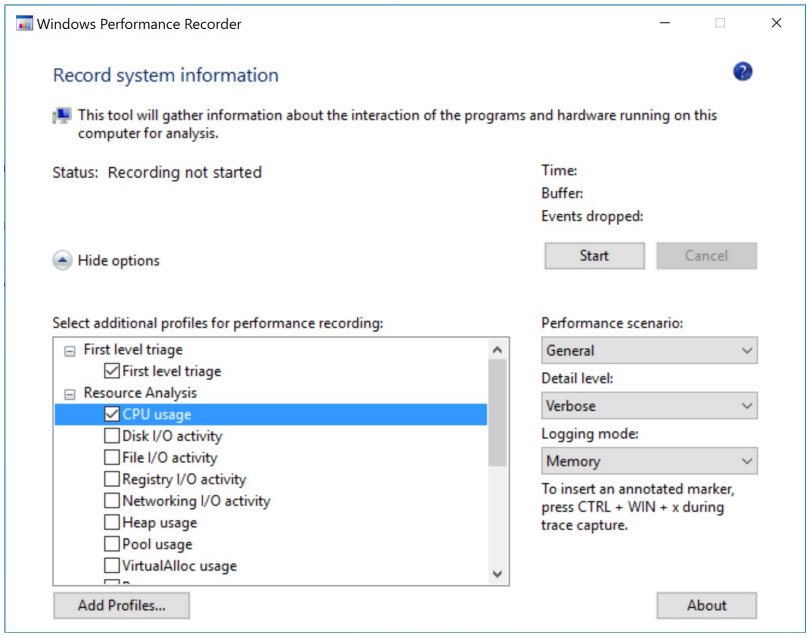
Task Manager

Monitoring the CPU usage presents similar issues to the ones encountered when monitoring the memory usage. Task Manager can help find out the current CPU usage for a process.



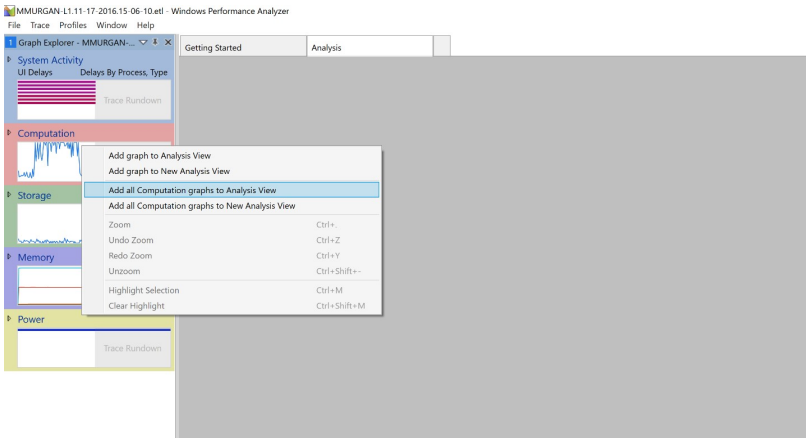
Windows Performance Recorder

Considering the same scenario as the one presented in the previous section: what if the CPU usage briefly spikes up and then goes back to normal, how can you catch this? Using Task Manager would mean having someone continuously watching what is happening to catch the moment when the spike occurs. Use Windows Performance Recorder, with the same settings as in the screenshot below.

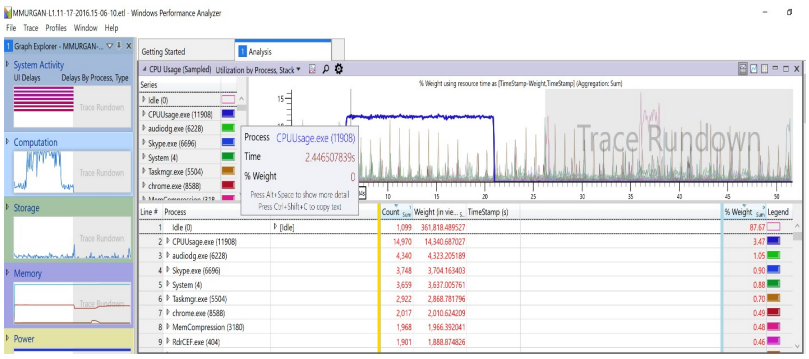


Windows Performance Analyzer

After clicking start, run CPUUsage.exe (part of the resources provided at the end of the tutorial), which generates CPU usage. Save and open the catch.



Right-click on to the Computation area and click “Add All Calculation graphs to Analysis View”.



The graph shows the impact of CPUUsage.exe, which inflicts a 12% CPU usage. Part of the problem is solved, it was determined who is generating the CPU usage. Now, to further debug the situation, as in the previous case of the memory, if this process was not written by you, check if it is useful, and if not, make sure to stop it. If it is useful, but it's not yours, you can try to find an update to fix the problem, or report the problem to the producer. If the program is written by you (this course - Performance Evaluation - targets the processes written by us), then it is important to determine what causes this problem. Unfortunately, unlike in the case of monitoring the memory usage, there is no tool that shows

the stack with the problem, so you need to create one. Open EvenimenteProcMon, which has the purpose of integrating your messages with ProcessMonitor so they can be viewed as the process unfolds. It is necessary to understand any code, not perfectly, but at least to get the big picture of what is going on.

A ProcessMonitor class with 5 functions was created:

- **OpenProcMon** opens up a handle for the ProcessMonitor's message interface.
- **CloseProcMon** closes this handle.
- **ProcMonLog** writes the message that is passed as a parameter to the ProcessMonitor interface.
- **MyProcMon** is the class constructor. It is called when a MyProcMon object is declared.
- **~ MyProcMon** is the class destructor. It is called to destroy the MyProcMon object.

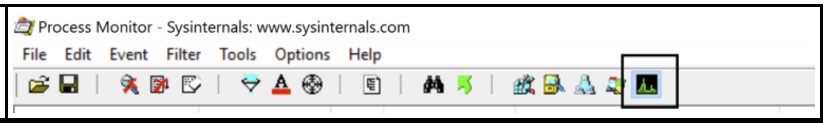
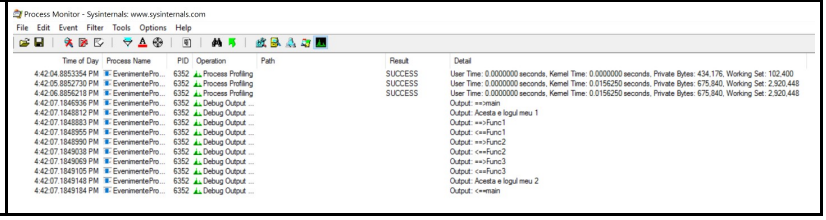
The code below highlights that it was declared globally:

```
MyProcMon __procMon;
```

This means that at the start of the process, before executing the main function, when the global variables are initialized, our class instance will be constructed along with the implicit handle for the ProcessMonitor message interface. The handle is closed when the object is destroyed, after the program's execution ends.

Another class was declared, ProcMonLogFunc, with the purpose of highlighting when entering and leaving a function. This led to defining the following macro, which declares a ProcMonLogFunc object and passes it the name of the current function as a parameter.

```
#define DBGTRACE_FN_ () ProcMonLogFunc __my_log __ (__ FUNCTIONW__)
```

Start ProcessMonitor and change the filter to ProcessName contains EvenimenteProcMon. Select the profiling button as shown below.	
Running the program, generates a Process Monitor capture like this one.	

Notice messages such as Output: ==⇒ Func1 and Output: ⇐== Func1, with the associated times for these events in the left-hand side of the screenshot, in the Time of Day column. The difference between the times (4:42:07.1848883 and 4:42:07.1848955) is 72, and since the times after the comma are expressed in hundreds of nanoseconds, this means that func1 took 7.2 microseconds.

As it is inefficient to calculate by hand the times for each function, save the output in csv format (File → Save and choose the "Comma-Separated Values" option). The generated file will look like this:

```
"4:42:07.1846936 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: ==>main"
"4:42:07.1848812 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: Acesta e logul meu 1"
"4:42:07.1848883 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: ==>Func1"
"4:42:07.1848955 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: <==Func1"
"4:42:07.1848990 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: ==>Func2"
"4:42:07.1849038 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: <==Func2"
"4:42:07.1849069 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: ==>Func3"
"4:42:07.1849105 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: <==Func3"
"4:42:07.1849148 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: Acesta e logul meu 2"
"4:42:07.1849184 PM","EvenimenteProcMon.exe","6352","Debug Output Profiling","","","Output: <==main"
```

Making a parser in Python would make it easy to notice in which of the functions was spent the most time. If you only want to take into account the CPU usage, you need to have logging messages before and after every I/O operation, in order to not count in their time.

NON-DEMO TASK

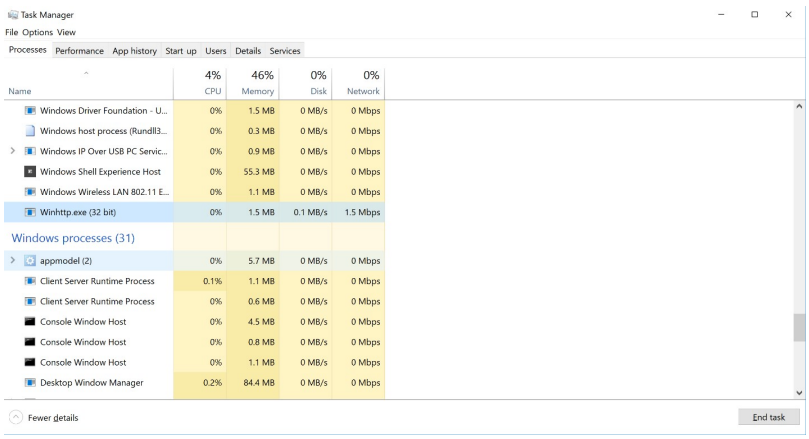
Integrate CPUUsage with ProcessMonitor and find out the total time spent in every function.

03. [40p] Network Monitoring

Task A [20p] - Go through tutorial

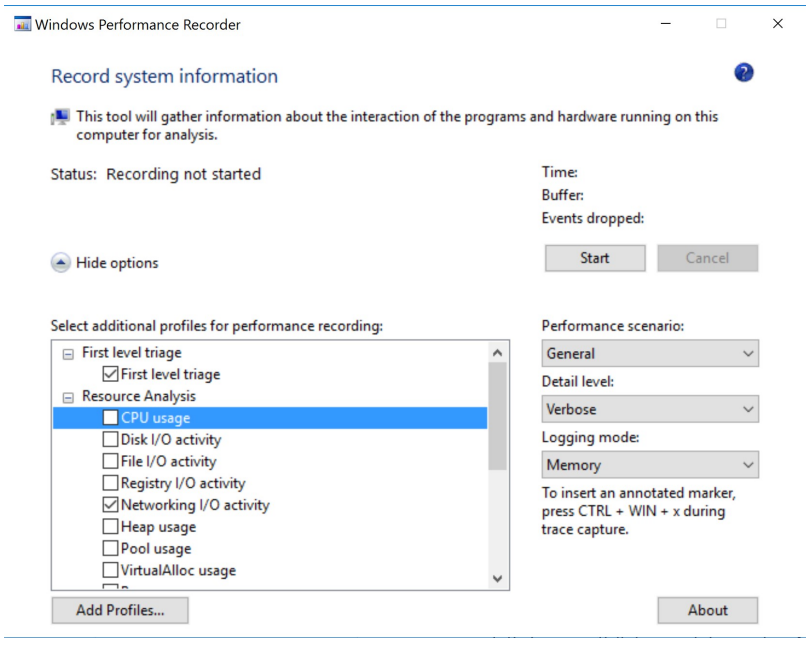
Task Manager

The amount of network traffic generated by a process can be seen using Task Mananger.



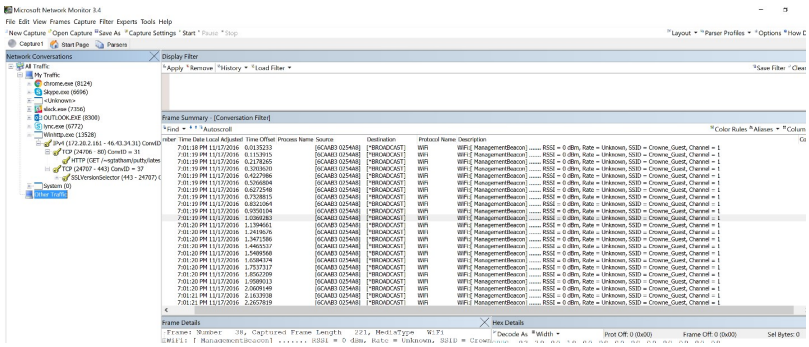
Windows Performance Recorder

The resources for this tutorial include Winhttp.exe, a program that downloads putty.exe. The above screenshot displays its network activity. However, if the process generating the network activity is unknown, you can use Windows Performance Recorder with the following settings. Save and open the capture to view it. The statistics offered by Windows Performance Analyzer are for the total use of the network, rather than per process statistics.



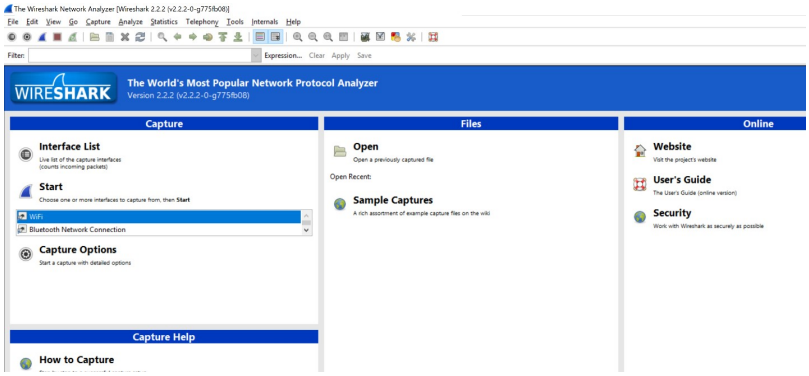
Microsoft Network Monitoring

For this reason, we are calling upon another tool developed by Microsoft. Install it, start it using “Run as administrator”, and select the network interface through which the traffic is expected to pass (cable, wifi, …). You should get a capture such as this one:

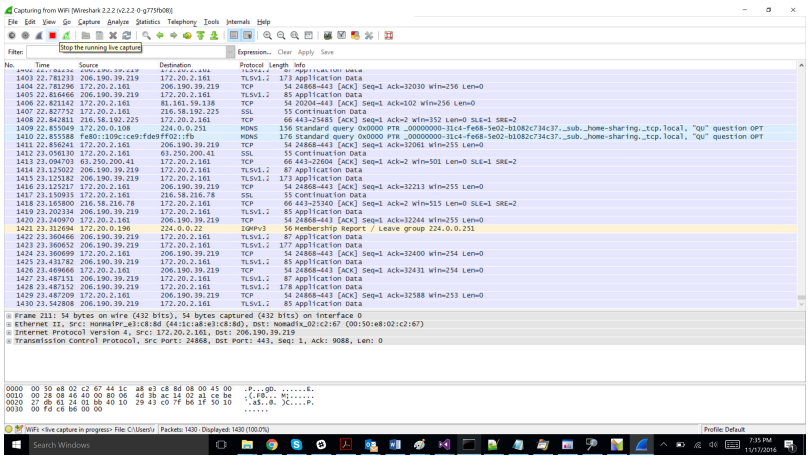


Wireshark

As in the case of the CPU, inspecting the events taking place on the network involves some amount of work for the analyst. However, this being a simple case, you can just expand the view on the traffic generated by Winhttp.exe, and notice the request for *putty.exe*. If it is not clear why some requests are there or why they last so long, you can integrate the application that you wish to investigate with ProcessMonitor. This way you can insert logging elements to find out what request are made and how long they take. The part with timing the requests and traffic can be determined straight from Network Monitor by considering the times of the packets. For displaying all traffic on a http connection (it can also be https as long as you control the server, but this in not in the scope of this tutorial), you can use another tool, Wireshark [https://www.wireshark.org/download.html]. Install Wireshark (**64bit!!!**) accepting the default settings. Start it and select the interface that you want to listen to.



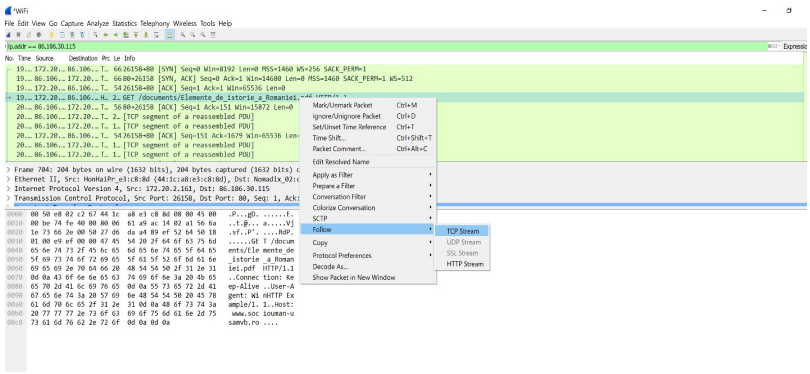
Click the *Start* button and run Winhttp.exe. After Winhttp.exe stops, click the Stop button in Wireshark.



This way you have obtained a traffic capture while winhttp.exe was running. Viewing the code for winhttp.exe, it can be noticed that it makes a request to www.sociouman-usamvb.ro [http://www.sociouman-usamvb.ro]. Use the ping command to get the IP address for this url.

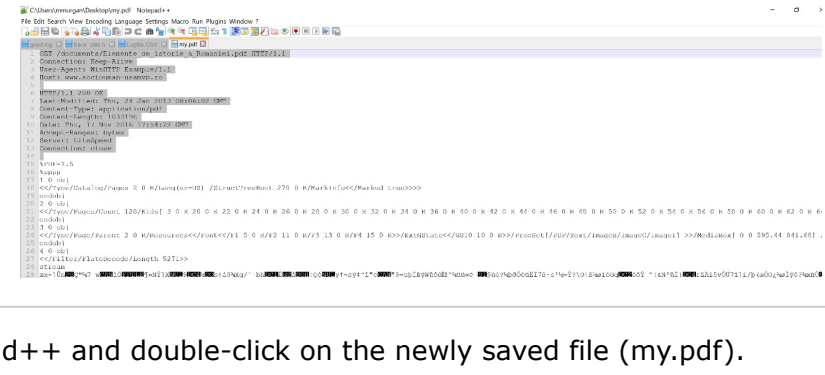
```
D:\Logs2\Winhttp\Debug>ping www.sociouman-usamvb.ro

Pinging sociouman-usamvb.ro [86.106.30.115] with 32 bytes of data:
Reply from 86.106.30.115: bytes=32 time=143ms TTL=48
Reply from 86.106.30.115: bytes=32 time=143ms TTL=48
Reply from 86.106.30.115: bytes=32 time=143ms TTL=48
Reply from 86.106.30.115: bytes=32 time=144ms TTL=48
```

Switching back to Wireshark, add a filter for ip.addr = 86.106.30.115 (make sure to use the IP address identified using ping command). Right click Get documents and choose Follow TCP Stream.

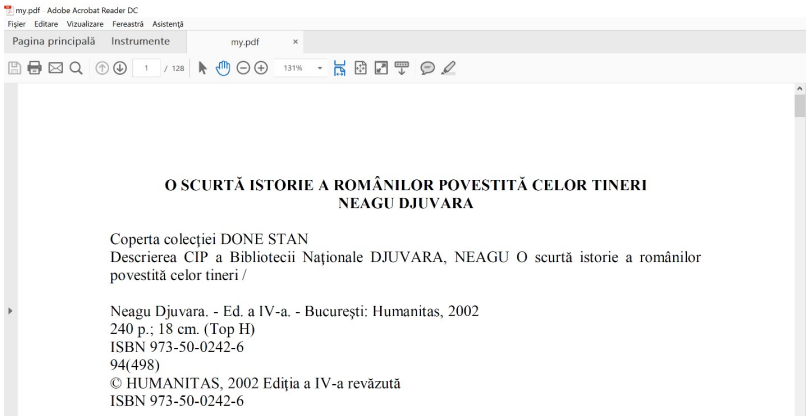


In the bottom part of the Wireshark window, at the "*Show and save data as*" option choose "Raw". Save the capture (using the "Save as" button) as "*my.pdf*".

Use Notepad++ to open the my.pdf file and remove the headers as shown in the screenshot below.



Save it, close Notepad++ and double-click on the newly saved file (my.pdf).



Task B [20p] - Conclusions

NON-DEMO TASK

- Follow the steps above and provide screenshots of the pdf file.

04. [10p] Feedback

NON-DEMO TASK

- Please take a minute to fill in the **feedback form [https://docs.google.com/forms/d/e/1FAIpQLSfsMBl2EFu10jJG2qHEiSsR-qYr3wkzQPfDwjhChKnjRtDT_w/viewform]** for this lab.

POWERED BY Bitdefender®

Lab 10 - Machine Learning

Objectives

- Understand basic concepts of machine learning
- Remember examples of real-world problems that can be solved with machine learning
- Learn the most common performance evaluation metrics for machine learning models
- Analyse the behaviour of typical machine learning algorithms using the most popular techniques
- Be able to compare multiple machine learning models

Exercises

The exercises will be solved in Python, using various popular libraries that are usually integrated in machine learning projects:

- Scikit-Learn [https://scikit-learn.org/stable/documentation.html]: fast model development, performance metrics, pipelines, dataset splitting
- Pandas [https://pandas.pydata.org/pandas-docs/stable/]: data frames, csv parser, data analysis
- NumPy [https://numpy.org/doc/]: scientific computation
- Matplotlib [https://matplotlib.org/3.1.1/users/index.html]: data plotting

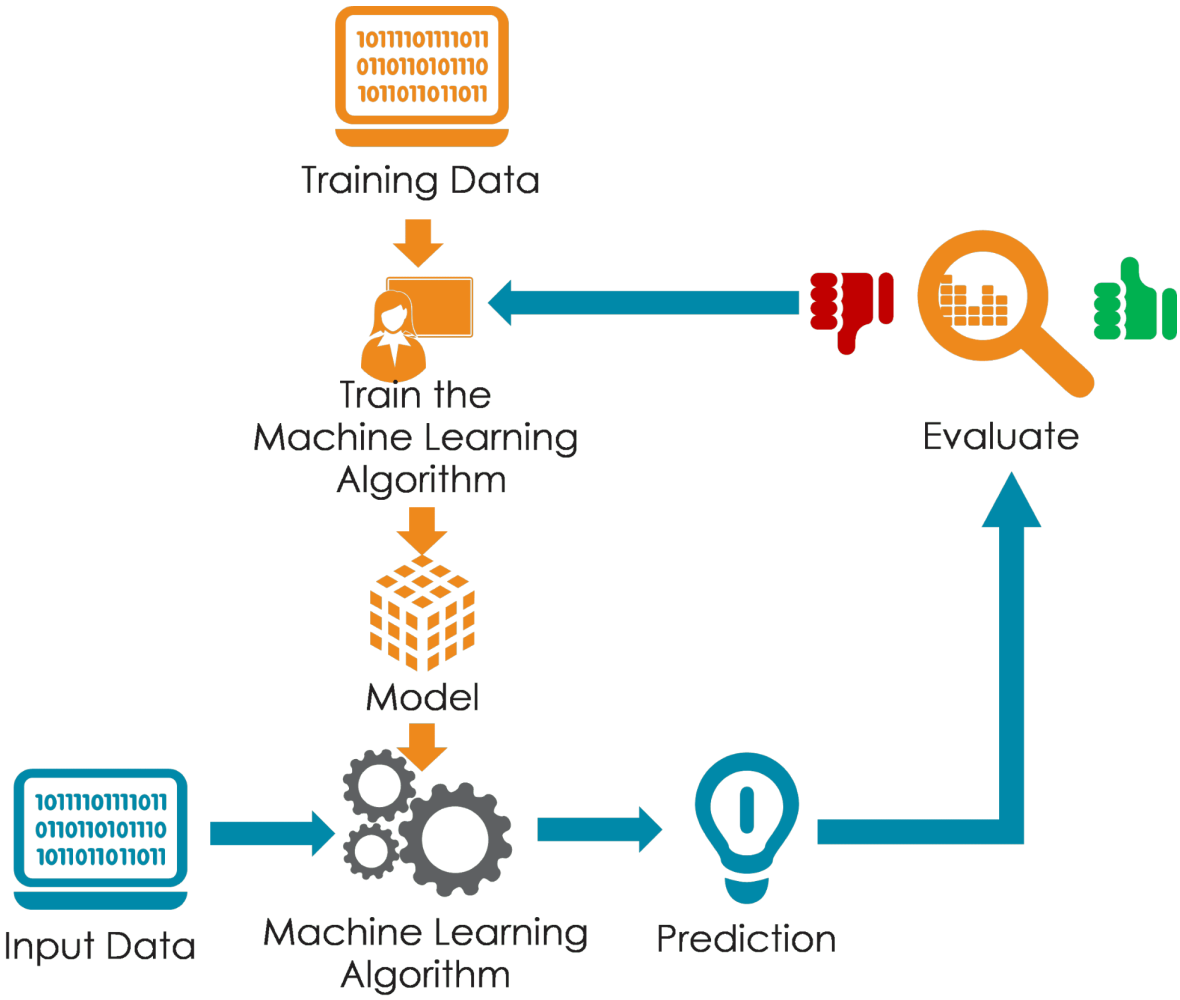
All tasks are tutorial based and every exercise will be associated with at least one "**TODO**" within the code. Those tasks can be found in the *exercises* package, but our recommendation is to follow the entire skeleton code for a better understanding of the concepts presented in this laboratory class. Each functionality is properly documented and for some exercises, there are also hints placed in the code.

Because the various **tasks** and **exercises** are **spread throughout the laboratory text**, they are marked with a ⚠ emoji. Make sure you look for this emoji so that you don't miss any of them!

1. Introduction

Just like chess players improve their technique by watching hundreds of games from top players, computers can be able to perform certain tasks by "looking" at data. These computers are sometimes called "**machines**" and this data observation step is also known as "**learning**". Simple, isn't it? So let's define "**Machine Learning**" (ML) more formally, as stated by Tom Mitchell. A program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E. Let's get more **-h** and provide an example. Say that T is the process of playing chess, E is a set of chess matches on which the algorithm was trained and P is the probability for the program to win the next game of chess.

What it's truly important however, is that machine learning algorithms, unlike traditional ones, are not required to be pre-programmed explicitly to solve a particular task. Just like in the previous example, they **learn** from the data fed into them and afterwards, are able to **predict** results for new scenarios. As you might expect, this isn't just black magic. The majority of ML algorithms are based on some maths and statistics, but also on a fair amount of engineering. 😎



https://medium.com/@tekaround/train-validation-test-set-in-machine-learning-how-to-understand-6cdd98d4a764 [https://medium.com/@tekaround/train-validation-test-set-in-machine-learning-how-to-understand-6cdd98d4a764]

Machine learning algorithms learn from data and they **don't** have to be programmed explicitly!

2. Supervised Learning vs Unsupervised Learning

Generally, in machine learning, there are two types of tasks: **supervised** and **unsupervised**.

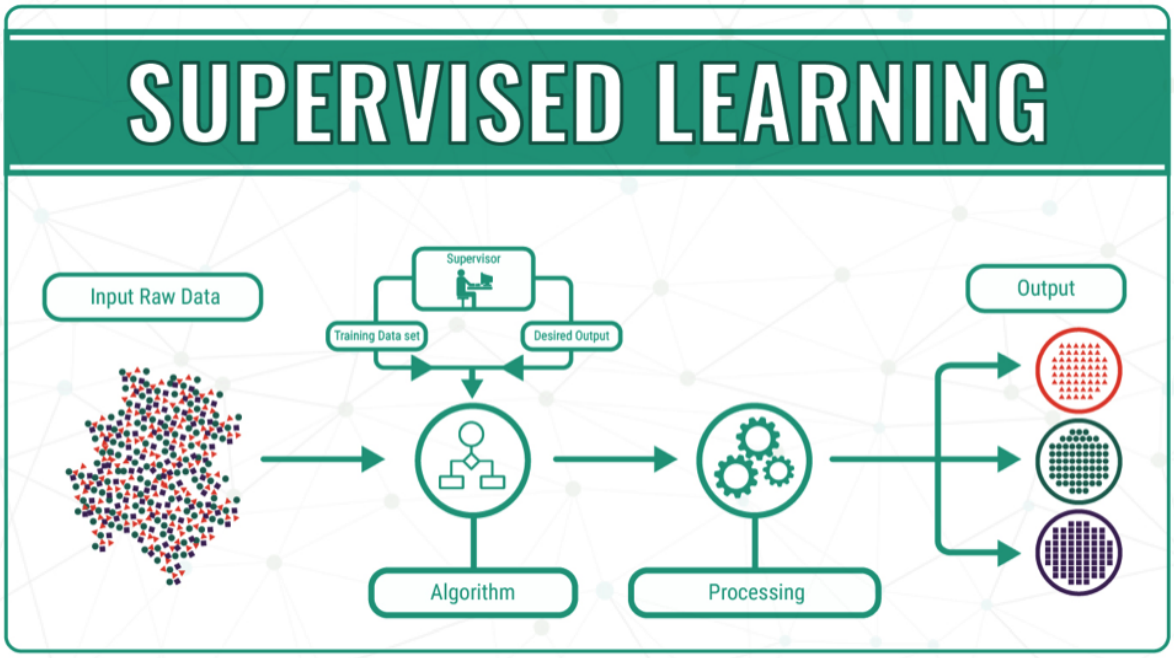
A. Supervised Learning

We refer to the former when we have prior knowledge of what the output of the system should look like. More formally, after going through lots of (X, y) pairs, the model should be able to determine that **mapping function ĥ** which approximates **f(X)=y** as accurately as possible. In machine learning, the entire set of (X, y) pairs is often called **corpus** or **dataset** and an algorithm that learned from this corpus is sometimes called a **trained model** or simply a **model**.

For example, let's say that you want to build a smart car selling platform and you want to provide sellers with the ability to know what should be a "fair price" based on the **specs** and **age** of their cars. For that, you would need to provide your model with a set of tuples that might look like this:

(bhp, fuel_type, displacement, weight, torque, suspension, car_age, car_price)

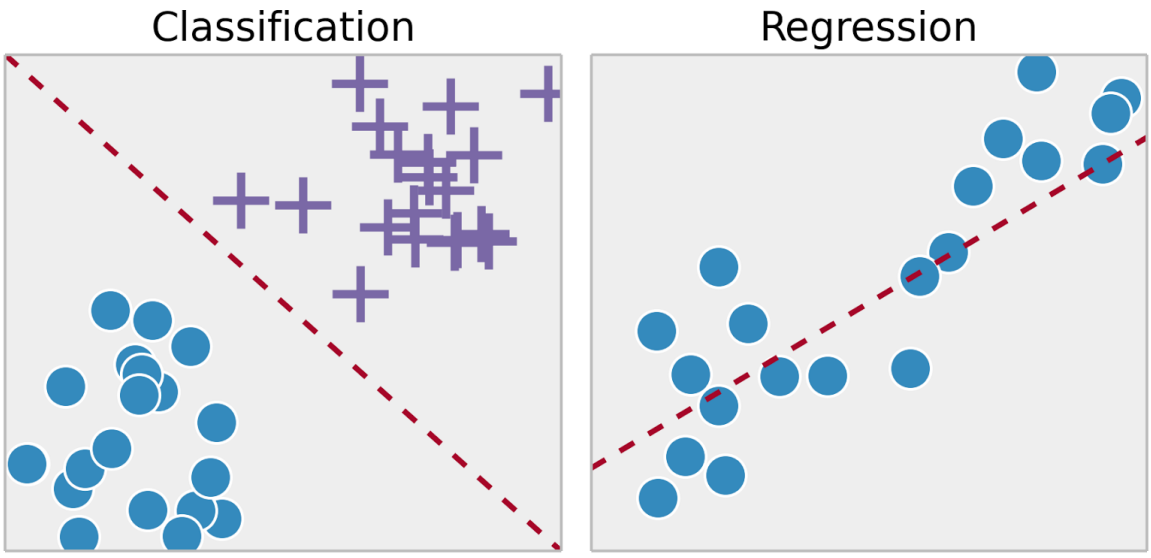
In this case, **X** is formed from the tuple slices *(bhp, fuel_type, displacement, weight, torque, suspension, car_age)* and each slice component is called a **feature**. Likewise, **y** is represented by the *car_price* and it's usually called **label** or **ground truth**. After learning from the provided corpus, a process that is commonly called **model fitting**, our model should be able to predict an approximation of the car price **ŷ** for new ages and specs **X**.



https://bigdata-madesimple.com/machine-learning-explained-understanding-supervised-unsupervised-and-reinforcement-learning/ [https://bigdata-madesimple.com/machine-learning-explained-understanding-supervised-unsupervised-and-reinforcement-learning/]

In the dataset of each **supervised** machine **learning** task, there will always be a **label y** that can be associated with a set of **features X**!

Because a **numerical** or **continuous** output is expected as a result, the aforementioned scenario is an example of a **regression task**. If y was a label that can take the values "cheap", "fair" or "expensive", then that would be called a **classification task** and y can also be referred to as a **class**.

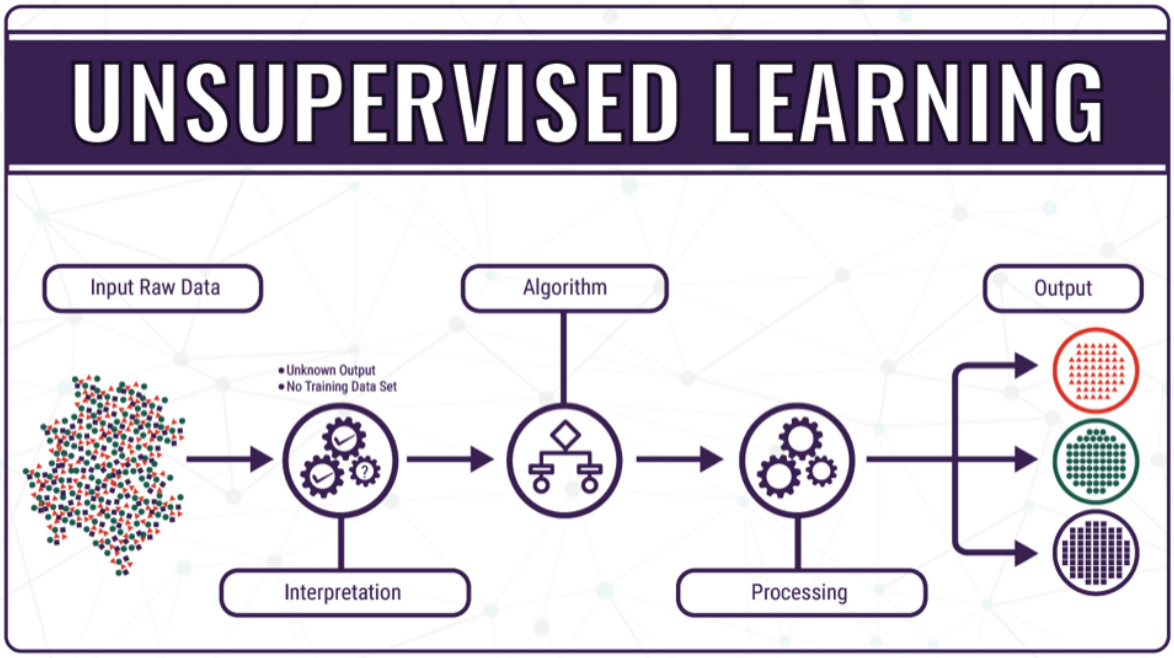


https://towardsdatascience.com/do-you-know-how-to-choose-the-right-machine-learning-algorithm-among-7-different-types-295d0b0c7f60 [https://towardsdatascience.com/do-you-know-how-to-choose-the-right-machine-learning-algorithm-among-7-different-types-295d0b0c7f60]

Two examples of **supervised learning** tasks are **regression** and **classification**!

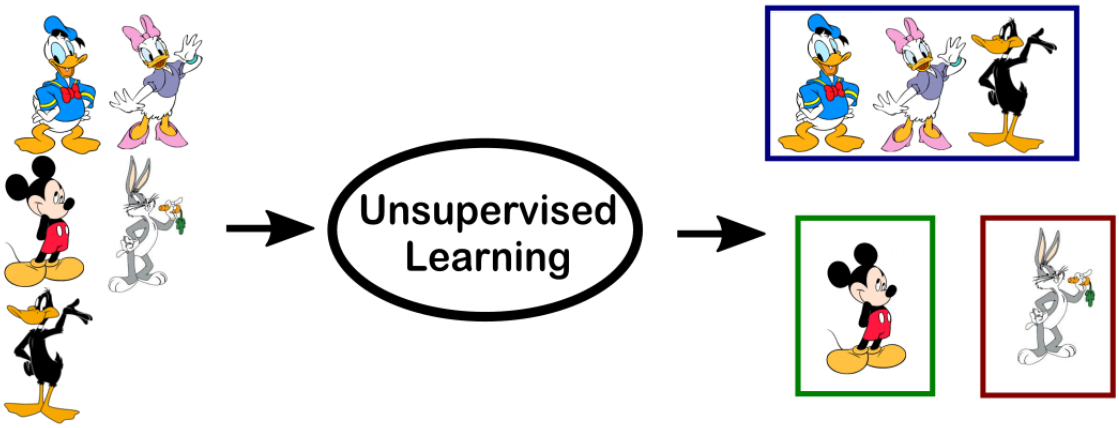
B. Unsupervised Learning

In the case of **unsupervised learning**, the training data is not labeled anymore, so the machine learning algorithm must take decisions without having a ground truth as reference. That means that your dataset is no longer made out of (X, y) pairs, but only of X entries. Consequently, the job of the model would be to provide meaningful **insights** about those X entries, by finding various **patterns** in the data.



https://medium.com/mlrecipies/supervised-learning-v-s-unsupervised-learning-using-machine-learning-98de75200415 [https://medium.com/mlrecipies/supervised-learning-v-s-unsupervised-learning-using-machine-learning-98de75200415]

The most common tasks for this learning approach are **clustering algorithms**. In these type of problems, the model must learn how to group various data together, forming **components** or **clusters**. Say that having a table with information about all the Facebook users, you would like to recommend personalized ads for every single person. This sounds like a very difficult task, because there is a huge number of users on the platform. But what if we group users together based on their **traits** (or **features** as your Data Scientist within you might say)? Then, we can suggest the same ads bundle to a whole group of users, making the advertising process easier and cheaper.



https://towardsdatascience.com/introduction-to-machine-learning-for-beginners-eed6024fdb08 [https://towardsdatascience.com/introduction-to-machine-learning-for-beginners-eed6024fdb08]

In the dataset of each **unsupervised** machine **learning** task, there will exist **only X** entries used to identify **patterns** and **insights** about data.

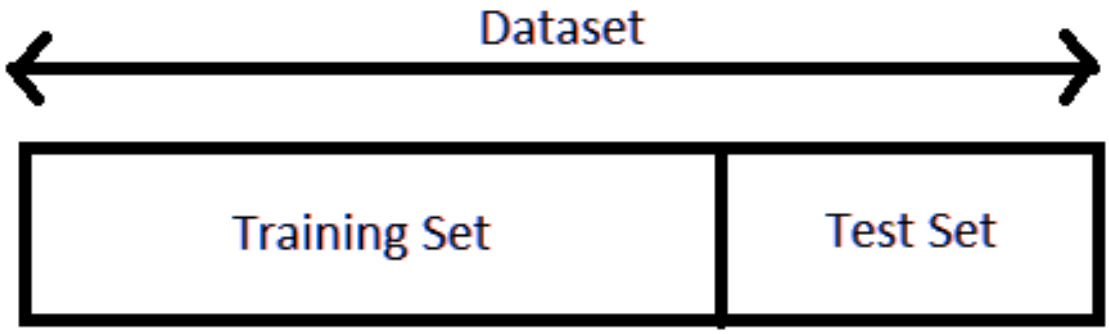
3. Performance Evaluation in Machine Learning

Generally, with machine learning algorithms, in the early phase of development, the focus is less on the computational resources (RAM, CPU, IO etc.) and more on the ability of the model to **generalize** well on the data. That means that our algorithm is trained well enough to provide accurate answers for data that has never been seen before (predict a car price for a new car posted on the platform, classify a new vehicle or assign a freshly registered Facebook user to a group). Hence, at first, we try to build a robust and accurate model and then we work towards making it function as computationally inexpensive as possible.

In machine learning, traditional evaluation metrics (RAM, CPU, IO) are not used as frequently!

A. Training & Test Sets

Evaluating machine learning algorithms is dependent on the type of problem we are trying to solve. But in most cases, we would want to split the data into a **training set** and a **test set**. As the name suggests, the former will be used to actually train the model and the later will be used to verify how well the model generalizes on unseen data. On the most common machine learning tasks, the split ratio ranges from **80-20** to **90-10**, depending on the size of the corpus. When a huge amount of data is operated (say millions or billions of entries), a smaller proportion of the dataset is used as a test set (even 1%) because the actual number of test entries are considered sufficiently numerous. For instance, 1% of 10M entries is 100k and that's generally regarded as **a lot of data** to test on.



https://data-flair.training/blogs/train-test-set-in-python-ml/ [https://data-flair.training/blogs/train-test-set-in-python-ml/]

In most problems, the data is split into a **training set** used in model training and a **test set** used in performance evaluation!

B. Classification Problems

Let's begin our performance analysis journey by analyzing a **classification problem**. Let's suppose that you have a model that was trained to predict whether a given image corresponds to a cat or not. As you already know, this is a **supervised learning** task in which the model must learn to **predict** one of **two classes** - "cat" or "non-cat". In this case, the model is also called a **binary classifier**.

http://www.british-shorthair-kitten.com/ [http://www.british-shorthair-kitten.com/]

Confusion Matrix

You fit the model with your training data, you ask it for predictions on the test set and now you have two results: the ground truth **y** that was part of your dataset and the predictions **ŷ** that have just been yielded by your machine learning algorithm. So how do you assess your model in this particular scenario? One way is to build a **confusion matrix**, like the one below:

Confusion Matrix

	Actually Positive (1)	Actually Negative (0)
Predicted Positive (1)	True Positives (TPs)	False Positives (FPs)
Predicted Negative (0)	False Negatives (FNs)	True Negatives (TNs)

https://glassboxmedicine.com/2019/02/17/measuring-performance-the-confusion-matrix/ [https://glassboxmedicine.com/2019/02/17/measuring-performance-the-confusion-matrix/]

Now don't get confused (excuse the pun 😉). It's actually quite simple to interpret it. There are **4 possible predictions** yielded by your binary classifier:

- **True Positives (y = ŷ and ŷ="cat"):**

The model identified a cat in the image and in that image it is actually a cat.

- **False Positives (y != ŷ and ŷ="cat"):**

The model identified a cat in the image, but there was no cat in the image.

- **False Negatives (y != ŷ and ŷ="non-cat"):**

The model did not recognize a cat in the image, but there was a cat in the image.

- **True Negatives (y = ŷ and ŷ="non-cat"):**

The model did not recognize a cat in the image and in the image there was no cat indeed.

The **first step** in analyzing the performance of a **classifier** is to build a **confusion matrix**!

These 4 scenarios can have different levels of importance, depending on the problem one wants to solve. For a smart antivirus, keeping the number of false negatives as low as possible is crucial, even if that means an increase in the false positives. Naturally, it's better to have annoying warnings than having your computer infected because the antivirus was unable to identify the threat. But if you don't necessarily have unusual requirements for you model and you just want to assess its generic performance, there are 3 metrics that can be inferred from this matrix:

Accuracy

This is the most intuitive metric and represents the number of correctly predicted labels over the number of total predictions, the actual value of prediction (positive or negative) being irrelevant. Again, judging the model by this single metric can be misleading. Consider the case of credit card transaction frauds. There could be a lot of transactions that are perfectly valid, but only a handful of them are frauds. A model that trains on this data could learn to predict that all of the transactions are safe and the ones that are fraudulent are just outliers. Calculating the accuracy on a dataset of 1 million transactions would yield 99.9%. But our model is useless, because letting 1000 frauds unnoticed cannot be allowed.

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision

The precision is the total number of correctly classified positive examples divided by the total number of predicted positive examples. If the precision is very high, the probability for our model of classifying non-cat images as cat images is quite low.

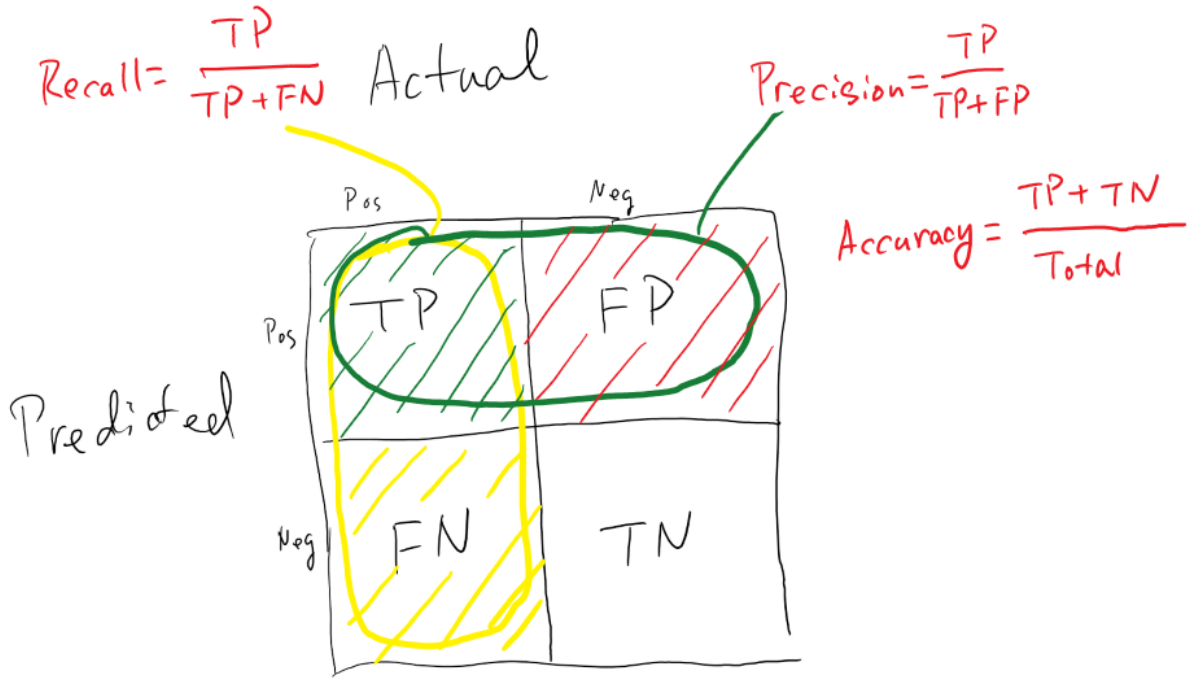
$$precision = \frac{TP}{TP + FP}$$

Recall

The recall is the total number of correctly classified positive examples divided by the total number of actual positive examples. If the recall is very high, the probability for our model of misclassifying cat images is quite low.

$$recall = \frac{TP}{TP + FN}$$

When you have **high recall** and **low precision**, most of the cat images are correctly recognized, but there are a lot of false positives. In contrast, when you have **low recall** and **high precision**, we miss a lot of cat images, but those predicted as cat images have a high probability of being indeed cat images and not something else.



https://towardsdatascience.com/understanding-confusion-matrix-a9ad42dcfd62 [https://towardsdatascience.com/understanding-confusion-matrix-a9ad42dcfd62]

F1 Score

Ideally, you would want to have both **high recall** and **high precision**, but that is not always possible. So you can choose the trade-off you are most comfortable with or you can combine the 2 metrics into 1, by using the **F1 score**, which is actually the harmonic mean of the 2:

$$F1 = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

This single metric is more generic, goes from **0** (worst) to **1** (best) and together with accuracy, can give you a solid intuition on the performance of your model.

From the **confusion matrix** you can extract the **accuracy**, **precision** and **recall**!

Generic Confusion Matrix

Don't take the terms "positive" and "negative" written in the confusion matrix above literally! They are actually placeholders for the 2 classes ("cat" and "non-cat") the model is trying to predict. This means that you can use more generic classes like "cat" and "dog" or "human" and "animal", depending on the problem you want to solve. As a consequence, **each metric** (accuracy, precision, recall or F1 score) is computed **per class** and if you want to obtain a **generic score** for the model, you must compute their **average**. For instance, if you have a recall of **0.6** for "**cat**" and **0.5** for "**dog**", the **average recall** of the model will be **0.55**.

Consequently, the confusion matrix can be generalized to more than 2 classes (see the image below), having the same metrics and ways of computing them.

		Predicted		
		Greyhound	Mastiff	Samoyed
Actual	Greyhound	P_{GG}	P_{MG}	P_{SG}
	Mastiff	P_{GM}	P_{MM}	P_{SM}
	Samoyed	P_{GS}	P_{MS}	P_{SS}

https://dev.to/overrideveloper/understanding-the-confusion-matrix-264i [https://dev.to/overrideveloper/understanding-the-confusion-matrix-264i]

⚠ [5p] Exercise 1

In this exercise, we are going to download the skeleton and setup the Python environment for this lab. For that, you should:

1. Download the **skeleton**: lab_12_ml_revisited.zip
2. Make sure **Python 3** is installed:

```
sudo apt-get update
sudo apt-get install python3.6
```

3. Make sure **pip** and **virtualenv** are installed:

```
sudo apt-get install python3-pip
sudo pip3 install virtualenv
```

4. **Create** a Python 3 virtual environment in the skeleton directory:

```
virtualenv venv
```

5. **Activate** your Python 3 virtual environment:

```
source venv/bin/activate
```

6. **Install** all the necessary dependencies specified in the *requirements.txt* file:

```
pip3 install -r requirements.txt
```

If you want to **deactivate** the virtual environment, simply type in this command:

```
deactivate
```

We usually code within a virtual environment because it's very simple to separate the dependencies between projects.

If you face an issue with **pip** or **matplotlib**, please see the following **troubleshooting**:

pip:

- **ImportError:** cannot import name 'main'

```
$ sudo su
$ pip install ...
```

- **AttributeError:** 'module' object has no attribute 'SSL_ST_INIT'

```
$ sudo python -m easy_install --upgrade pyOpenSSL
```

matplotlib:

- **ImportError:** No module named '_tkinter', please install the python3-tk package

```
$ sudo apt-get install python3-tk   # ignore errors
```

⚠⚠ **NON-DEMO TASK**

Run a command that validates the fact that you are using a virtual environment and one that validates the installation of the pip packages.

⚠ [30p] Exercise 2

In this exercise, you will learn how to properly evaluate a **classifier**. We chose a **decision tree** for this example, but feel free to explore other alternatives. You can find out more about decision trees here [https://medium.com/greyatom/decision-trees-a-simple-way-to-visualize-a-decision-dc506a403aeb]. For all the associated tasks, you will use the ***diabetes.csv*** file placed in the **resources** directory. The model must learn to determine whether the patient suffers from diabetes (**0** or **1**) by looking at a dataset consisted of the following **features**:

- Number of pregnancies
- Glucose level
- Blood pressure
- Skin thickness
- Insulin level
- Body Mass Index (BMI)
- Diabetes pedigree function (likelihood of diabetes based on family history)
- Age

The **solution** for this exercise should be written in the **TODO** sections marked in the ***classification.py*** file. Please follow the skeleton code and understand what it does. To run the code, uncomment **perform_classification()** in ***app.py***.

This exercise has **multiple tasks** spread throughout the sections below. Please read the tutorial carefully so that you won't miss any of these tasks. Remember to always follow the ⚠ emoji!

⚠ [15p] Task 2.A

Evaluate the classifier by **manually** computing the **accuracy**, **precision**, **recall** and **F1 score**. These metrics are derived from the **confusion matrix** but you don't have to build this matrix yourself. You can use *Scikit-learn* for that (Confusion Matrix [https://lmgtfy.com/?q=scikit-learn+confusion+matrix]).

Before anything else, we must compute the **confusion matrix**. Luckily, the **metrics** package of the *Scikit-learn* library has just the right function for this task:

```
cm = confusion_matrix(y_test, y_pred)
cm[0][0], cm[1][1] = cm[1][1], cm[0][0]
```

Please note that in order to compute a binary confusion matrix that looks just like the ones used in the above diagrams, a small swap has to be performed.

For this exercise we simply identify the **TP**, **TN**, **FP**, **FN** terms on the **confusion matrix** and then compute the various metrics using their formulas. For instance, for the **precision**, we can write the following lines of code:

```
precision = cm[0][0] / (cm[0][0] + cm[0][1])
```

You should be able to compute the **accuracy**, **recall** and **F1 score** by yourself.

⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK A**.

⚠ [15p] Task 2.B

Evaluate the classifier using the **metrics** package from the *Scikit-learn* library. Again, **accuracy**, **precision**, **recall** and **F1 score** are required.

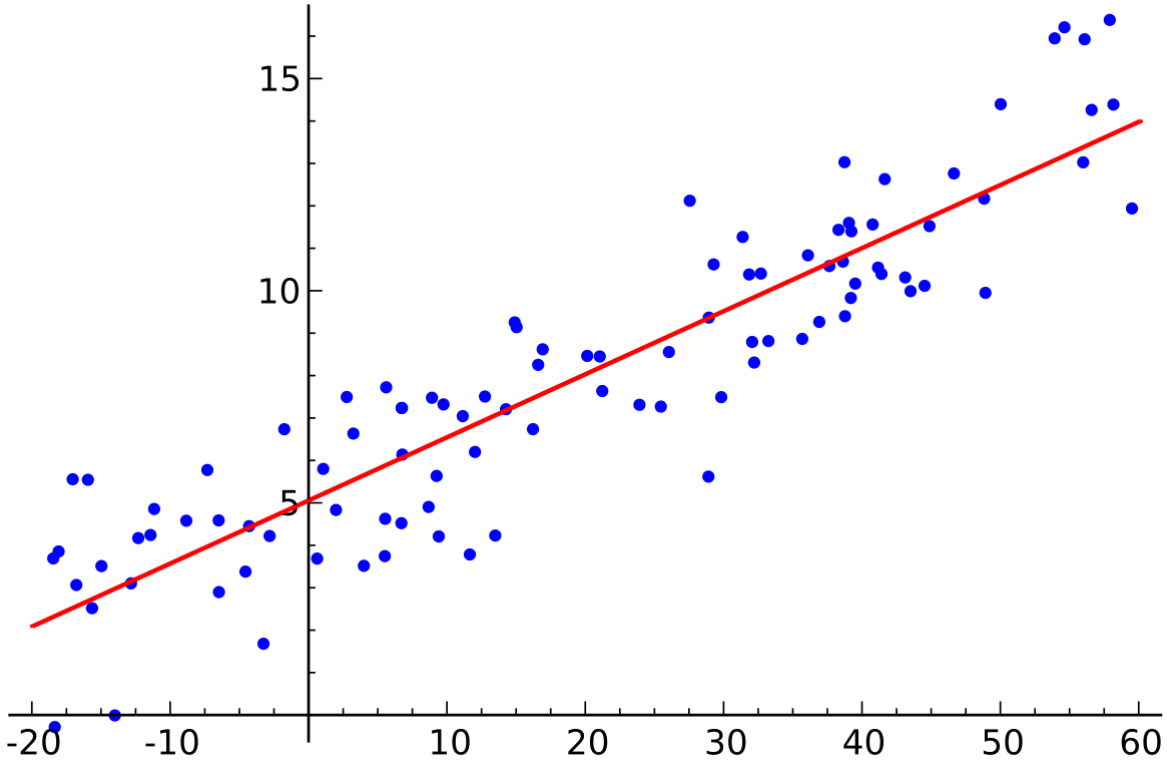
HINT: There are 2 *Scikit-learn* functions that will help you with this computation: **accuracy_score** and **precision_recall_fscore_support**. Because we are computing these metrics on a binary classifier, think about what is the suitable value for the **average** parameter of the **precision_recall_fscore_support** function.

⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK B**.

C. Regression Problems

As it was previously mentioned, **supervised machine learning** can also imply solving **regression tasks**, where a **numerical** or **continuous** value is used as a label. Let's take an example. Suppose that you want to predict the budget for an advertisement campaign, based on the revenue of the company. For this task, your model will learn from a training set of (X, y) pairs and will try to find the best $\hat{\mathbf{h}}$ that approximates the behaviour of the function **f(X) = y**. If we assume that the relationship between the two is **linear**, your model must learn how to **draw a line** that fits the points (X, y) the best. A visual representation can be seen in the figure below:



https://stackabuse.com/multiple-linear-regression-with-python/ [https://stackabuse.com/multiple-linear-regression-with-python/]

Root Mean Squared Error

But how do we know if that line is drawn correctly? Well, first we must define an **error** or **loss function** that can mathematically indicate us how far from the points the line has been drawn. In linear regression tasks, the most common approach is to use the **root mean squared error (RMSE)**, depicted below. You can sometimes get rid of the root and just compute the **MSE** to avoid extra computations.

$$RMSE = \sqrt{\frac{1}{n}\sum_{j=1}^{n}(y_j - \hat{y}_j)^2}$$

In this formula, $\mathbf{y_j}$ is the ground truth and $\mathbf{\hat{y}_j}$ is the result of the partially learned function $\mathbf{\hat{h}(X_j)}$. Basically, the model must apply successive corrections to $\hat{h}$, such that the predicted $\hat{y}_j$ values lead to a smaller RMSE. This process of minimizing the loss is also called **optimization** and is one of the foundational principles of machine learning. You can find more about it here [https://towardsdatascience.com/linear-regression-using-gradient-descent-97a6c8700931].

Similarly to the classification tasks, for regression problems, the value of the RMSE can be used as a **performance metric** for the model.

R^2 Correlation

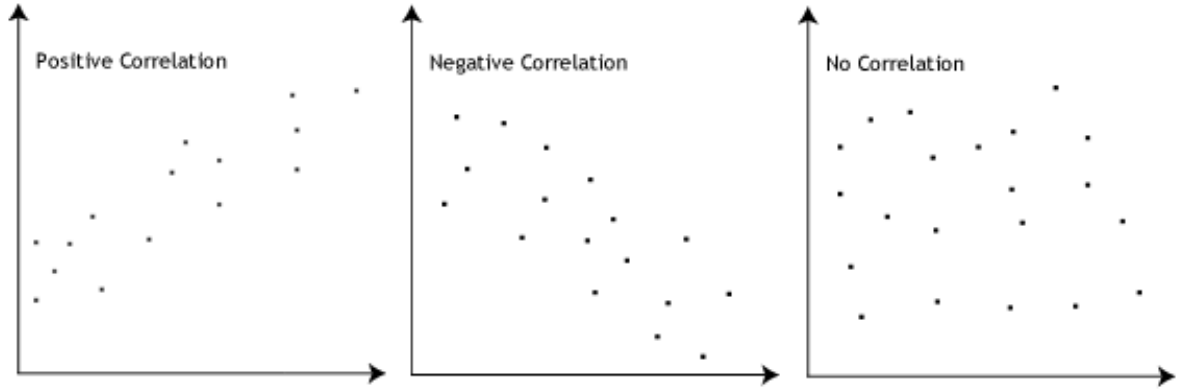
Now let's say that your model was properly trained and you have some predicted labels $\mathbf{\hat{y}}$ and some ground truth values $\mathbf{y}$. In the case of classification problems, with these two pieces of information you could immediately compute the F1 score or the accuracy. But with regression, a 0 to 1 score cannot be simply derived. One metric that can be used however, is the $\mathbf{R^2}$ **correlation** and it is computed using the following formula:

$$R^2 = 1 - \frac{\sum_{j=1}^{n} (y_j - \hat{y}_j)^2}{\sum_{j=1}^{n} (y_j - \bar{y})^2}$$

where $\hat{\mathbf{y}}_\mathbf{j}$ is the predicted value, $\bar{\mathbf{y}}$ is the mean of the ground truth labels and $\mathbf{y_j}$ is the ground truth. This score lies between **-∞** and **1** and has the following interpretation:

- **Close to 1:** high positive linear correlation between X and y
- **Close to 0:** a linear correlation between X and y cannot be identified
- **Close to -∞:** high negative linear correlation between X and y

These 3 scenarios are visually represented in the figures below:



https://statistics.laerd.com/stata-tutorials/pearsons-correlation-using-stata.php [https://statistics.laerd.com/stata-tutorials/pearsons-correlation-using-stata.php]

For **regression tasks** you can use **RMSE** and **R-squared score** as performance metrics!

⚠ [35p] Exercise 3

In this exercise, you will learn how to properly evaluate a **regression model**. We chose a **simple linear regressor** for this example, but feel free to explore other alternatives. You can find out more about linear regressors here [https://towardsdatascience.com/a-beginners-guide-to-linear-regression-in-python-with-scikit-learn-83a8f7ae2b4f]. For all the associated tasks, you will use the ***weather.csv*** file placed in the **resources** directory. The model must learn to determine what is the **maximum temperature** for a certain day (**y**) based on the **minimum temperature** (**X**).

The solution for this exercise should be written in the **TODO** sections marked in the ***regression.py*** file. Please follow the skeleton code and understand what it does. To run the code, uncomment **perform_regression()** in ***app.py***.

This exercise has **multiple tasks** spread throughout the sections below. Please read the tutorial carefully so that you won't miss any of these tasks. Remember to always follow the ⚠ emoji!

⚠ [10p] Task 3.A

Evaluate the regressor by **manually** computing the **Root Mean Squared Error (RMSE)**, **Mean Absolute Error (MAE)** and **R² score**.

For the **RMSE** we simply apply the formula above:

```
rmse = list(np.sqrt(np.sum((y_test - y_pred) ** 2) / len(y_test)))[0]
```

You should be able to compute **MAE** and **R² score** on your own!

We know that **MAE** was not covered in the first section of this laboratory class and that is why you must use your powerful Google [https://lmgtfy.com/?q=mean+absolute+error] skills to solve this one.

⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK A**.

⚠ [10p] Task 3.B

Evaluate the regressor using the **metrics** package from the *Scikit-learn* library. Again, **RMSE**, **MAE** and **R² score** are required.

You might not find a function that computes the actual **RMSE**, but the **MSE**. Can you do the **R**?

⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK B**.

⚠ [15p] Task 3.C

Train the model on variously sized chunks of the original dataset and notice how the **RMSE** changes. To better illustrate the behaviour, you should build a **plot** having the **data size** on the **X axis** and the **RMSE value** on the **Y axis**. Moreover, you should be able to **explain** the observed behaviour to the assistant.

Because this is rather a plotting task, we will do the hard work for you and compute the list of RMSE values for the various chunk sizes:

```
n = min(X.shape[0], max_chunk_size)
chunk_size = int((n - min_chunk_size) / chunks)

# Create 2 lists used in the plotting logic
size_list = []
rmse_list = []

# Train a model for each chunk
for i in range(0, chunks):

    # Compute the size of the current chunk
    size = min_chunk_size + (i + 1) * chunk_size
    size_list.append(size)

    # Select a chunk from the whole dataset
    sample_X = X.sample(n=size, random_state=42)
    sample_y = y.sample(n=size, random_state=42)

    # Split the data into a training set and a test set with a 80-20 ratio
    X_train, X_test, y_train, y_test = train_test_split(sample_X, sample_y, test_size=0.2, random_state=42)

    # Build a linear regressor
    regressor = LinearRegression()

    # Fit data from the training set to the regressor
    regressor.fit(X_train, y_train)

    # Make predictions on the test set
    y_pred = regressor.predict(X_test)

    # Compute the rmse
    rmse, _, _ = evaluate_regressor(y_test, y_pred, 'smart')
    rmse_list.append(rmse)
```

Please read the code and try to understand it by following the comments. Building the plot is on you!

Play with the following parameters and observe how the plot changes:

```
CHUNKS = 100
MIN_CHUNK_SIZE = 1000
MAX_CHUNK_SIZE = 1000000
```

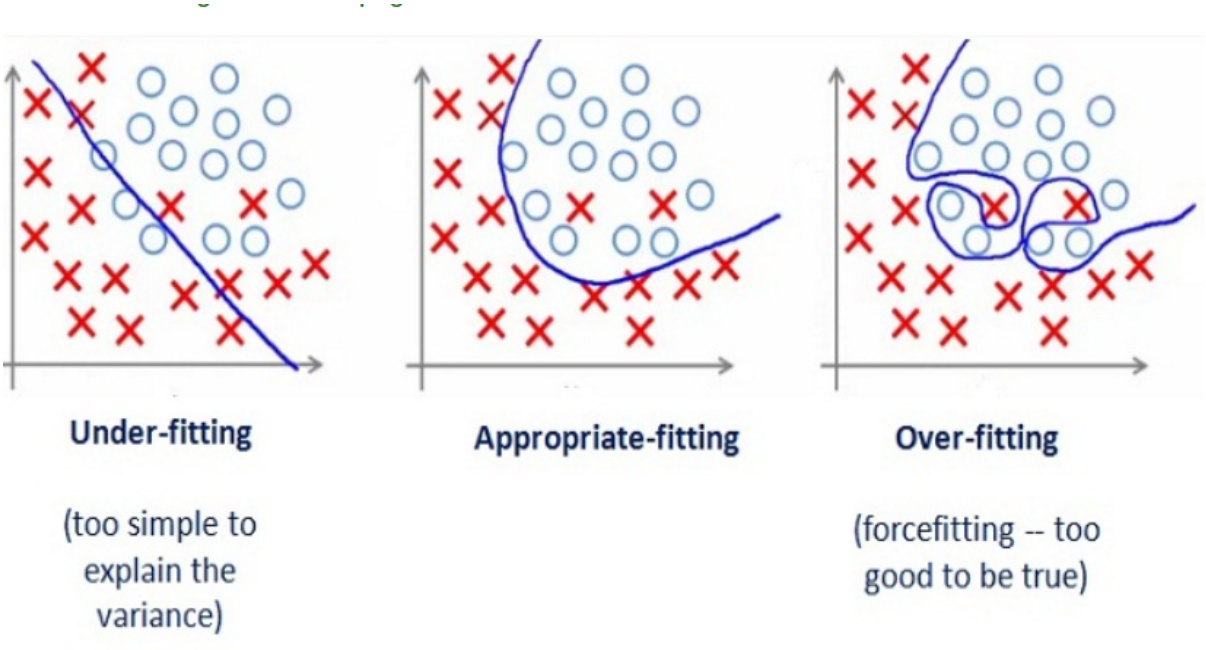
⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK C**. Make **at least 5** changes to the chunk-related parameters. That means that **5 plots should be generated**. Save each plot **in a separate file**.

D. Underfitting vs Overfitting

Splitting the data into a **training set** and a **test set** is not only helping us to determine the **accuracy** or **error** of the model's predictions, but can also give us an insight about its **behaviour** or **generalization capabilities**.

Let's take a trained binary classifier as an example and the accuracy as a performance metric. If it has a **small test score**, but a **high training score**, the model relied too much on the training data and is not able to generalize well on entries that has never seen before. In this case, we say that it has **overfit** the training data. In this particular case, the model is said to have a **high variance**. In contrast, if the model learned a function that is too generic, the problem of **underfitting** or **high bias** occurs. This time, the problem can be inferred from a **training score** that is **too small**. These situations were visually represented in the figures below:



https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76 [https://medium.com/greyatom/what-is-underfitting-and-overfitting-in-machine-learning-and-how-to-deal-with-it-6803a989c76]

The underlying causes of the aforementioned problems depend heavily on the model and how its **hyperparameters** were fine-tuned. Discussing them is outside the scope of this laboratory class, but one can learn more about those topics from these articles: Underfitting & Overfitting [https://towardsdatascience.com/what-are-overfitting-and-underfitting-in-machine-learning-a96b30864690], Hyperparameters [https://towardsdatascience.com/understanding-hyperparameters-and-its-optimisation-techniques-f0debba07568].

Besides looking at the usual **performance metrics**, one must also notice if the model **appropriately fits** the data!

⚠ [15p] Exercise 4

In this exercise, you will learn how to properly evaluate the **data fitting behaviour** of the **binary classifier** from **exercise 2**. For all the associated tasks, you will use the ***diabetes.csv*** file again, which is placed in the same **resources** directory. This time, **3 models** will be trained with different parameters and your job is to analyse their behaviour. For that, you will look at the **accuracy** of each model computed on both the **training set** and the **test set**.

The solution for this exercise should be written in the **TODO** sections marked in the ***fitting.py*** file. Please follow the skeleton code and understand what it does. To run the code, uncomment **perform_multiple_classifications()** in ***app.py***.

⚠ [10p] Task 4.A

For each model, make predictions on both the **training set** and **test set** and compute the corresponding **accuracy values**.

The model is already trained so you can directly use it to yield predictions on the two sets. And in order to evaluate these predictions, you can use the already familiar **evaluate_classifier** function from ***classification.py***.

⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK A**.

⚠ [5p] Task 4.B

Comment the results by specifying which is the **best model** in terms of fitting and which are the models that **overfit** or **underfit** the dataset.

⚠⚠ **NON-DEMO TASK**

Solve the tasks marked with **TODO - TASK B**.

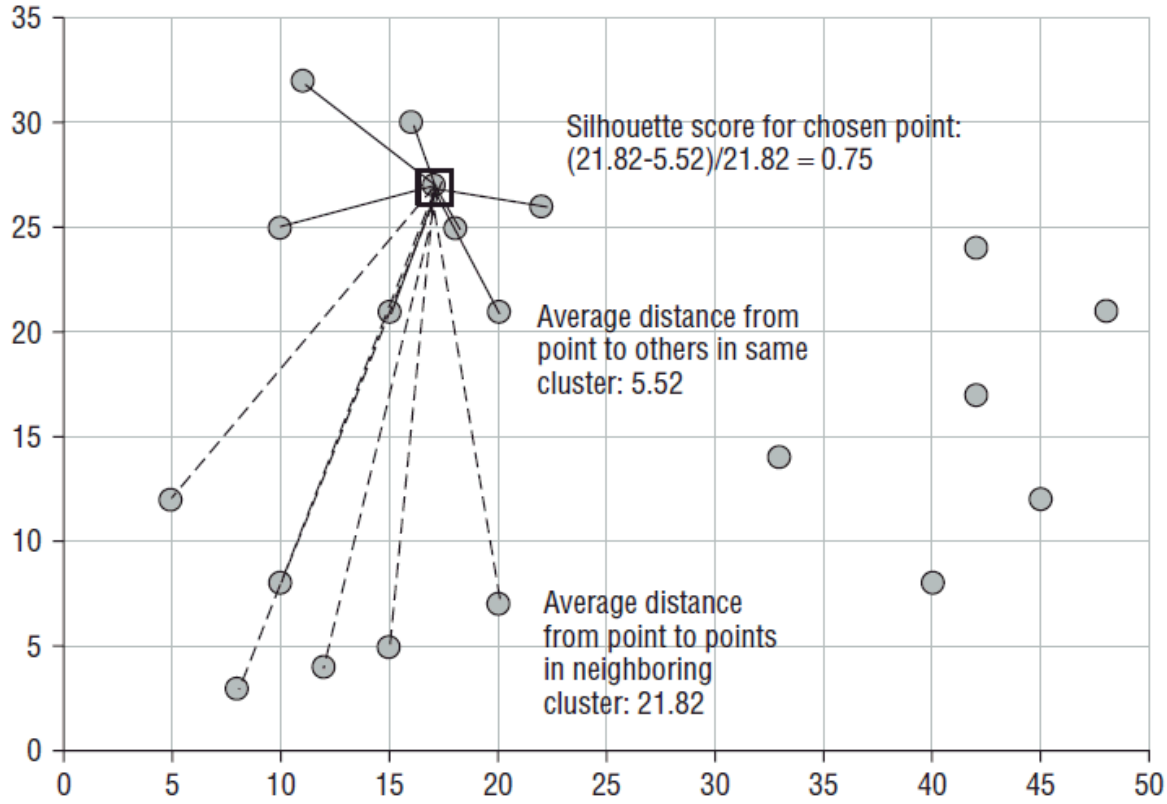
E. Clustering Algorithms

In the end, we should talk a little bit about **unsupervised learning**. As you already know, for such tasks, there is **no ground truth** - just a set of X values that might have some **underlying structure** or **pattern** that can be learned. Evaluating a model without having something as reference might seem a pretty difficult task. And in some scenarios, you might be right!

Say that we wanted to group images containing handwritten digits into **clusters** and at the end of the learning process, our model grouped the data like this:

https://towardsdatascience.com/graduating-in-gans-going-from-understanding-generative-adversarial-networks-to-running-your-own-39804c283399 [https://towardsdatascience.com/graduating-in-gans-going-from-understanding-generative-adversarial-networks-to-running-your-own-39804c283399]

At first glance, the clustering outcome looks good, but how can we express this "good-looking" result in a more formal manner? One solution is to measure how compact the clusters are, yet distant from one another, by computing a **silhouette score**. Because the formulas are too cumbersome to write in here, I will leave you a link [https://en.wikipedia.org/wiki/Silhouette_(clustering)], where everything is explained clearly. However, one might understand the concept by looking at this simple example:



http://blog.data-miners.com/2011/03/cluster-silhouettes.html [http://blog.data-miners.com/2011/03/cluster-silhouettes.html]

⚠ [15p] Exercise 5

In this exercise, you will learn how to properly evaluate a **clustering model**. We chose a **K-means clustering algorithm** for this example, but feel free to explore other alternatives. You can find out

more about K-means clustering algorithms here [https://towardsdatascience.com/understanding-k-means-clustering-in-machine-learning-6a6e67336aa1]. For all the associated tasks, you don't have to use any input file, because the clusters are generated in the skeleton. The model must learn how to group together **points in a 2D space**.

The solution for this exercise should be written in the **TODO** sections marked in the ***clustering.py*** file. Please follow the skeleton code and understand what it does. To run the code, uncomment **perform_clustering()** in ***app.py***.

⚠ [5p] Task 5.A

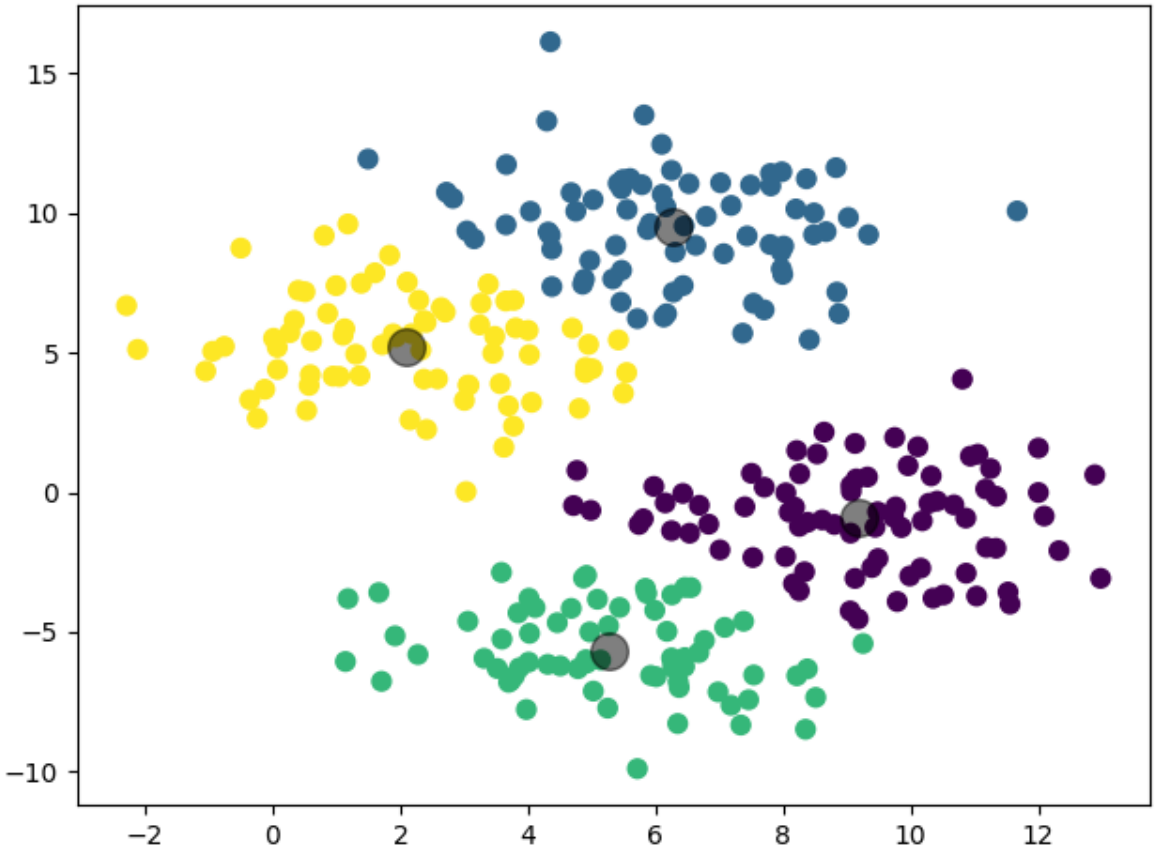
Compute the **silhouette score** of the model by using a *Scikit-learn* function found in the **metrics** package.

⚠⚠ **NON-DEMO TASK**

Solve the tasks marked with **TODO - TASK A**.

⚠ [10p] Task 5.B

Fetch the **centres of the clusters** (the model should already have them ready for you 🙂) and **plot** them together with a **colourful 2D representation** of the data groups. Your plot should look similar to the one below:



You can also play around with the **standard deviation** of the generated blobs and observe the different outcomes of the clustering algorithm:

```
CLUSTERS_STD = 2
```

You should be able to discuss these observations with the assistant.

HINT: The **plotting code** is very similar to the one found in the skeleton. You can also Google [https://lmgtfy.com/?q=plot+k+means+clusters+python] it out. 😉

⚠⚠ **NON-DEMO TASK**

Look at the hint above and solve the tasks marked with **TODO - TASK B**. Make **at least 3** changes to the standard deviation. That means that **3 plots should be generated**. Save each plot **in a separate file**.

⚠ [10p] Exercise 6

⚠⚠ **NON-DEMO TASK**

Please take a minute to fill in the **feedback form [https://forms.gle/k7FqUM16AcrkgKTJA]** for this lab.

References

Classification Dataset [https://www.kaggle.com/uciml/pima-indians-diabetes-database/data]

Regression dataset [https://towardsdatascience.com/a-beginners-guide-to-linear-regression-in-python-with-scikit-learn-83a8f7ae2b4f]

ep/labs/10.txt · Last modified: 2020/12/05 13:04 by vlad.stefanescu